CLASSIQ · QUANTUM OFFICE HOURS

Classiq 최근 토크시리즈 상세 학습 노트

모델링 · 합성 · 실행 · 해석
네 회의 다시보기와 실제 질의응답

새 플랫폼과 AI 에이전트, 금융 모델링과 NVIDIA CUDA-Q 연동을 원영상의 흐름에 따라 읽는다. 원리의 유도와 손계산, 용어집과 복습문제 풀이를 함께 수록했다.

책임 편집 김현중 · AI 보조 OpenAI Codex
기준일 2026-10-22 · 판본 1.1
발표 요약과 독립 보충 해설을 구분한 한국어 학습 자료

Classiq 최근 토크시리즈 상세 학습 노트

책임 편집 김현중 · AI 보조 OpenAI Codex · 근거 기준일 2026-10-22 · 판본 1.1

Classiq의 제품 중심 웨비나 **Quantum Office Hours** 세 회와 NVIDIA 공동 금융 웨비나 한 회를 한국어로 정리했다. 자연어로 문제를 기술하는 단계에서 Qmod 모델, 회로 합성, 시뮬레이션과 결과 해석으로 이어지는 과정을 설명한다. 핵심은 **모델을 만드는 편의, 합성된 회로의 비용, 최종 계산 결과의 유용성을 각각 확인해야 한다**는 것이다. 원영상의 발표와 실제 질의응답을 회차별로 읽고, 필요한 수학은 별도의 보충 해설과 풀이로 학습할 수 있다.

수록 회차와 다시보기

회차	행사일	주제	공식 다시보기
1	2026-10-21	Building Quantum Applications with the New Classiq Platform	새 플랫폼
2	2026-10-15	Building Practical Quantum Finance Applications with Classiq and NVIDIA CUDA-Q	CUDA-Q 금융 응용
3	2026-07-08	From Idea to Quantum Circuit with AI Agents	AI 에이전트
4	2026-08-27	Quantum Computing for Finance: From Use Case to Application	금융 문제의 모델링

가장 최근 발표 두 편을 먼저 배치하고, 그 다음 기초가 되는 두 편을 배치했다. 행사일은 공식 행사 페이지를 따른다. YouTube 업로드일과 행사일은 다르다. 7월·8월 세션은 10월에 다시보기로 업로드되었다. 공식 행사 페이지의 일부 요일·시간대 표기는 날짜와 맞지 않아, 여기서는 행사 날짜와 영상 내 재생시간을 사용한다.

읽는 순서

Classiq를 처음 접하면 3회 AI 에이전트 → 4회 금융 모델링 → 2회 CUDA-Q 금융 응용 → 1회 새 플랫폼 순서로 읽는다. 최근 기능부터 확인하려면 목차 순서대로 읽으면 된다. 수학이 필요한 부분에서는 원리와 계산 예제로 이동하고 용어집에서 영문 용어를 확인한다.

각 회차의 시간표시는 해당 원영상으로 이동한다. 원영상을 그대로 번역한 대본이 아니라 내용을 재구성한 학습 노트다. **실제 질의응답**은 영상에서 확인된 질문과 발표자의 답변을 요약한다. 질문 전문이 읽히지 않는 경우에는 답변으로부터 재구성한 질문이라는 사실을 표시한다. **보충 해설·학습 질문·후속 확인 질문**은 편집 과정에서 추가했으며 발표자의 발언으로 취급하지 않는다.

그림을 읽는 법. 발표 화면은 코드·결과·도표의 관련 부분을 실제 다시보기에서 발췌하고 영상 시각을 붙였다. 설명용 도식은 노트의 수학적 예제와 가정을 바탕으로 별도 제작했다. 캡션에서 두 종류를 구별하며, 본문에서 어느 부분을 읽고 무엇을 판단할지 설명한다. HTML에서는 그림을 누르면 원본 크기로 열 수 있다. PDF에도 같은 그림과 출처를 수록했다.

2·3·4회는 YouTube 영어 자동자막을 시간정보와 함께 검토했다. 1회는 공개 영어 자막의 다운로드가 비어 있어 영상 음성을 로컬 음성인식으로 전사해 확인했다. 자동자막과 음성인식은 Qmod·CUDA-Q·IQAE 등의 명칭, 숫자와 사람 이름을 틀릴 수 있다. 화면이나 공식 문서로 확인되지 않은 정밀 수치를 확정된 실험 결과로 옮기지 않았다. 여기서 설명하는 코드와 수학은 해당 시점의 자료이며 SDK의 최신 API와는 차이가 있을 수 있다.

이 시리즈에서 구별할 것

Qmod는 구현할 양자 연산과 데이터의 의미를 기술한다. 합성기는 그 의미를 만족하는 회로 구현을 찾는다. CUDA-Q는 양자 커널과 고전 연산을 연결하고 선택한 타깃에서 실행하는 데 쓰인다. CPU나 GPU에서 양자 회로를 시뮬레이션한 결과와 실제 양자 프로세서에서 측정한 결과는 실행 환경이 다르다. 개발 시간이 줄었다거나 GPU 시뮬레이션이 빨라졌다는 것만으로 최선의 고전 알고리즘보다 양자 계산이 우수하다고 결론 낼 수는 없다.

각 발표가 보여주는 기능과 독자가 확인해야 할 계산 비용을 함께 읽는다. 예를 들어 합성된 회로의 큐비트가 줄었다면 게이트 수나 깊이는 어떻게 변했는지, 하이브리드 반복이 빨라졌다면 JIT 컴파일·전송·샷·고전 최적화 중 어느 비용이 줄었는지 따져본다. 이 관점은 금융 밖의 화학·최적화 응용에도 적용할 수 있다.

학습 자료

- 원리와 손계산 예제: 가역 계산, 위상 오라클, QAOA, 진폭 추정, 전체 실행 비용.
- 한영 용어집: 회차에 필요한 용어의 정의와 구별.
- 학습 질문과 완전 풀이: 원영상 질문과 별도로 만든 복습 문제.
- 출처와 작성·검증 공개: 회차별 자료와 확인 범위.

읽기용 파일

통합 HTML · 통합 PDF

편집 가능한 원본은 이 폴더의 Markdown 파일이다. 공개 원문·음성·자막 수집물과 QA 화면은 별도 작업 폴더에 보관하며, 읽기용 파일에는 포함하지 않는다. 이 노트는 Classiq의 공식 교재나 강사의 감수를 받은 자료가 아니다.

1.1판은 실제 발표 화면 13개와 독립 설명 도식 6개, 총 19개 그림을 본문의 해당 위치에 추가한 판본이다. 화면에서 더 명확하게 읽을 수 있는 QAOA 결과의 샷 수와 성공확률 분모도 함께 반영했다.

01. 새로운 Classiq 플랫폼으로 양자 응용 만들기

- 정식 제목: **Classiq Quantum Office Hours: Building Quantum Applications with the New Classiq Platform**
- 진행: Dr. Erik Garcell, Dr. John “Jack” Glennon
- 공식 다시보기: Classiq Technologies YouTube, 59분 36초
- YouTube 공개 시각: 2026년 10월 21일 13:19 PDT, 한국 시각 10월 22일 05:19. 이는 업로드 시각이며 행사 개최 시각과 구분한다.
- 정리 근거: 공식 영상 음성을 로컬에서 전사한 뒤 문맥을 검토했다. 이름·제품명·전문용어는 교정했으며, 아래 시각은 영상 시작점을 기준으로 한 대략적인 발언 위치다. YouTube 설명란에는 금융 사례를 다룬다는 문구도 있으나, 본 정리는 실제 영상에서 확인되는 시연 내용을 따른다.

이 세션에서 봐야 할 것

핵심은 자연어 요청으로 회로를 한 번 생성하는 데서 그치지 않고, **문제의 의미를 설명하고, Qmod 모델을 만들고, 합성된 회로를 살펴보고, 실행 결과를 원래 문제와 연결하는 개발 흐름**이다. 같은 간단한 Grover 탐색 예제를 이전 인터페이스와 새 인터페이스에서 비교함으로써 에이전트가 제공하는 설명·탐색·반복 작업의 차이를 보여 준다.

발표자는 Classiq를 양자·고전 개발자가 양자 알고리즘을 설계하고 디버깅하고 실행하는 기업용 개발 플랫폼으로 소개한다. 이 설명은 제품의 목표와 제공 기능에 대한 소개다. 작은 시뮬레이터 예제가 산업 문제에서의 양자 우위를 검증하는 실험은 아니다.

00:00-12:38 | 도입, 청중 조사와 에이전트 구조

무엇을 자동화하는가

07:16부터 설명하는 기본 흐름은 Qmod로 고수준 모델을 기술하고, 합성 엔진이 이를 회로로 변환하며, 시각화 도구로 확인한 뒤, 클라우드 또는 온프레미스의 게이트 기반 하드웨어에 실행하는 것이다.

여기서 Qmod와 합성 엔진의 역할을 구분해야 한다. 사용자는 어떤 계산·조건·알고리즘을 구현할지 기술하고, 합성 엔진이 그 의도를 구체적인 회로로 만든다. 에이전트는 이 앞뒤 과정에서 사용자의 요구를 정리하고, 도구를 호출하고, 결과를 설명한다.

07:52-08:41 발표자는 AI가 개발자의 시간을 구현 세부사항과 오류 수정에서 설계·알고리즘 선택으로 옮길 수 있다고 설명한다. 모르는 알고리즘을 조회하거나 현재 데이터에 어떤 방법이 적절한지 탐색하는 것 역시 에이전트의 사용 예로 든다. 이는 에이전트가 생성한 선택이 자동으로 최적이라는 뜻은 아니다. 선택의 이유와 전제조건을 확인하는 용도로 읽어야 한다.

웹과 로컬 개발 환경을 연결하는 공통 계층

08:47부터 웹 플랫폼과 사용자의 로컬 개발 환경이라는 두 접근 경로를 소개한다. 두 경로는 공통 AI 계층에 연결되고, 이 계층이 스킬·도구·MCP 서버를 통해 Classiq 문서와 알고리즘 예제 저장소에 접근한다.

09:21-10:55 소개의 중요한 기능은 다음과 같다.

스킬	화면에서 소개한 기능	개발자가 확인할 점
Modeling	Qmod 언어 또는 SDK로 양자 알고리즘을 설계한다.	입력 범위·목적함수·제약조건이 요구와 맞는가?

스킬	화면에서 소개한 기능	개발자가 확인할 점
Synthesis	제약과 선호를 반영해 Qmod 코드를 최적화한 양자 프로그램으로 컴파일한다.	어떤 제약과 최적화 목표를 주었는가?
Execution	시뮬레이터와 하드웨어에서 샘플링·관측량·상태벡터·변분 반복을 실행한다.	실행 백엔드와 결과의 종류가 목적에 맞는가?
Post-processing	히스토그램·에너지 곡선·확률이 큰 상태를 해석한다.	측정 결과가 원래 문제에서 무엇을 뜻하는가?
Analysis	게이트 수·깊이·하드웨어 적합성을 분석한다.	알고리즘의 답뿐 아니라 구현 비용도 적절한가?
Qiskit Translation	기존 Qiskit 코드를 Qmod 모델로 변환한다.	변환 전후의 의미가 보존되는가?

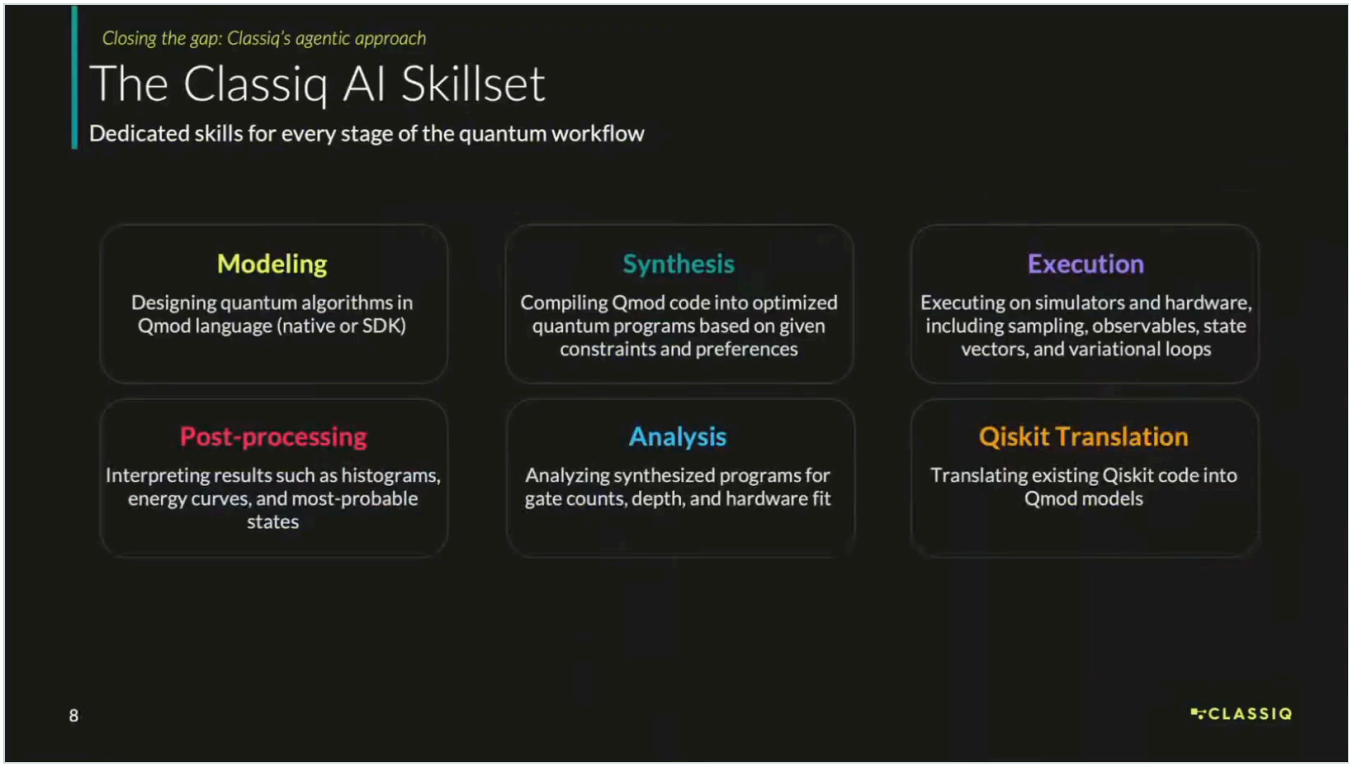


그림 1-1. **개발 단계별 AI 스킬 여섯 가지.** 위쪽은 모델 작성→합성→실행, 아래쪽은 결과 해석·회로 분석·코드 변환이다. 아래 행을 하나의 순차적 처리 순서로 읽을 필요는 없다. 원 슬라이드의 명칭을 위 한국어 표와 대응시킬 수 있다. 출처: Classiq Technologies, 09:26.

이 여섯 명칭과 역할은 약 09:26의 발표 화면을 함께 확인했다. Post-processing은 결과의 의미를 해석하는 기능이고, Analysis는 회로의 구조·자원을 분석하는 기능이라는 차이가 중요하다.

청중 조사

04:18-06:07 첫 설문은 양자컴퓨팅 경험을 묻는다. 진행자는 대략 절반이 이미 양자 프로그램을 만들어 본 적이 있다고 설명하며, 기초 학습자와 정기적으로 개발하는 사람도 함께 참석했다고 말한다. 이 설문은 시연의 설명 수준을 정하기 위한 현장 조사다.

11:16-12:26 두 번째 설문은 고전 코딩에서 AI 에이전트를 어떻게 쓰는지 묻는다. 진행자는 코드 작성·디버깅 보조가 가장 흔하고, 자동 완성·짧은 제안부터 전체 작업 흐름 위임까지 사용 방식이 다양하다고 설명한다. 정확한 막대그래프 수치는 음성에 나오지 않아 수치화하지 않았다.

12:44-18:12 | 시연 1: 곱이 6인 정수 쌍을 Grover 탐색으로 찾기

문제를 자연어로 정확히 고정한다

13:26의 요청은 다음 문제다.

> 0부터 7까지의 정수 x, y에 대해 x × y = 6을 만족하는 모든 순서쌍을 Grover 탐색으로 찾는다.

이 문장은 원문의 긴 인용이 아닌 문제의 한국어 재서술이다. 가능한 입력은 8 × 8 = 64개이고, 답은 (1, 6), (2, 3), (3, 2), (6, 1) 네 개다. x와 y를 각각 3비트로 표현하므로 탐색 레지스터는 6큐비트로 구성할 수 있다. 실제 합성 회로의 전체 큐비트 수에는 구현을 위한 보조 자원이 추가될 수 있으므로, 6을 전체 회로의 자원 수로 단정해서는 안 된다.

에이전트가 모델 작성 전에 계산한 것

13:56-14:40 에이전트는 탐색 공간, 정답 수와 반복 횟수를 설명한 후 Qmod 코드를 만든다. 시연은 작은 문제이므로 정답 네 개를 고전적으로도 바로 확인할 수 있다. 그 결과를 사용해 Grover 반복 횟수를 정한다.

15:59-16:16에 보이는 모델은 Grover 연산자를 세 번 적용한다. 학습을 위해 수식으로 확인하면, N = 64, M = 4일 때 $\sin^2\theta = M/N = 1/16$이고, k번 적용 후 이상적인 성공 확률은 $\sin^2((2k+1)\theta)$다. k = 3이면 약 96%가 된다. **이 확률 계산은 정리자의 보충 설명이며 발표자가 보여 준 측정 빈도를 대신하는 값이 아니다.**

작은 문제에서 미리 정답 수를 세는 절차를 대규모 미지의 문제에 그대로 적용할 수는 없다. 이 시연에서는 에이전트가 알고리즘의 매개변수와 설명을 함께 생성하는 과정을 학습하는 데 의미가 있다.

생성한 Qmod 모델을 읽는 순서

14:52-15:49 Jack은 x와 y를 묶은 사용자 정의 양자 자료구조, 양자 숫자, 조건을 판별하는 함수, 프로그램의 진입점인 main을 살핀다. x × y = 6이라는 조건은 고수준 산술식으로 표현한다. 개발자가 곱셈과 비교의 모든 게이트를 수작업으로 배치하는 과정은 시연하지 않는다.

이때 학습자가 확인해야 할 흐름은 다음과 같다.

1. 입력의 범위가 $x, y \in \{0, \cdots, 7\}$로 보존되는지 확인한다.
2. 조건 함수가 x × y = 6을 표시하는지 확인한다.
3. 균등한 초기 상태를 준비하는지 확인한다.
4. 조건을 표시하는 위상 오라클과 확산 연산을 결합한 Grover 연산자를 반복하는지 확인한다.
5. 측정 후 얻은 비트열을 x, y 값으로 해석한다.

회로를 한 단계씩 펼쳐 보는 이유

16:34-18:12 시각화 도구에서는 처음에 함수와 변수 중심의 고수준 회로가 보이고, 그 안을 열어 개별 큐비트와 더 낮은 수준의 연산을 살핀다.

Hadamard 변환은 각 큐비트의 Hadamard 게이트로 내려가 볼 수 있고, Grover 연산자는 위상 오라클과 확산 연산으로 나누어 볼 수 있다. 조건 함수도 더 펼치면 게이트 수준까지 확인할 수 있다. 확산 부분에서는 기저 변환과 0 상태에 대한 반사를 설명한다.

이 계층 구조의 의미는 회로가 복잡해질수록 모든 게이트를 한 화면에 놓고 해석하기 어려워진다는 데 있다. 잘못된 조건을 찾는 작업은 고수준에서, 특정 연산의 구현이나 비용을 보는 작업은 더 낮은 수준에서 수행할 수 있다.

실행 결과와 보고서를 읽는다

18:33-20:12 Jack은 실행 버튼을 누르고 히스토그램과 자료표를 확인한다. 네 정답 상태의 측정 확률이 각각 약 25% 수준으로 높아지고, 나머지는 낮아지는 모습을 설명한다. 이어 분석 보고서는 정답 여부, 각 답의 확률과 정답에 해당하는 측정 비율을 보여 준다. 발표 중 보고된 정답 측정 비율은 약 95%다.

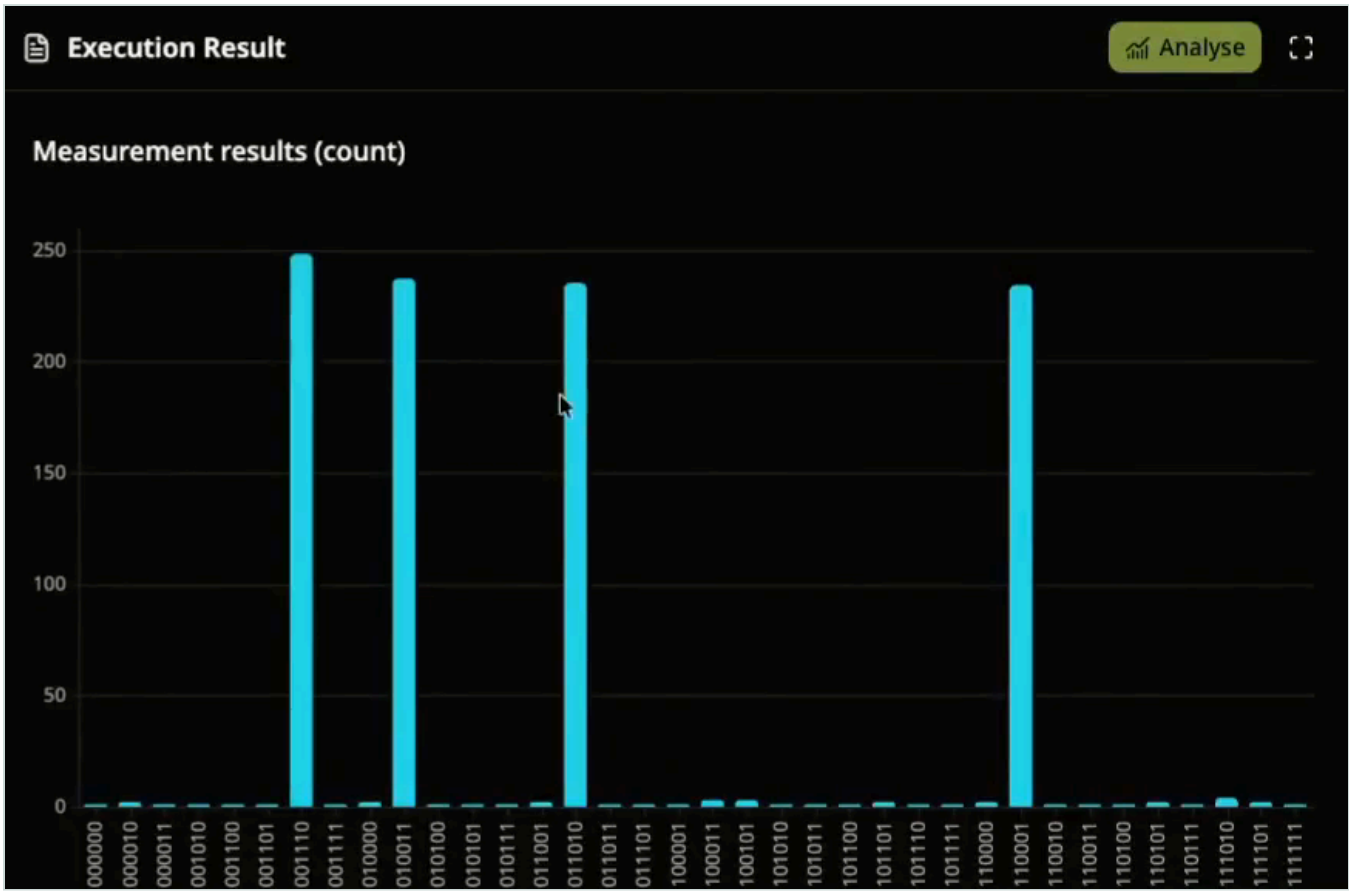


그림 1-2. **발표에서 실행한 Grover 예제의 측정 분포.** 가로축은 비트열, 세로축은 측정 횟수(count)다. 네 높은 막대가 곱이 6인 순서쌍에 대응한다. 확률로 읽으려면 총샷 수로 나누어야 하며, 이 그림은 별도의 이상적 확률 계산과 구별한다. 출처: Classiq Technologies, 18:55.

여기서 “각각 약 25%”는 반올림한 설명이다. 정확히 네 상태에 25%씩이고 나머지 상태에도 양의 확률이 있다는 뜻으로 읽으면 합이 100%를 넘는다. 상세 수치는 보고서의 개별 값으로 확인해야 한다. 시연의 포인트는 측정 비트열이 원래 문제의 x, y 쌍으로 연결된다는 점이다.

20:32-29:25 | 시연 2: 자산 5개 중 3개를 고르는 QAOA와 제약 보존 믹서

요구를 비즈니스 언어에서 수학적 모델로 옮긴다

20:47 Jack은 자산 5개, 기대 수익률과 공분산을 고려해 3개 자산으로 구성된 포트폴리오를 고르는 QAOA 문제를 보여 준다. 실행 시간이 걸려 미리 생성한 작업을 설명하는 방식이다. 따라서 화면에 보이는 모든 작업을 발표 중 실시간으로 수행한 것으로 읽으면 안 된다.

에이전트는 위험과 기대 수익률로 구성한 목적함수, 선택 개수 3이라는 카디널리티 제약, 이진 선택 변수 다섯 개와 예제 데이터를 제안한다. 21:23에는 목적함수를 최소화하기 때문에 기대 수익률 항에 음의 부호가 붙는다고 설명한다.

정리자의 수학적 표기로 쓰면 대표적인 형태는 다음과 같다.

- $x_i \in \{0, 1\}$: i번째 자산의 선택 여부

- Σ: 수익률 공분산 행렬, μ: 기대 수익률 벡터
- 목적: 위험 가중치 λ에 대해 $\lambda x^T \Sigma x - \mu^T x$를 최소화
- 제약: $\Sigma_i x_i = 3$

이 식은 발표의 위험·수익·선택 개수 관계를 이해하기 위한 표준화된 재서술이다. 영상 속 변수 이름이나 정규화 상수까지 전사한 원본 코드가 아니다.

첫 구현: 전체 비트열 위의 QAOA

21:59-24:11 첫 모델은 Hadamard 변환으로 비트열 전체의 균등 중첩을 만들고, 매개변수화한 비용층과 믹서층을 반복한다. 비용층에는 위험·수익 목적함수와 제약을 고려하는 로직을 넣는다. 시연에서 QAOA 층 수는 3이다.

Jack은 Qmod 코드의 비용 함수, 선택 개수를 세는 함수, 비용층, 믹서층, QAOA ansatz와 main을 따라간다. 합성된 회로 역시 함수 단위로 펼쳐 비용층과 믹서층의 3회 반복을 확인한다.

24:11-25:21 다음 요청은 매개변수를 최적화하고 수렴 곡선과 최적 포트폴리오의 측정 확률을 보고하라는 것이다. 에이전트는 비용의 기대값을 구할 Pauli 항을 만들고, 도중의 오류를 수정하며, 실행 결과를 모아 보고서를 만든다. 25:20의 화면 보고서는 전체 2,000샷 중 237샷(11.85%)이 선택 개수 제약을 만족하고, 보고서가 최적이라고 표시한 포트폴리오 `{0,2,4}`가 43샷(**2.15%**) 나왔다고 적고 있다.

오류를 스스로 수정했다는 점은 개발 편의의 사례다. 모든 오류가 자동으로 잡히거나 출력된 보고서가 독립적인 정답 증명이라는 뜻은 아니다.

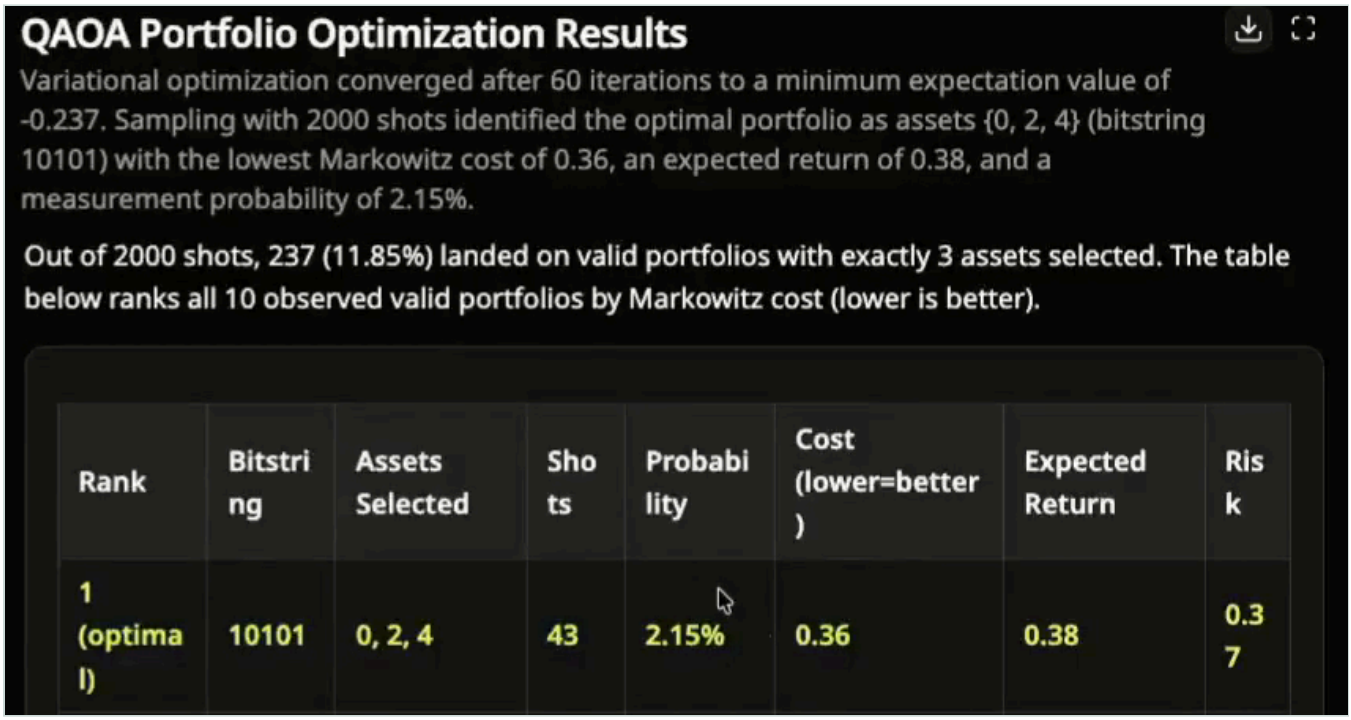
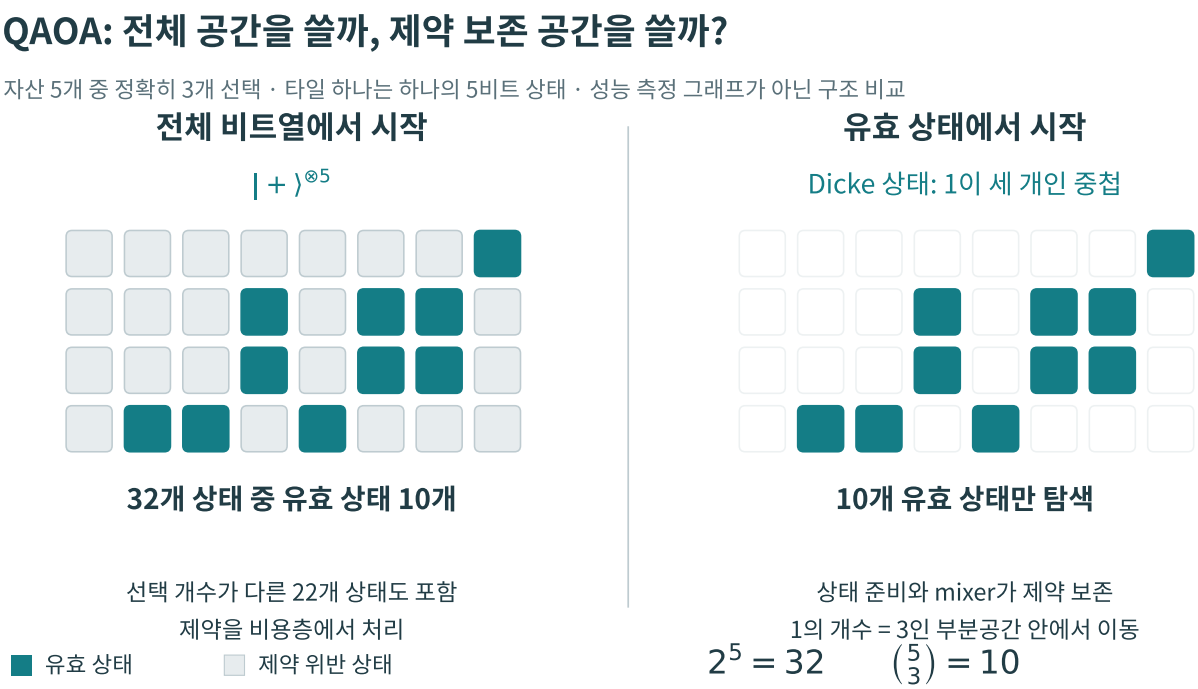


그림 1-3. **첫 QAOA 결과의 분모를 확인하는 원화면.** 위 문장에 전체 샷 2,000개와 유효 샷 237개가 명시돼 있고, 첫 행의 `10101`은 43샷·2.15%다. 유효 상태만 보여주는 표에서도 확률은 전체 샷 기준이라는 점을 확인할 수 있다. 원보고서의 최적성 주장을 이 노트가 독립 재실행한 것은 아니다. 출처: Classiq Technologies, 25:20.

두 번째 구현: 제약을 보존하는 부분공간만 탐색한다

25:38 Jack은 같은 문제를 다시 풀되 선택 개수 제약을 보존하는 믹서로 바꾸고, 정확히 3개 자산이 선택된 상태만 동일한 가중치로 중첩하도록 요청한다. 발표에서는 이를 **Grover mixer QAOA**라고 부른다.

26:13 생성된 코드에는 Dicke 상태를 준비하는 내장 함수가 등장한다. Dicke 상태는 여기서 “1이 정확히 세 개인 다섯 비트 상태들의 균등 중첩”으로 이해하면 된다. 가능한 상태 수는 조합 수 C(5, 3) = 10개다. 전체 다섯 비트 공간의 32개 상태와 구분된다.

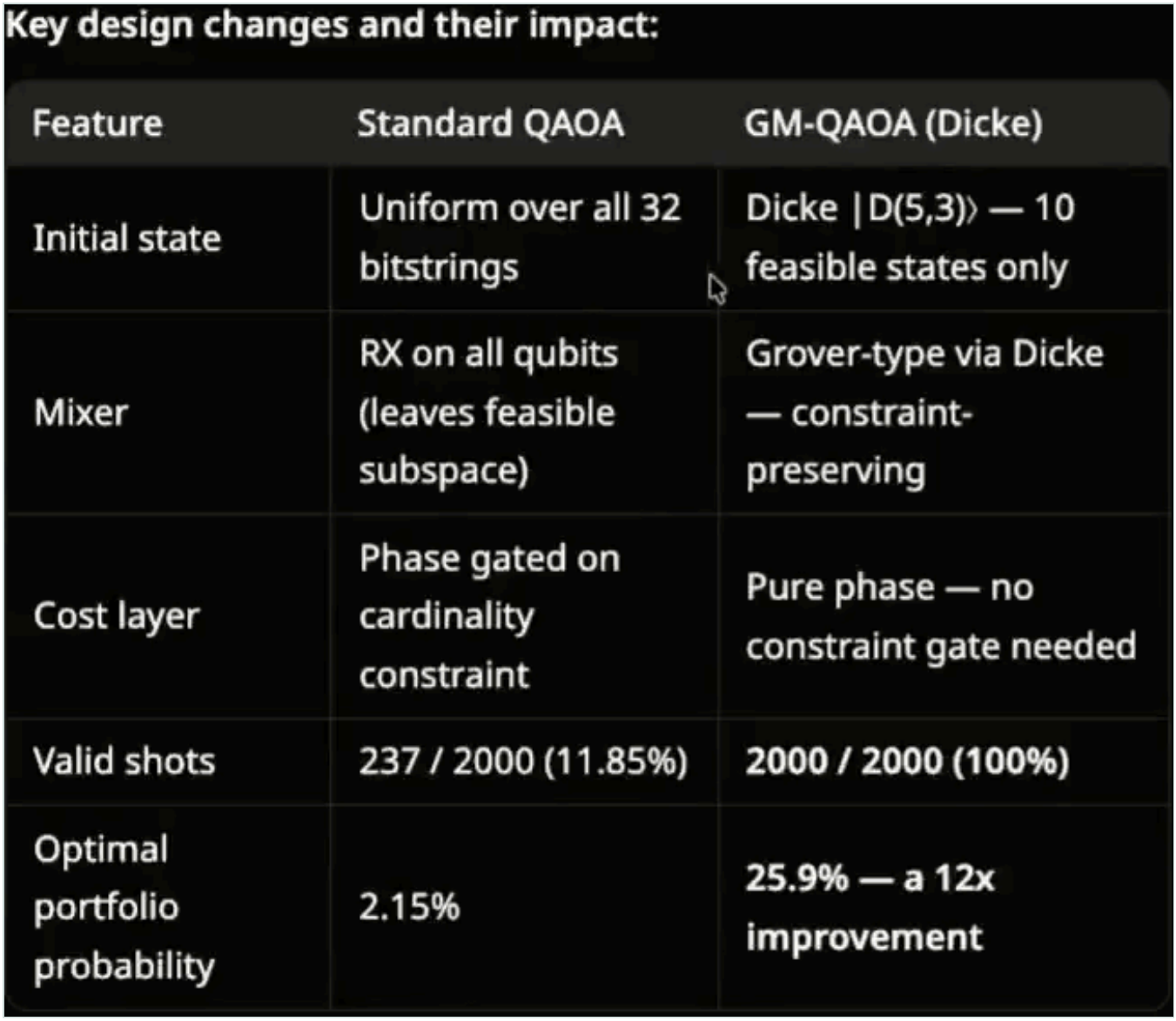


공간을 줄이는 설계가 준비·mixer 비용까지 없애는 것은 아니다. 실제 해 품질은 별도 검증한다.

그림 1-4. **전체 공간과 제약 보존 공간의 차이.** 타일 하나가 하나의 5비트 상태이며, 청록색 10개는 정확히 세 자산을 선택하는 상태다. 왼쪽은 나머지 22개 상태도 포함하고, 오른쪽은 Dicke 상태 준비와 제약 보존 믹서로 유효 공간 안에서 움직인다. 정리자가 32개 비트열을 열거해 만든 개념도이며 측정 분포를 그린 그림은 아니다.

비교점	시연의 첫 구현	Grover mixer를 사용한 구현
시작 상태	전체 비트열에 대한 균등 중첩	자산을 정확히 3개 고른 상태만의 균등 중첩
제약과 탐색 공간	유효하지 않은 선택 개수의 상태도 존재	믹서가 선택 개수를 보존하는 부분공간에서 움직이도록 설계
회로에서 확인한 변화	Hadamard 준비와 비용·믹서 반복	Dicke 상태 준비와 해당 부분공간의 믹서
전체 2,000샷 중 유효 샷	237샷, 11.85%	2,000샷, 100%
보고서에 표시된 최적 해의 측정 확률	43/2,000 = 2.15%	518/2,000 = 25.9%

27:17-29:25 두 번째 결과에서는 같은 포트폴리오의 측정 확률이 높아졌다고 설명하고, 3개 자산을 선택하는 유효 상태들의 측정 분포와 수렴 곡선을 비교한다. 28:00의 보고서는 유효 샷이 2,000/2,000, 최적 포트폴리오가 518/2,000(**25.9%**)라고 적고 있다. 첫 결과의 구두 근삿값 2%와 두 번째의 약 25%를 이 화면의 정확한 분모·표시값으로 보완했다.



Key design changes and their impact:

Feature	Standard QAOA	GM-QAOA (Dicke)
Initial state	Uniform over all 32 bitstrings	Dicke |D(5,3)⟩ — 10 feasible states only
Mixer	RX on all qubits (leaves feasible subspace)	Grover-type via Dicke — constraint-preserving
Cost layer	Phase gated on cardinality constraint	Pure phase — no constraint gate needed
Valid shots	237 / 2000 (11.85%)	**2000 / 2000 (100%)**
Optimal portfolio probability	2.15%	**25.9% — a 12x improvement**

그림 1-5. **어떤 변경이 어떤 비율과 함께 보고됐는가.** 첫 세 행은 시작 상태·믹서·비용층의 차이이며, 아래 두 행은 유효 샷과 최적 포트폴리오의 비율이다. 확률의 분모는 두 구현 모두 전체 2,000샷이다. 보고서의 약 12배라는 표현은 이 예제의 최적 포트폴리오 빈도 비율이며 실행시간 가속비가 아니다. 출처: Classiq Technologies, 28:00.

이 비교에서 얻을 학습 내용은 **제약을 비용층에서 처리하는 방식과 탐색 공간 자체에 반영하는 방식이 다르다**는 점이다. 원비교표는 첫 구현을 카디널리티 조건으로 비용 위상을 적용하는 방식으로, 두 번째를 제약 게이트가 필요 없는 순수 비용 위상으로 설명한다. 첫 구현에 일반적인 벌점 항이 쓰였다고 단정하지 않는다. 또한 전체 샷 중 유효한 해가 나오는 비율과 전체 샷 중 특정 최적 해가 나오는 비율은 다른 지표다. 단일 예제의 차이를 QAOA 일반의 개선율, 금융 수익 개선, 고전 최적화 대비 속도 향상으로 해석할 근거는 이 영상에 없다. 최적화 설정·시드·게이트 비용 등을 통제한 독립 벤치마크도 본 정리에서 재현하지 않았다.

보고서가 제품 기능으로 중요한 이유

28:12 Erik은 생성된 코드와 주석뿐 아니라 문제의 맥락·결과·의미를 묶은 보고서가 다른 팀이나 관리자에게 설명할 때 도움이 된다고 강조한다. Jack은 비교 그래프가 제약을 만족하는 상태를 기준으로 구성된 점을 짚는다. 이는 후처리가 단순한 그래프 그리기를 넘어 문제의 의미를 보존해야 한다는 사례다.

29:42-39:30 | 실제 청중 질문과 답변 1

아래는 영상에서 실제로 읽고 답한 질문을 한국어로 재구성한 것이다. 답변은 발표자의 설명이며, 별도로 계약·보안 인증·지원 목록을 검증한 결과와는 구분한다.

Q1. 어떤 LLM 플랫폼을 지원하는가? — 29:42

답변 장면 Erik은 Classiq 문서 사이트의 AI 검색에 질문을 넣어 찾는다. 화면에서 조회한 Quantum Engineering Plugin의 연결 환경은 **Claude Code, Cursor, Codex, Copilot**이라고 설명한다. 설치 방법과 MCP 서버 설정 문서로 이어지는 경로도 보여 준다.

여기서 답변이 제시한 목록은 개발 도구·코딩 환경의 이름이다. 이 발언만으로 지원하는 모든 기저 LLM 모델과 버전을 확인한 것은 아니다.

Q2. 기존 Classiq 사용자에게 새 플랫폼의 가장 큰 변화는 무엇인가? — 31:19

답변 장면 Erik은 두 층을 구분한다. 합성 엔진은 계속 개선하고 있으며, 최근에는 양자 신경망 등에서 효율 개선을 했다고 설명한다. 일상적인 사용에서는 AI 에이전트와 대화하면서 방법을 묻고, 코드를 작성·수정하고, 전체 응용의 초기 구현을 생성할 수 있다는 변화가 크다고 답한다.

Jack은 기존 플랫폼 기능이 유지되는 가운데 빠른 피드백과 브레인스토밍이 더해졌다고 설명한다. 따라서 새 기능은 초보자를 위한 자연어 입문 도구뿐 아니라 숙련자가 구현 초안을 만들고 방법을 비교하는 도구로도 제시된다. 영상의 이 대목에는 합성 속도 개선의 수치 비교는 나오지 않는다.

Q3. 기존 Qiskit 노트북을 가져와 Classiq 코드로 바꿀 수 있는가? — 34:10

답변 및 즉석 시연 Erik은 가능하다고 답하고, Grover 알고리즘을 다룬 Qiskit 온라인 문서·튜토리얼을 Qmod 버전으로 바꾸도록 요청한다. 에이전트가 문서를 가져오고 Qiskit 개념과 Classiq의 대응 개념을 표로 설명하며 코드와 변환 노트를 생성하는 과정을 보여 준다.

발표자는 수작업 CZ 게이트나 위상 오라클의 일부 구현을 Qmod의 고수준 구성으로 표현할 수 있다고 설명한다. 학습 포인트는 줄 단위 문법 치환보다 추상화 수준이 바뀐다는 점이다. 영상에서는 이 예제 변환의 설명과 생성 결과를 보여 준다. 임의의 모든 Qiskit 노트북이 수정 없이 변환된다는 보장은 이 시연만으로 만들 수 없다.

Q4. 금융기관은 외부로 데이터를 내보내기 어려운데 실제 협업에서 어떻게 처리하는가? — 37:13

답변 장면 Erik은 AWS 미국 동부의 인프라, 보안 조치와 인증 준수에 관해 설명한 뒤 다음 접근을 제시한다.

- 초기 검토에는 공개 예제·샘플 데이터로 시작한다.
- 실제 고객 데이터를 다루면 NDA와 고객의 데이터 공유 절차를 따른다.
- 고객이 사용하는 파일 공유 방식과 보안 요구에 맞춰 협업한다.
- 개발하거나 공동 개발한 결과의 지식재산은 최종 고객에게 속한다고 설명한다.
- Jack은 고객 정보의 성격을 유지하면서 민감한 내용은 드러내지 않도록 익명화하는 방법도 덧붙인다.

이는 현장 답변의 요약이다. 구체적인 인증의 번호·범위·유효기간, 계약별 소유권 조항, 데이터 보관·모델 학습 정책까지 영상에서 확인된 것으로 확대하지 않는다.

39:55-44:36 | 시연 3: 논문을 설명받고, 프로토콜을 코드로 확인한다

39:55 Erik은 새 논문을 만났을 때 에이전트에게 설명을 요청하는 자신의 사용 방식을 소개한다. 예제의 주제는 **암호화된 양자 정보의 복제(encrypted cloning)**다. 처음에는 매체·논문 소개를 언급하다가 Physical Review Letters 논문을 가리킨다고 정정한다. 발표 화면의 링크는 arXiv:2501.02757이며, 원문에서 제목과 저자를 확인했다. 논문은 Koji Yamaguchi와 Achim Kempf의 **Encrypted Qubits can be Cloned**, Physical Review Letters 136, 010801 (2026)이다. 논문 원문·초록

중요한 개념적 질문은 "양자 상태를 복제한다는 설명이 복제 불가능 정리와 어떻게 양립하는가?"다. 에이전트는 주장, 해결하려는 문제, 프로토콜, 복호화와 키의 역할, 관련 수학 및 응용을 설명한다. Erik은 설명 이후 해당 프로토콜을 Qmod로 구현하고, 암호화·복제·복호화 후 상태가 보존되는지 검증하고, 결과 보고서를 작성하도록 요청한다.

42:40-44:21 미리 반복 작업을 해 둔 결과로 이동해 주석이 달린 코드와 보고서를 보여 준다. 논문과 Classiq의 유니터리 표현 규약이 달라 변환이 필요했고 에이전트가 그 차이를 설명했다고 말한다. 여기서 학습할 점은 논문의 수식과 SDK의 표현이 겉모양까지 같아야 하는 것은 아니라는 점이다. 기저 순서·연산자 규약·위상·입출력 의미를 맞춰야 한다.

43:51 발표자가 읽는 보고서의 설명은, 한 암호화 사본을 복호화할 때 사용한 키 자원이 소모되어 다른 사본을 추가로 복호화할 수 없으므로 복제 불가능 정리와 일치한다는 것이다. 이 제한은 논문의 초록에서도 명시한다. 논문 초록 따라서 이 시연을 임의의 알려지지 않은 양자 상태를 독립적으로 사용 가능한 두 개의 원본으로 만드는 기술로 해석하면 안 된다.

이 부분은 논문을 학습하고 구현 초안을 만드는 개발 도구의 사례다. 본 정리에서는 원논문의 모든 정리·가정을 검증하거나 화면 속 구현이 논문 전체를 재현하는지 독립적으로 재실행하지 않았다.

44:39-50:42 | 시연 4: 로컬 개발 환경, 하드웨어 비교, 내결함성 자원 추정

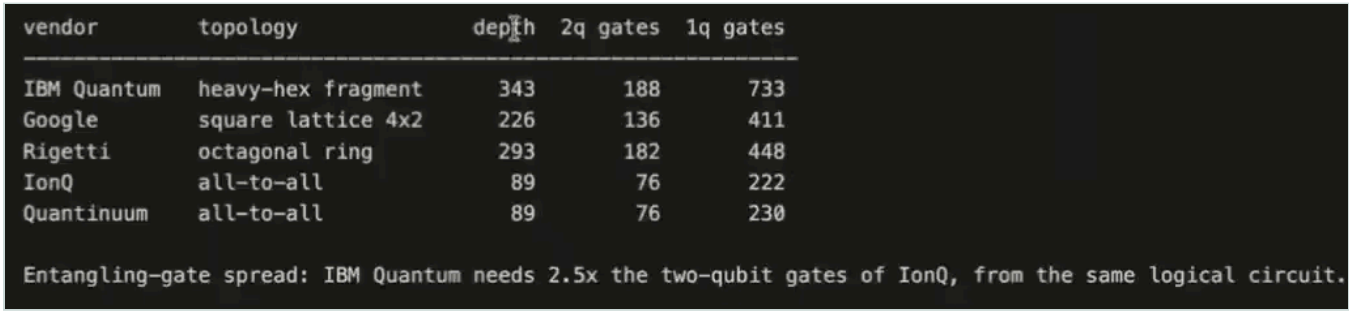
웹 인터페이스와 로컬 환경은 무엇이 다른가?

44:39 Erik은 웹과 로컬 개발 환경의 차이를 묻는 청중 질문을 받아 자신의 노트북과 Claude를 함께 사용하는 화면을 보여 준다. Classiq Quantum Engineering Plugin에서 설정·모델링·합성·분석·실행·후처리·Qiskit 변환 스킬과 MCP 연결을 불러오고, 같은 도구를 로컬 코드 작성에 사용하는 방식이다.

이 시연에서의 구분은 모델이나 알고리즘의 의미보다 개발 작업의 접점에 있다. 웹에서는 대화·모델·회로·결과를 플랫폼 안에서 함께 살피고, 로컬에서는 자신의 노트북·IDE와 코딩 에이전트를 중심으로 같은 Classiq 기능을 호출한다.

같은 해밀토니안을 다른 하드웨어에 합성한다

45:46-46:49 간단한 해밀토니안 예제로 회로를 합성하고, IBM·Google·Rigetti·IonQ·Quantinuum 등 서로 다른 게이트 집합과 구조를 갖는 대상으로 다시 합성한 결과를 비교한다. 깊이, 2큐비트 게이트 수, 1큐비트 게이트 수를 확인하고, 추가로 충실도와 실행 시간의 추정값을 반영한 비교 그래프를 만든다.



vendor	topology	depth	2q gates	1q gates
IBM Quantum	heavy-hex fragment	343	188	733
Google	square lattice 4x2	226	136	411
Rigetti	octagonal ring	293	182	448
IonQ	all-to-all	89	76	222
Quantinuum	all-to-all	89	76	230

Entangling-gate spread: IBM Quantum needs 2.5x the two-qubit gates of IonQ, from the same logical circuit.

그림 1-6. **하드웨어 조건이 바꾸는 합성 회로의 자원.** 먼저 `topology`를 읽고 `depth`, `2q gates`, `1q gates`를 비교한다. 원표의 수치는 이 모델·합성 설정의 결과이며 실제 QPU에서 측정한 정확도·실행시간의 순위는 아니다. 게이트의 기본 단위도 장치별로 다를 수 있다. 출처: Classiq Technologies, 46:30.

서로 다른 하드웨어가 같은 회로 파일을 그대로 실행한다는 설명보다, **같은 고수준 모델을 하드웨어 조건에 맞춰 다시 구현한다**는 설명이 더 정확하다. 장치별 연결 구조와 기본 게이트가 달라지면 깊이와 게이트 수가 달라질 수 있다.

발표자가 그래프에 보이는 실행 시간과 충실도를 설명할 때 “예상” 값을 포함한다고 말한다. 이는 여러 실제 QPU에서 동일한 날 동일 조건으로 측정한 실험 성적표와 구분해야 한다. 소프트웨어가 제시하는 추정치의 가정과 하드웨어 사양의 시점을 함께 확인해야 한다.

내결함성으로 옮기면 논리 회로와 물리 자원을 함께 본다

47:05-48:41 다음 요청은 같은 예제를 내결함성 버전으로 바꾸라는 것이다. 발표자는 Clifford+T 게이트 표현, 격자 수술(lattice surgery), 여러 코드 거리, 목표 오류 예산, 필요한 물리 큐비트와 실행 시간의 관계를 보여 준다. 논리적 관점에서 표면 코드 배치와 라우팅으로 내려가 보는 시각화도 소개한다.

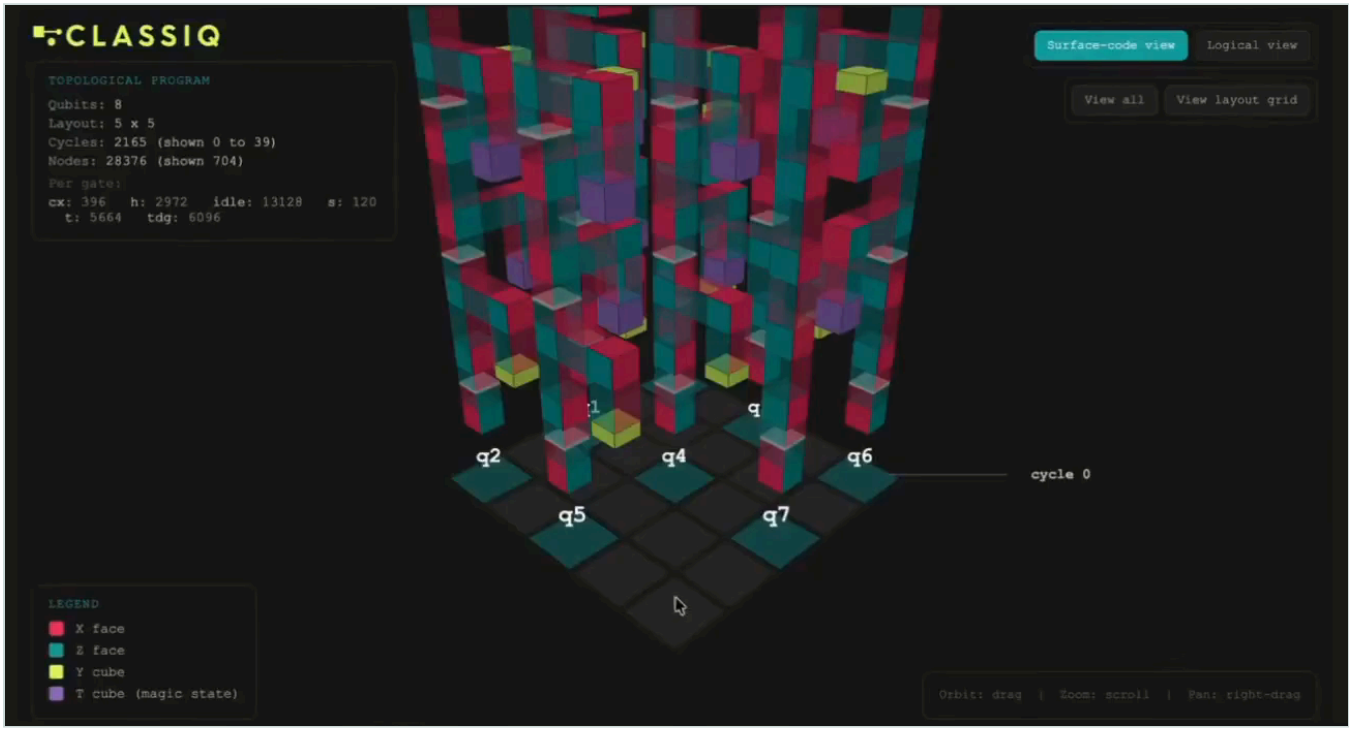


그림 1-7. **표면 코드의 공간 배치와 논리 시간을 함께 본다.** 수평면의 q0-q7은 패치의 위치이고, 위로 쌓인 방향은 논리적 시간이다. 왼쪽 요약은 전체 2,165주기 중 0-39주기를 표시한다고 적고 있다. 아래 범례의 X·Z 면과 Y·T 큐브는 프로그램의 시공간 표현이며 물리 장치의 사진이나 독립 측정 결과가 아니다. 출처: Classiq Technologies, 48:00.

그 뒤 문제 크기와 Trotter 단계 수를 늘릴 때 물리 큐비트 자원이 어떻게 커지는지 탐색한다. 이 부분의 의미는 “작은 논리 회로가 곧 적은 물리 큐비트”라는 가정을 피하고, 신뢰도 목표와 오류 정정 비용까지 포함해 응용의 자원을 계획한다는 것이다. 화면 속 특정 자원 수는 가정한 오류 모형·코드 거리·스케줄·회로 근사와 결합된 예제 값이다.

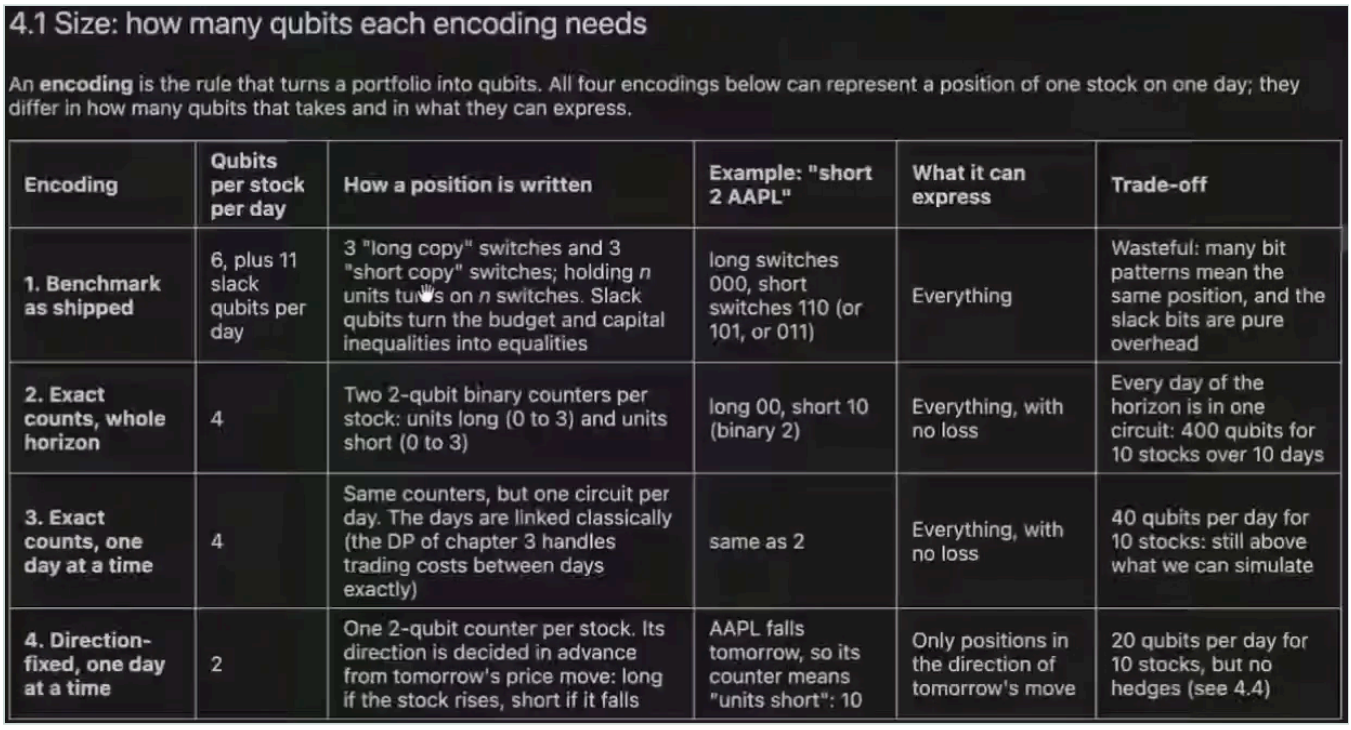
공개 벤치마크 문제를 축소할 때 무엇을 잃는가?

48:54-50:42 앞서 데이터 공유 질문에서 약속한 공개 데이터 예시로 Quantum Optimization Benchmarking Library를 소개한다. 거래 비용과 공매도를 포함한 다기간 포트폴리오 문제를 노트북으로 모델링하고, 위험·이익·공매도 수수료·현금 이자·거래 비용을 표현한 작업을 보여 준다.

그다음 원래 큰 데이터를 더 작은 문제로 줄이거나 다른 방법으로 인코딩하는 네 가지 접근을 비교한다. Erik은 가장 작게 줄인 한 방법에서는 헤지 선택 능력을 잃을 수 있어 반드시 최선의 방법은 아니라고 말한다. 이 장면에서 가장 중요한 질문은 “큐비트를 얼마나 줄였는가?”와 함께 **“축소하면서 원래 금융 문제의 어떤 자유도와 의미를 제거했는가?”**다.

50:10에서 읽기 좋게 보이는 원표의 네 접근을 정리하면 다음과 같다. 첫 행은 원래 벤치마크의 스위치식 표현이고, 아래 세 행은 다른 인코딩·처리 대안이다. 음성에서 정정하며 말한 큰 자원 수를 그대로 압축비로 옮기는 대신, 원표의 표현 단위와 범위를 따른다.

화면의 접근	표현과 처리 단위	10개 주식 예제에서 제시된 자원	의미와 교환 조건
Benchmark as shipped	주식·하루마다 롱 스위치 3개와 숏 스위치 3개, 별도로 하루당 slack 11큐비트	주식·하루당 6개에 하루별 slack 추가	같은 보유량을 여러 비트열로 표현할 수 있고 예산·자본 부등식을 등식으로 바꾸는 slack 공간 사용
Exact counts, whole horizon	주식·하루마다 4큐비트. 롱 보유 수 0-3과 숏 보유 수 0-3을 각각 2비트 이진 카운터로 표현	10일 전체를 함께 다루면 400큐비트	전체 시간 범위를 한꺼번에 표현
Exact counts, one day at a time	주식·하루마다 같은 4큐비트. 날짜 간 연결은 고전적 동적 계획법으로 처리	하루당 40큐비트	표에서는 바로 위 접근의 표현 내용을 유지한다고 설명하지만, 한 번에 양자 회로로 다루는 시간 범위가 다름
Direction-fixed, one day at a time	주식·하루마다 2큐비트. 표는 롱·숏 방향을 다음날 가격 움직임에 따라 미리 고정한다고 설명	하루당 20큐비트	방향 선택을 고정하여 헤지 자유도를 잃는다고 명시



4.1 Size: how many qubits each encoding needs

An **encoding** is the rule that turns a portfolio into qubits. All four encodings below can represent a position of one stock on one day; they differ in how many qubits that takes and in what they can express.

Encoding	Qubits per stock per day	How a position is written	Example: "short 2 AAPL"	What it can express	Trade-off
1. Benchmark as shipped	6, plus 11 slack qubits per day	3 "long copy" switches and 3 "short copy" switches; holding *n* units turns on *n* switches. Slack qubits turn the budget and capital inequalities into equalities	long switches 000, short switches 110 (or 101, or 011)	Everything	Wasteful: many bit patterns mean the same position, and the slack bits are pure overhead
2. Exact counts, whole horizon	4	Two 2-qubit binary counters per stock: units long (0 to 3) and units short (0 to 3)	long 00, short 10 (binary 2)	Everything, with no loss	Every day of the horizon is in one circuit: 400 qubits for 10 stocks over 10 days
3. Exact counts, one day at a time	4	Same counters, but one circuit per day. The days are linked classically (the DP of chapter 3 handles trading costs between days exactly)	same as 2	Everything, with no loss	40 qubits per day for 10 stocks: still above what we can simulate
4. Direction-fixed, one day at a time	2	One 2-qubit counter per stock. Its direction is decided in advance from tomorrow's price move: long if the stock rises, short if it falls	AAPL falls tomorrow, so its counter means "units short": 10	Only positions in the direction of tomorrow's move	20 qubits per day for 10 stocks, but no hedges (see 4.4)

그림 1-8. **큐비트 수의 분모와 표현 범위를 읽는 원표.** 두 번째 열은 주식 하나·하루 단위의 수이고 오른쪽 열은 10개 주식 예의 전체 기간 또는 하루 자원을 설명한다. 마지막 행은 롱·숏 방향을 미리 고정하므로 헤지 자유도를 잃는다. 위의 한국어 표는 이 원화면을 풀어 쓴 것이다. 출처: Classiq Technologies, 50:10.

따라서 400·40·20을 같은 문제·같은 시간 범위·같은 표현력을 유지한 단순 압축률로 비교하면 안 된다. 400은 전체 10일, 40과 20은 하루 단위이며, 마지막 방식은 선택 가능한 해의 의미도 바뀐다. 일별 처리에서는 고전 동적 계획법의 상태 저장·전이 계산 비용도 확인해야 하므로 회로 폭 감소가 전체 계산량 감소를 보장하지 않는다. 이 표는 발표자가 에이전트로 탐색한 설계 대안의 예이고, 정리자가 동일한 데이터로 다시 검증한 벤치마크는 아니다.

원표의 “다음날 가격 움직임”이 실행 시점에 만들 수 있는 예측인지, 이미 알려진 미래 가격을 이용한 예제 가정인지는 영상만으로 확정하지 못했다. 방향 고정 모델을 실제 의사결정에 연결하려면 이 입력 정보의 시점을 확인해야 한다. 또한 이 표는 인코딩의 상태 공간을 비교하므로 산술용 보조 공간 등을 포함한 최종 장치 요구량과는 구별한다.

50:58-55:14 | 시연 5: Qiskit 코드 변환 후 IBM과 IonQ 대상으로 다시 합성

51:32 Jack은 이번에는 온라인 문서 링크 대신 기존 Qiskit 코드를 직접 제공한다. 앞의 x × y 예제와는 다른 Grover 탐색이며 두 개의 표시 상태를 찾는다. 요청은 같은 논리를 Qmod로 재구현하라는 것이다.

에이전트는 찾는 상태, 오라클과 확산 연산, 반복 횟수, 사용할 고수준 함수 및 Qiskit과 Qmod의 대응을 설명한다. 52:26에는 비트 순서 차이와 조건 함수의 설계를 짚는다. 변환에서 비트열 표기의 순서를 잘못 옮기면 문법상 실행되어도 다른 상태를 표시할 수 있으므로, 이런 대응 설명이 중요하다.

52:56-53:19 합성 회로의 Hadamard 변환, Grover 연산자, 위상 오라클과 확산 연산을 확인하고 실행한다. 두 목표 상태의 측정 확률이 각각 50%에 가까워지는 결과를 설명한다.

53:34-54:49 이어 IBM 장치와 IonQ 장치용으로 각각 다시 합성해 회로 지표를 비교한다. 발표자가 설명한 결과는 다음과 같다.

- 회로 폭은 두 구현에서 같다.
- 기본 게이트 집합이 다르다.
- 연결 구조 차이와 함께 깊이가 달라지고, 이 예제에서는 IBM 쪽이 더 깊다.
- 보고서에서는 IonQ용 구현의 전체 게이트 수가 약 4배 적다고 설명한다.

이 역시 이 예제와 선택한 합성 조건의 회로 지표다. 게이트 하나의 의미·시간·오류율이 서로 다를 수 있으므로, 게이트 수의 비율을 그대로 실제 하드웨어 성능의 비율로 바꾸어 읽으면 안 된다. 실측 충실도·실행 시간·대기열·비용까지 판단하려면 추가 근거가 필요하다.

55:45-58:17 | 실제 청중 질문과 답변 2

Q5. Qiskit 코드를 옮길 때 고수준 언어가 garbage collection을 더 효율적으로 처리하는가? — 55:45

답변 장면 Erik은 Qmod의 지역 변수가 스코프를 벗어날 때 계산을 되돌려 0 상태로 만들고 큐비트를 재사용할 수 있도록 자동 처리한다고 설명한다. 보조 큐비트 관리도 고수준 모델의 이점으로 제시한다.

공식 문서로 보충한 정확한 의미: 양자 언컴퓨테이션은 중간 계산을 되돌리고 다른 큐비트와의 얽힘을 풀어 보조 자원을 재사용 가능하게 만드는 과정이다. Qmod의 자동 처리는 지역 변수 사용에 대한 규칙 아래 이루어지며, 규칙 위반은 컴파일 오류가 될 수 있다. 모든 임의의 양자 상태를 필요할 때 지워도 된다는 뜻은 아니다. Classiq 공식 Uncomputation 문서

따라서 기존 코드를 옮긴 뒤에는 단순히 출력 확률만 볼 것이 아니라, 중간 결과가 어떤 변수에 저장되고 언제 더는 필요하지 않은지도 함께 확인할 수 있다. 영상에서는 특정 원본 노트북의 변환 전후 자원 절감량을 측정한 별도 실험까지 제시하지는 않는다.

Q6. 다른 QPU를 벤치마크할 때마다 코드를 리팩터링해야 하는가? — 56:52

답변 장면 Erik은 “벤치마크”의 의미에 따라 다르다고 전제하고, 같은 해밀토니안 모델을 유지한 채 대상 백엔드를 바꿔 다시 합성할 수 있다고 답한다. 기본 게이트와 연결 구조를 반영해 다중 제어 연산 등 내부 구현을 선택한다고 설명한다.

이 답변에서 확인할 수 있는 것은 모델의 이식성과 하드웨어 조건을 반영한 합성이다. 실제 QPU에서의 동일 조건 실험 결과를 얻는 일과, 합성 회로의 깊이·게이트 수를 비교하는 일은 구분해야 한다.

도입 전 질문도 있었는가? — 02:25

도입 전 질의에는 결과를 고려한 합성과 독립적인 검증에 관련한 질문을 읽는 대목이 있다. 질문 전체가 음성에서 명확하게 복원되지 않아 완전한 질문 문장으로 확정하지 않는다. 진행자는 양자 커널과 CUDA-Q 연계, 동적 회로를 언급하며 더 깊은 논의는 행사 후 이어 갈 수 있다고 답한다. 이 짧은 답을 독립 검증 방법이 충분히 제시되었다는 결론으로 확대하지 않는다.

58:17-59:36 | 마무리

진행자는 내결함성 엔진과 관련한 IDC 보고서와 문의 경로를 안내하고, 현장에서 다루지 못한 질문은 행사 후 답하겠다고 말한다. 영상에 나오지 않은 사후 답변이나 채팅 전부는 확보하지 않았으므로 본 Q&A 목록에 포함하지 않았다. 보고서의 시장 전망 수치는 이 세션의 구현 원리를 이해하는 데 직접 필요하지 않아 여기서는 인용하지 않았다.

복습 질문 — 정리자가 만든 질문

아래 질문은 실제 청중 질문과 구분되는 학습용 질문이다.

1. x, y가 각각 0부터 7일 때 탐색 상태가 왜 64개인가? 6큐비트 탐색 레지스터와 합성 회로의 전체 큐비트 수는 왜 다를 수 있는가?
2. 정답 수를 아는 작은 Grover 예제와 정답 수를 모르는 실제 탐색 문제에서 반복 횟수를 정하는 어려움은 어떻게 다른가?
3. 자산 5개 중 정확히 3개를 고르는 유효 상태는 몇 개인가? 전체 상태 공간에서의 탐색과 제약 보존 부분공간에서의 탐색은 무엇이 다른가?
4. QAOA 최적 해 측정 확률이 2.15%에서 25.9%로 바뀌었다는 사실만으로 양자 우위를 주장할 수 없는 이유를 세 가지 들어 보라.
5. Qiskit 코드를 Qmod로 옮길 때 비트 순서, 오라클과 측정 해석 중 무엇을 검증해야 하는가?
6. 물리 큐비트 수를 추정할 때 논리 회로의 큐비트 수 외에 어떤 가정이 필요한가?
7. 금융 문제를 적은 큐비트로 줄이면서 헤지 능력을 잃었다면, 줄어든 자원 수와 모델의 충실성을 어떤 기준으로 비교해야 하는가?
8. 생성된 보고서가 그럴듯해도 사용자가 독립적으로 확인해야 할 항목은 무엇인가?

짧은 해설

1. 8 × 8 = 64개다. 보조 계산·언컴퓨테이션 등의 구현 자원이 추가될 수 있어 탐색 레지스터 6개와 전체 회로 폭은 다를 수 있다.
2. 정답 수를 모르면 회전각 θ와 가장 좋은 반복 횟수를 미리 정하기 어렵다. 다른 반복 횟수를 시도하거나 정답 수를 추정하는 절차의 비용을 포함해야 한다. 작은 문제에서 전수열거한 정답 수를 무료로 안다는 전제를 일반 문제로 옮기지 않는다.
3. 유효 상태는 C(5,3)=10개다. 전체 32개 상태에서 비용층으로 제약을 처리하는 것과 처음부터 10개 유효 상태의 공간을 보존하는 것은 준비·믹서·탐색 방식이 다르다.
4. 단일 인스턴스·두 구현의 결과이고, 시작 상태·믹서·비용층 등 여러 설계가 함께 바뀌었으며, 고전 기준선과 전체 실행 비용을 비교하지 않았다. 분모는 둘 다 2,000샷임을 원화면에서 확인했지만 최적화·시드·게이트 비용까지 통제한 일반적 비교가 되는 것은 아니다.
5. 입력 비트 순서, 목표 상태를 표시하는 조건, 오라클과 확산 연산, 반복 횟수, 측정 후 변수 해석을 확인한다. 작은 입력의 기준 분포와 비교하고 전역 위상과 관측 가능한 상대위상을 구별한다.
6. 물리 오류율, 목표 실패확률, 오류정정 코드와 코드 거리, 논리 게이트 분해, T 자원, 배치·라우팅·스케줄과 주기 시간이 필요하다. 선택한 가정과 회로 근사에 따른 추정값으로 기록한다.

7. 전체 기간과 하루 단위 계산을 구분하고, 고전 DP 비용과 고정한 롱·숏 방향이 만드는 목적값 손실을 함께 비교한다. 헤지 자유도가 다르면 원래 문제의 정확한 해와 축소 문제의 해를 같은 결과로 세지 않는다.
8. 요구의 수학적 번역, 제약조건, 실행 백엔드, 비교 기준, 성공 확률의 분모, 샷 수, 자원 수와 데이터 축소의 의미를 우선 확인한다. 보고서 자체가 주장하는 정확성을 작은 기준 계산이나 별도 검사로 대조한다.

02. NVIDIA × Classiq 금융 모델을 양자회로와 CUDA-Q 실행으로 연결하기

2026년 10월 15일 세션 · 다시보기 약 58분 48초 · 근거 확인 2026년 10월 22일

발표자는 Classiq의 Vincent van Wingerden과 NVIDIA의 Esperanza Cuenca Gomez다. 공식 행사 안내는 10월 15일 행사로 기록하며, Classiq 공식 다시보기는 10월 20일 한국 시각에 공개된 영상이다. 아래는 실제 시간 자막 전체와 시연 화면의 코드·출력·질문 슬라이드를 대조한 상세 노트다. 개념을 보완하거나 직접 재계산한 부분은 별도로 표시했다.

이 세션에서 배울 것은 금융 문제를 푸는 양자 알고리즘의 선택과 구현 책임을 나누는 방법이다. 분석가는 목적함수와 제약조건 또는 확률분포와 지급액을 정하고, Classiq는 이를 양자회로로 합성하며, CUDA-Q는 합성한 회로를 커널로 받아 반복 실행한다. 두 시연에서 실제 선택한 CUDA-Q 백엔드는 화면상 `qpp-cpu`다. GPU 가속은 이 실행 경로를 확장하는 기능과 별도 벤치마크로 설명되며, 이 영상의 노트북 출력을 GPU 측정값으로 읽으면 안 된다.

다시보기 길잡이

구간	실제 다룬 내용	확인할 핵심
00:03-03:08	발표자 소개, 참여자의 양자 경험 설문	입문자부터 구현 경험자까지 함께 보는 세션
03:12-06:43	NVIDIA의 양자·고전 이종 계산 구조와 CUDA-Q	NVIDIA는 큐비트 제작보다 가속 계산·실행 플랫폼을 설명
07:11-13:46	Classiq의 모델·합성·시각화·실행, AI 코딩 도우미	어떤 계산을 할지와 회로를 어떻게 만들지를 분리
14:01-18:13	포트폴리오 최적화의 수학적 구성	수익률, 공분산, 정수 배분량, 예산
19:00-21:17	중간 청중 질문 3건	비용·라이선스, SDK 연결, Classiq의 차이
21:18-30:44	Pyomo → QAOA → CUDA-Q 시연	3개 자산의 정수 배분 문제, 12큐비트 회로
30:49-32:36	금융 양자 도입의 출발점 설문	사업 문제 → 수학적 모델 → 알고리즘 순서
32:39-37:50	유럽형 콜옵션과 IQAE 소개	확률분포와 지급액을 결합한 기대값 추정
38:03-45:13	분포 준비 → 진폭 부호화 → 반복 추정 시연	6개 자산 레지스터 큐비트, 64개 격자점
45:21-48:55	GPU 활용, 확장 방향, 대학 교육·연구	기존 온프레미스·클라우드 GPU의 활용 제안
49:09-57:08	마지막 청중 질문 6건	알고리즘, 노이즈, 내부 구현, 컴파일러, 화학, 중성원자

1. 두 플랫폼은 계산 과정의 어느 부분을 맡는가

03:12부터 Esperanza는 양자컴퓨터를 고전컴퓨터와 분리된 독립 장치로 소개하지 않는다. CPU·GPU·QPU가 함께 작동하는 이종 계산 구조를 먼저 제시한다. 양자회로를 만들고 실행하며 결과를 평가하는 과정에 고전 계산이 계속 필요하기 때문이다. CUDA-Q는 Python과 C++에서 양자 커널을 표현하고, 선택한 QPU 또는 시뮬레이터에 실행하는 플랫폼으로 설명된다.

07:11부터 Vincent는 양자 프로그래밍의 어려움을 개별 게이트를 직접 연결하는 작업에서 찾는다. 자산 수, 표현 정밀도 또는 목표 하드웨어가 바뀔 때 회로 구현까지 다시 손보아야 한다면 금융 모델 연구의 반복 속도가 떨어진다. Classiq의 제안은 다음 네 단계다.

1. **모델 작성:** 계산하려는 수학적 기능을 높은 수준에서 기술한다. 예컨대 `QNum`은 양자 레지스터가 나타내는 수를 변수처럼 다루도록 한다.
2. **합성:** 모델과 목표 하드웨어·최적화 조건을 입력받아 게이트 회로를 만든다. 발표자는 연결성, 게이트 집합, 사용 가능한 보조 큐비트와 최적화 우선순위를 구현 결정의 예로 든다.
3. **시각화:** 합성된 프로그램을 열어 변수와 실제 게이트 수준의 구조를 확인한다.
4. **실행:** 연결된 실제 하드웨어 또는 CPU·GPU 시뮬레이터에 보낸다.

12:33부터 AI 코딩 도우미도 소개한다. 논문 구현이나 분자 바닥상태 계산 같은 요청으로 Classiq SDK 코드를 만들 수 있다는 설명이다. 발표자가 강조한 점은 사람이 높은 수준의 출력을 읽고 검토할 수 있다는 것이다. 이 기능의 정확도 평가나 AI가 만든 금융 모델의 정당성 검증은 세션에서 수행하지 않는다.

보충 해설: `QNum`을 실수형 변수처럼 쓸 수 있다는 설명은 양자컴퓨터가 무한 정밀도 실수를 저장한다는 뜻이 아니다. 실제 회로에서는 정해진 수의 큐비트와 부호·소수 비트 설정으로 유한한 수를 표현한다. 높은 수준의 표기가 표현 비용이나 수학적 가정을 없애지는 않는다.

2. 포트폴리오 예제는 무엇을 최적화하는가

14:01-18:13의 소개는 수익과 위험을 함께 고려하여 자산 배분을 고르는 문제다. 수익률이 큰 자산에만 몰아서 투자하면 위험과 공분산이 불리할 수 있다. 따라서 기대수익 벡터와 공분산 행렬을 함께 사용한다. 발표는 공매도와 추가 한도도 모델링할 수 있다고 설명하지만, 실제 시연은 공매도가 없는 작은 정수 배분 사례다.

21:47-24:58의 코드에서 입력을 다음처럼 고정한다. 숫자는 교육용 예시이며 시장 데이터나 퍼센트 단위 수익률로 제시되지 않는다.

항목	화면에서 확인한 값	뜻
자산 수	3	각 자산을 몇 단위 살지 결정
기대수익 벡터	`(3, 4, −1)`	자산별 수익 기여
배분량	`w_i ∈ {0,…,7}`	분할 매수 없는 정수 단위
예산	`w_1+w_2+w_3 ≤ 6`	세 자산의 단위 가격을 모두 1로 둔 제약
공분산 행렬	아래 식	자산별 분산과 자산 간 동반 변동
최소화 목표	`wᵀΣw − μᵀw`	위험에서 기대수익을 뺀 값

$$\mu = \begin{pmatrix} 3 \\ 4 \\ -1 \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} 0.9 & 0.5 & -0.7 \\ 0.5 & 0.9 & -0.2 \\ -0.7 & -0.2 & 0.9 \end{pmatrix}.$$

여기서 `w_i`는 자산별 보유량이고, $\mu^{\mathsf{T}}w$는 총 기대수익, $w^{\mathsf{T}}\Sigma w$는 포트폴리오 분산에 대응하는 위험 항이다. 대각항은 각 자산 자체의 변동을, 비대각항은 두 자산의 함께 움직이는 정도를 반영한다. 이 예제에서 위험 계수를 따로 두지 않고 두 항을 그대로 빼므로, 계수와 단위의 선택은 이미 고정된 모델 가정이다.

Pyomo의 정수 변수·제약조건·목적함수로 이 부분을 작성한다. 24:43의 강조점은 이 단계에 큐비트나 게이트가 전혀 나오지 않는다는 것이다. 재무·최적화 담당자가 이해하는 모델을 먼저 완성한 뒤 양자 알고리즘으로 연결한다.

```python
portfolio_model = pyo.ConcreteModel("portfolio_optimization")
num_assets = len(returns)

# setting the variables
portfolio_model.w = pyo.Var(range(num_assets), domain=pyo.Integers, bounds=(0, 7))

w_array = list(portfolio_model.w.values())

# global budget constraint
portfolio_model.budget_rule = pyo.Constraint(expr=(sum(w_array) <= total_budget))

# setting the expected return and risk
portfolio_model.expected_return = returns @ w_array
portfolio_model.risk = w_array @ covariances @ w_array

# setting the cost function to minimize
portfolio_model.cost = pyo.Objective(
    expr=portfolio_model.risk - portfolio_model.expected_return, sense=pyo.minimize
)
```

그림 2-1. **업무 조건을 먼저 고정하는 실제 발표 코드.** `bounds=(0,7)`은 정수 보유량의 범위이고, 합계의 `<= total_budget`은 예산 상한이다. 아래에서 위험−기대수익을 최소화한다. 원화면의 관련 코드만 발췌했다. 출처: Classiq·NVIDIA, 23:30.

3. Pyomo 모델은 어떻게 QAOA 회로가 되는가

25:02-27:43에서 `CombinatorialProblem`에 Pyomo 모델, QAOA 층 수 3, 제약 위반 벌점 10을 전달한다. 합성 후 시각화한 회로는 12큐비트다. 따라서 자산이 3개라는 사실과 전체 회로가 3큐비트라는 주장은 구별해야 한다. 이 시연은 자산마다 0 또는 1을 고르는 이진 선택보다 넓은 정수 표현을 쓰며, 합성 회로에는 제약 처리에 필요한 자원도 포함된다.

```python
NUM_LAYERS = 3
PENALTY    = 10  # constraint penalty — must match penalty_energy in the CUDA-Q

combi = CombinatorialProblem(pyo_model=portfolio_model, num_layers=NUM_LAYERS, penalty_factor=PENALTY)

qmod = combi.get_model(preferences=Preferences(standalone=True))
```

그림 2-2. **수학 모델을 QAOA로 연결하는 설정.** 층 수 3과 벌점 10이 코드에 명시된다. 원화면의 주석도 CUDA-Q에서 평가하는 해밀토니안의 벌점을 같게 유지하라고 강조한다. 출처: Classiq·NVIDIA, 25:05.

보충 해설: QAOA는 비용에 따라 상태의 위상을 바꾸는 연산과 상태를 섞는 연산을 번갈아 적용한다. 각 층의 회전 매개변수 γ, β를 고전 최적화기가 조정한다. 3개 층이면 이 시연에서 최적화할 매개변수는 6개다. 좋은 해가 측정될 가능성을 높이려는 근사 알고리즘이며, 층 수 3과 제한된 반복으로 전역 최적해가 보장되는 것은 아니다. QAOA 원 논문이 제시하는 알고리즘의 일반적 성격을 설명한 것으로, 발표에서 금융 우위가 증명되었다는 의미는 아니다.

28:04-29:50의 연결은 두 종류다. 목적함수를 CUDA-Q의 스핀 연산자로 바꾸고, 합성된 회로를 `qprog_to_cudaq_kernel`로 CUDA-Q 커널에 옮긴다. 화면의 주석은 두 경로에서 벌점 값을 일치시켜야 한다고 명시한다. 다른 벌점을 쓰면 최적화기가 평가하는 비용과 회로가 표현하는 비용이 달라질 수 있기 때문이다.

이어 `cudaq.observe`로 주어진 매개변수의 해밀토니안 기대값을 구하고, `cudaq.optimizers.COBYLA`로 매개변수를 갱신한다. 화면에서는 최대 반복을 60으로 둔다. NVIDIA의 실행 문서에 따르면 `observe`는 관측량의 기대값을 구하는 함수이고 `sample`은 측정 분포를 얻는 함수다. 이 차이가 뒤에 나오는 출력 해석에 필요하다.

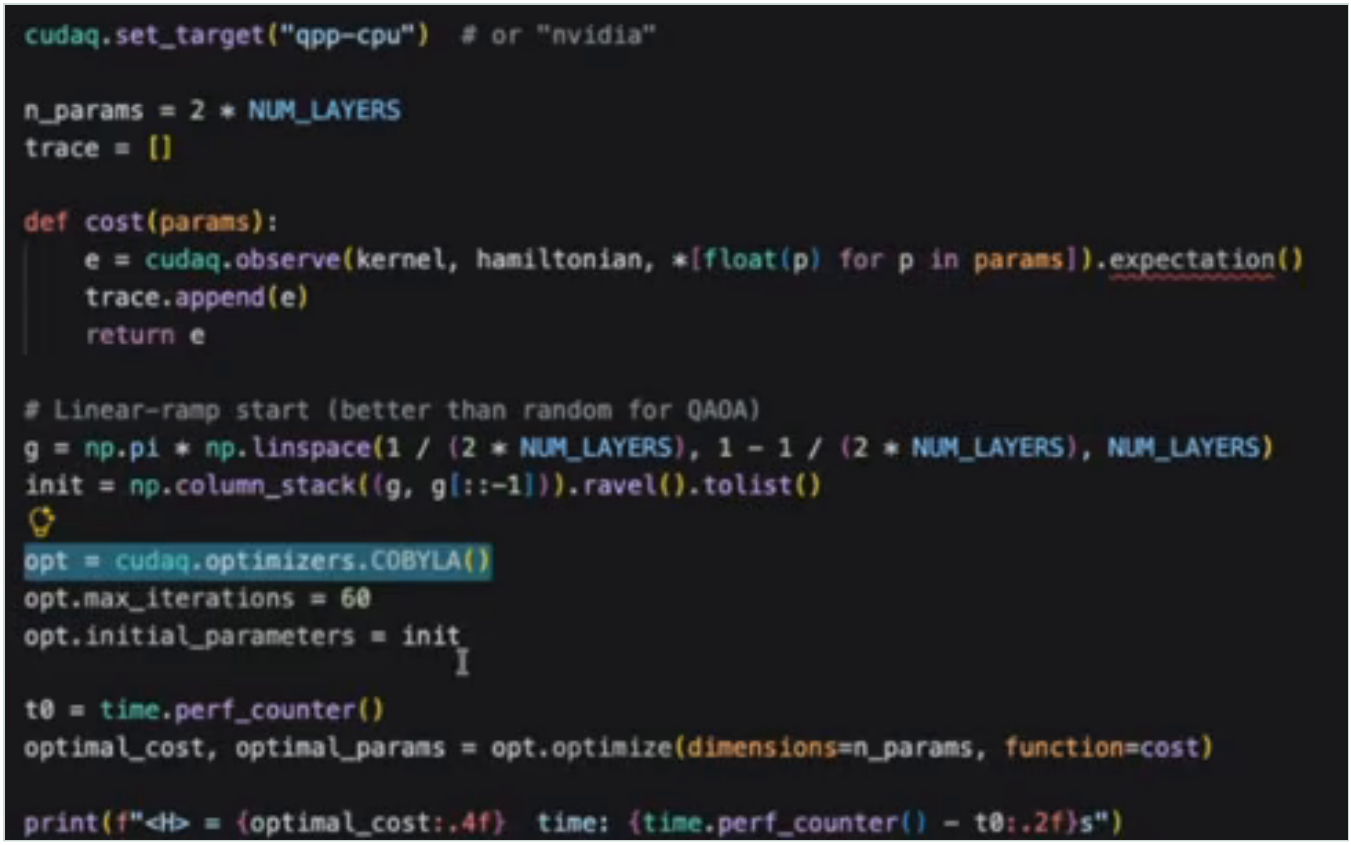


그림 2-3. **실제 데모의 실행 대상은 CPU 시뮬레이터다.** 맨 위의 `qpp-cpu`가 선택돼 있고 `nvidia`는 대안 주석이다. 가운데의 `observe` 호출을 고전 최적화기 COBYLA가 반복한다. GPU 가속 기능 소개와 이 노트북의 실제 실행 설정을 구별할 근거가 되는 화면이다. 출처: Classiq·NVIDIA, 29:25.

실제 출력과 읽는 방법

28:57의 코드 화면은 `cudaq.set_target("qpp-cpu")`를 선택하고 GPU용 `"nvidia"`를 주석으로 제시한다. 30:18의 출력은 다음과 같다.

화면의 값	기록	올바른 해석
최적화기가 반환한 `<H>`	662.9515	해밀토니안 기대값 출력
해당 최적화 호출 시간	7.80초	CPU CUDA-Q 경로에서 표시한 시간; 전체 작업 시간 아님
디코딩해 표시한 배분량	`[3,1,2]`	첫째 자산 3단위, 둘째 1단위, 셋째 2단위
그 배분량의 금융 목적함수	−4.6	위에 정의한 위험−수익 값
그 행의 표시 확률	0.0009765625	출력된 표본 행의 `prob`; 높은 성공확률의 근거로 쓰기 어려움

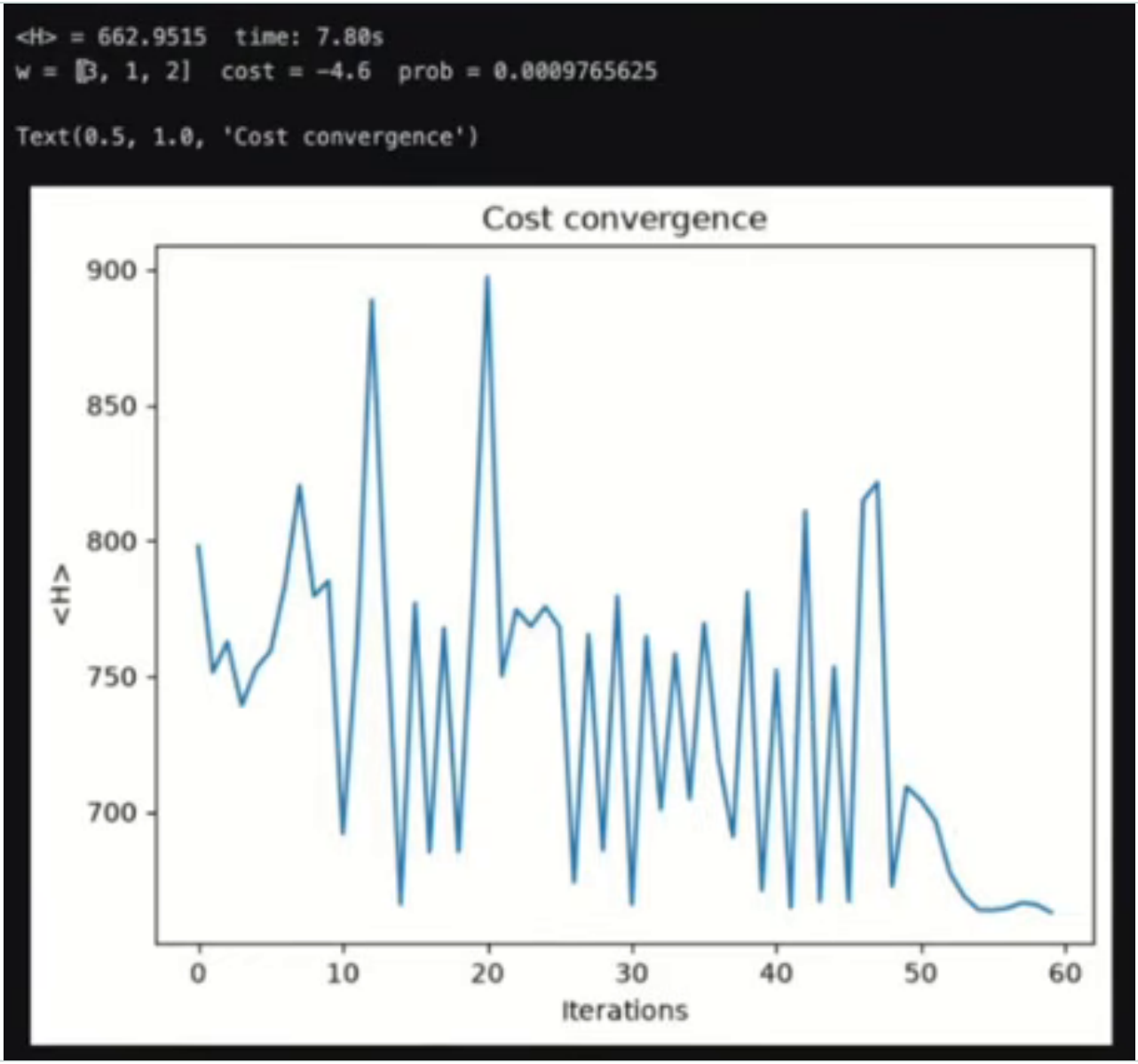


그림 2-4. **원영상의 실제 출력과 평가 곡선.** 위의 `<H>`는 전체 상태의 기대값이고 `cost=-4.6`은 표시한 한 배분의 비용이다. 가로축은 평가 반복, 세로축은 `<H>`이며 곡선은 오르내린다. 화면의 7.80초는 표시된 최적화 호출 시간이다. 출처: Classiq·NVIDIA, 30:18.

기대값 `<H>`와 개별 배분의 비용 −4.6은 서로 다른 값이다. 전자는 회로가 만드는 전체 상태에 대한 평가이고, 후자는 디코딩한 한 해의 비용이다. 예산 벌점과 표현 방식도 개입하므로 두 숫자를 같은 지표로 비교하면 안 된다. 특히 29:35의 코드는 CUDA-Q 최적화 뒤에 `combi.sample(optimized_params)`로 Classiq 표본을 얻고, 예산을 만족하는 행만 남긴 뒤 비용이 가장 낮은 행을 표시한다. 따라서 −4.6은 최적화한 상태 전체가 그 비용을 가졌다는 뜻이 아니다. 그래프는 약 60번의 평가 동안 크게 오르내리다가 마지막 구간에서 낮아진다. 발표자의 구두 설명을 매 단계 단조롭게 개선된 것으로 옮기지 않는다.

직접 재계산: 화면의 입력·목적함수·예산을 그대로 쓰면 가능한 정수 배분은 84개다. 이를 모두 계산했을 때 최저 비용은 −4.8이고, `[2,1,1]`과 `[1,2,1]`이 그 값을 준다. 시연 배분 `[3,1,2]`는 위험 6.4, 수익 11, 비용 −4.6으로 예산을 만족하지만 전역 최적해는 아니다. 이는 양자회로·CUDA-Q 실행을 독립 재현한 결과가 아니라, 작은 고전 최적화 문제의 목적함수만 정확 열거해 검산한 결과다.

이 사례가 보여준 성과는 금융 모델이 회로와 반복 실행까지 연결된다는 점이다. 최상의 고전 정수 최적화기보다 빠르다거나 실제 QPU에서 금융 우위를 얻었다는 결과는 이 출력에 없다.

4. 옵션 가격 문제는 왜 기대값 추정 문제가 되는가

32:39-37:50의 두 번째 사례는 유럽형 콜옵션이다. 만기 가격 `S_T`가 행사가격 `K`보다 높을 때만 차액을 지급하므로 지급액은 `max(S_T-K,0)`다. 만기 가격을 확률분포로 나타내면 구할 양은 그 분포에 대한 평균 지급액이다.

보충 해설: 일반적인 무차익 옵션 가격식은 위험중립 확률분포에 대해 할인한 기대지급액이다.

$$V_0 = e^{-rT}\,\mathbb{E}_{\mathbb{Q}}\big[\max(S_T - K, 0)\big].$$

`r`은 할인율, `T`는 만기까지의 시간이다. 다만 실제 시연 코드의 출력 열은 `Payoff`이고, 마지막 고전 기준값도 같은 격자 위의 지급액 평균이다. 따라서 아래의 약 0.177을 별도 할인·위험중립 보정까지 검증한 실제 시장가격으로 확장하지 않는다.

고전 Monte Carlo는 가격을 여러 번 표본추출해 지급액을 평균낸다. IQAE는 가격 분포와 지급액을 양자 상태의 확률로 부호화하고, 그 확률을 반복적으로 추정한다. IQAE 원 논문은 양자 위상 추정 없이 Grover 연산을 사용하며, 이론적 질의 복잡도에서 고전 Monte Carlo에 비해 이차적 개선을 준다는 결과를 제시한다. 그 이론에는 상태 준비·오라클 호출을 어떻게 비용으로 세는지라는 조건이 있다. 시뮬레이터에서 작은 예제를 실행한 시간만으로 이 금융 문제 전체의 양자 우위가 입증되지는 않는다.

5. 실제 IQAE 시연을 단계별로 따라가기

가격 분포를 유한 격자로 바꾸기

38:03-39:16에서 자산 가격을 로그정규분포로 둔다. 화면의 설정은 자산 레지스터 6큐비트, 로그 변수의 평균 `μ=0.7`, 표준편차 `σ=0.13`, 행사가격 `K=1.9`다. 이때 자산 레지스터는 $2^6=64$개 가격 격자점을 나타낸다. 이 6은 자산 분포를 표현하는 큐비트 수이며, 표시용 비트와 보조 자원을 포함한 전체 합성 회로 폭과 동일하다고 가정하지 않는다.

고전 전처리에서 가격 평균·표준편차를 구하고 평균의 ±3표준편차 범위에 격자를 만든다. 각 격자에서 확률밀도를 평가한 뒤 합이 1이 되도록 정규화한다. 따라서 연속 로그정규분포 자체가 정확히 보존되는 계산은 아니다. 가격 구간을 자른 오차와 64점 이산화 오차가 포함된다.

그래프에는 초록색 확률분포와 빨간색 지급액을 함께 그린다. 행사가격 1.9 이하에서 지급액은 0이고 그보다 높으면 선형으로 증가한다. 두 곡선을 함께 읽으면 '확률이 큰 가격'과 '지급액이 큰 가격'이 다르며, 기대지급액은 두 요소를 곱해 모두 합산한 값이라는 점을 알 수 있다.

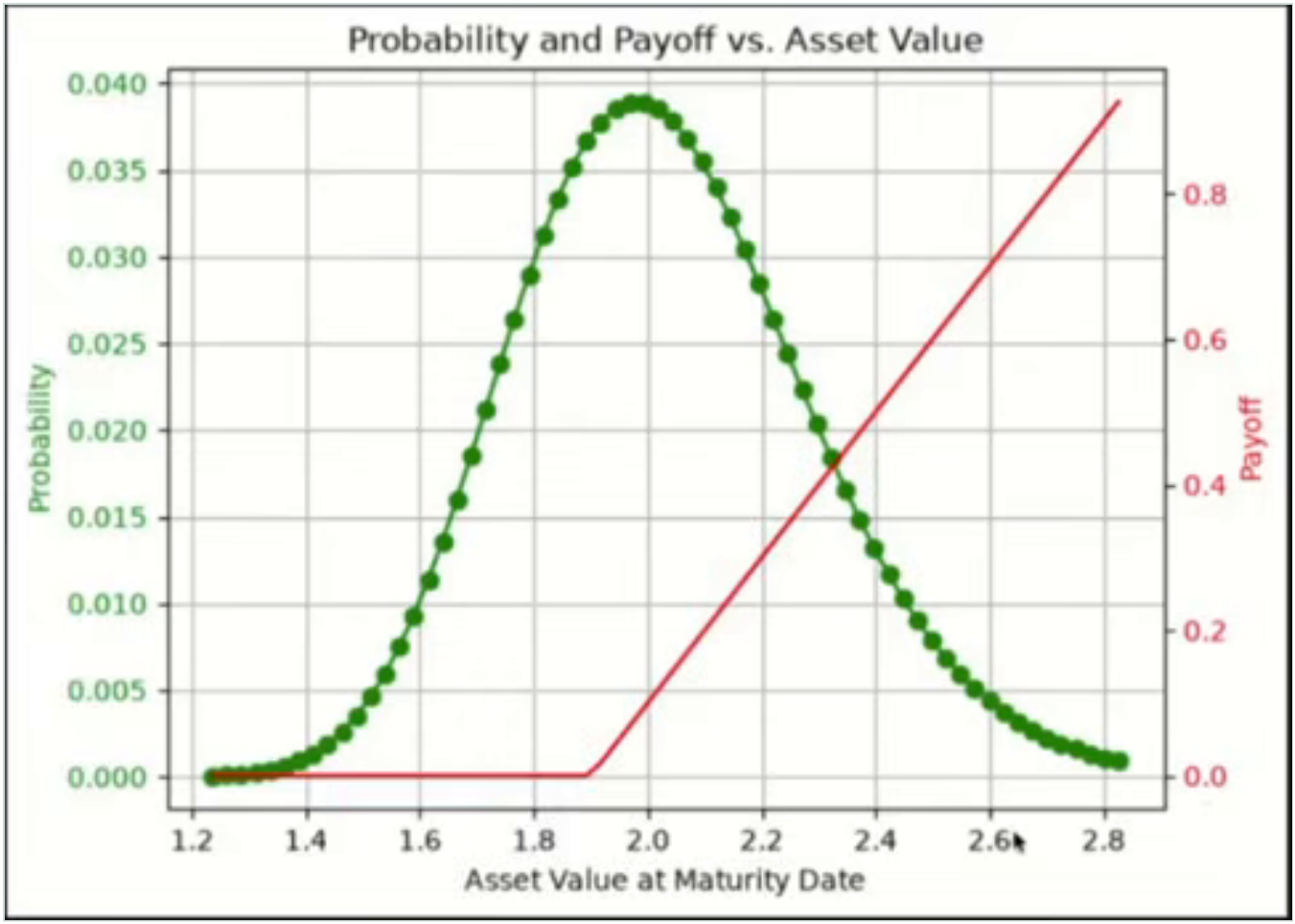


그림 2-5. **확률과 지급액을 함께 읽는 발표 그래프.** 초록 점은 가격 격자별 확률로 왼쪽 축을, 빨간 선은 지급액으로 오른쪽 축을 사용한다. 두 세로축의 눈금과 단위가 다르므로 곡선의 교차점 자체에 금융적 의미를 부여하지 않는다. 기대지급액은 같은 가격에서의 두 값을 곱해 합한 양이다. 출처: Classiq·NVIDIA, 40:15.

분포와 지급액을 양자 상태에 넣기

39:18-40:46에서 `@qfunc`로 양자 함수를 정의한다. `load_distribution`은 분포 준비를, `payoff`는 행사가격 조건을 확인한 지급액 부호화를 담당한다. 이를 `european_call_state_preparation`에서 합친다.

보충 해설: 가격 격자 확률을 `p_j`, 정규화한 지급액을 `f_j∈[0,1]`로 쓰면 핵심 상태는 다음처럼 이해할 수 있다.

$$A|0\rangle = \sum_j \sqrt{p_j}\,|j\rangle \left(\sqrt{1-f_j}\,|0\rangle + \sqrt{f_j}\,|1\rangle\right).$$

마지막 표시 큐비트가 1일 확률은 `a=Σ_j p_j f_j`다. 따라서 그 확률을 추정한 뒤 지급액의 정규화 계수를 곱하면 기대지급액을 얻는다. 이 식은 강연 코드의 기능을 설명하기 위한 재구성이며, 화면의 회로를 독립 시뮬레이션한 결과가 아니다.

반복 가능한 커널 만들기

40:48-42:44에서 표시 큐비트에 작용하는 오라클과 상태 준비를 결합한다. `main(k, indicator)`의 `k`가 진폭 증폭의 반복 횟수에 대응한다. 모델을 합성한 뒤 `qprog_to_cudaq_kernel`로 내보내고 `cudaq.draw`로 일반 CUDA-Q 커널처럼 그린다. 구현의 포인트는 Classiq 안에서만 실행되는 특수 결과를 만드는 것이 아니라 기존 CUDA-Q 실행 코드에 넣을 수 있는 커널을 만든다는 데 있다.

같은 추정기를 유지하고 실행 함수 바꾸기

42:47-45:13에서 두 수준의 인터페이스를 보인다. 먼저 Classiq의 `IQAE` 래퍼로 상태 준비 함수와 오차·신뢰도 설정을 전달해 실행한다. 이어 `GenericIQAE`에는 표본추출 함수를 인자로 넘긴다. 하나는 Classiq의 `sample`, 다른 하나는 `cudaq.sample`을 호출하고, 각각 표시 큐비트가 1인 횟수를 반환한다. 같은 고전 추정 반복에 어떤 실행기를 연결할지 바꿀 수 있음을 보여주는 부분이다.

IQAE의 반복은 앞서 QAOA에서 회전각을 조정한 변분 최적화와 다르다. IQAE에서는 증폭 반복 횟수와 측정 통계를 바탕으로 미지의 확률이 들어 있을 구간을 좁힌다. 두 방법 모두 고전 코드가 양자회로를 반복 호출하지만, 반복하는 목적과 갱신하는 양은 같지 않다.

화면 설정은 `EPSILON=0.02`, `ALPHA=0.01`, 반복당 `SHOTS=4000`이다. `ALPHA=0.01`에 해당하는 출력 신뢰구간은 99%다. 여기서 epsilon을 실제 옵션가격의 상대 오차 2%로 읽으면 안 된다. 정규화·재변환을 거쳐 지급액 단위의 오차가 정해진다. `max k`도 최대 Grover 반복 횟수이며 총 표본 수나 전체 오라클 호출 수와 다르다.

화면에 나온 수치는 얼마나 일치하는가

43:15의 첫 래퍼 출력은 추정치 0.1768, 99% 신뢰구간 `[0.1756,0.1779]`, 반복 2회, 최대 `k=9`다. 44:45의 최종 비교는 아래와 같다. 첫 래퍼와 최종 비교의 Classiq 값은 별도 표본추출 실행이므로 소수점 아래가 동일할 필요는 없다.

실행 경로	기대지급액 추정	최대 k	출력된 99% 신뢰구간
Classiq 표본추출 + GenericIQAE	0.17771	9	`[0.17658,0.17884]`
CUDA-Q 표본추출 + GenericIQAE	0.17675	4	`[0.17427,0.17923]`
같은 64점 격자의 고전 직접 합	0.17674	해당 없음	해당 없음



그림 2-6. **발표의 최종 비교 출력.** 위의 한국어 표는 이 원출력의 값을 옮긴 것이다. 두 추정값은 같은 격자 기준과 비교되며, 실행시간 열은 없다. 이 표로 GPU 속도 향상이나 실제 QPU 우위를 계산할 수는 없다. 출처: Classiq·NVIDIA, 44:45.

두 추정 신뢰구간에 같은 격자 기준값이 들어 있다. 화면의 전처리를 별도로 재계산한 고전 기대지급액은 0.1767391664로, 표시값 0.17674와 일치했다. 이 비교는 실행기를 바꿔도 이 예제의 기대지급액이 통계적으로 일관됨을 보여준다. 연속분포·실제 시장 데이터에 대한 정확성이나 고전 최선 방법 대비 속도 우위를 검증한 표는 아니다. 44:36의 코드도 `qpp-cpu`를 사용하며 `nvidia`로 바꾸면 GPU 상태벡터 시뮬레이션을 할 수 있다는 주석을 붙인다. 최종 비교표에는 실행시간 열이 없어, 이 표만으로 GPU 배속을 산출할 수 없다.

6. 강연에서 실제로 받은 질문과 답변

아래 9건은 영상에 선택되어 표시되거나 낭독되고 구두 답변이 이어진 청중 질문이다. 한국어 질문은 요지 번역이며, 발표자의 답변과 이 노트의 검토를 구분했다. 다시보기에는 전체 Zoom Q&A·채팅 기록이 제공되지 않아, 화면에 선택되지 않은 질문의 총수와 채팅으로만 답한 내용은 확인할 수 없다.

Q1. CUDA-Q와 Classiq를 누구나 무료로 사용할 수 있는가?

19:00-19:36. **답변:** CUDA-Q는 내려받아 사용할 수 있는 오픈소스 프로젝트다. Classiq는 라이선스가 필요하며 플랫폼에 가입하면 안내를 받을 수 있다고 답한다. **기록 범위:** 화면에서 질문 뒤가 말줄임으로 잘리고 발표자도 뒷부분을 놓쳤다고 말한다. 따라서 질문의 추가 조건은 복원하지 않았다. 무료 사용 가능 범위, 학술용 조건, QPU·클라우드 사용료까지 답한 것으로 확장하지 않는다.

Q2. Qiskit·PennyLane 등 다른 SDK와 연결하기 쉬운가?

19:44-20:19. **답변:** Classiq는 모듈식 접근을 취하며 높은 수준의 모델링과 다른 SDK의 실행을 조합할 수 있다고 설명한다. CUDA-Q 연결이 오늘 시연의 중심이라는 답이다. **기록 범위:** 이 영상에서 실제로 검증해 보이는 연동은 CUDA-Q다. Qiskit·PennyLane을 직접 실행하는 별도 시연은 없다.

Q3. Classiq는 Qiskit·PennyLane과 무엇이 다른가?

20:24-21:14. **답변:** Vincent는 Qiskit·PennyLane에서는 보통 논리 회로와 기능의 구현 방법을 사용자가 직접 지정하는 반면, Classiq에서는 계산할 기능을 표현하고 목표 하드웨어를 아는 합성기가 구현을 정한다고 설명한다. **검토:** 이 답변은 Classiq가 제시하는 주된 사용 방식의 비교다. 다른 SDK가 높은 수준의 라이브러리·컴파일 기능을 전혀 제공하지 않는다는 포괄적 주장으로 읽지 않는다.

Q4. 포트폴리오 최적화에 알려진 효율적 양자 알고리즘은 무엇인가? 가장 유명한 양자 알고리즘은 무엇인가?

49:09-50:11. 질문 슬라이드에서 두 질문이 함께 확인된다. **답변:** 유명한 알고리즘의 예로 Shor 알고리즘을 들고, 포트폴리오 최적화에서는 QAOA가 널리 쓰이며 기본형과 변형이 있다고 답한다. 제약을 만족하지 않는 공간을 피하거나 수렴을 개선하려는 변형도 언급한다. **남은 점:** 어떤 포트폴리오 입력군에서 어떤 고전 기준보다 효율적인지에 대한 복잡도 정리나 실측 비교는 답변에 없다. '자주 연구되는 알고리즘'과 '실용적 우위가 증명된 알고리즘'을 구별해야 한다.

Q5. 합성 최적화는 게이트 집합만 고려하는가, 장치의 노이즈도 고려하는가?

50:26-51:43. **답변:** 목표가 무엇인지에 달려 있다고 구분한다. 논리 회로 합성에서는 장치의 연결성과 게이트 집합을 고려하지만, 생성 회로가 너무 깊어 실제 장치에서 유효한 신호를 얻지 못할 수도 있다고 말한다. 별도의 내결함성 엔진은 오류정정 회로를 만들기 위해 목표 장치의 노이즈 특성을 고려한다고 답한다. **검토:** 모든 회로가 자동으로 노이즈를 극복한다는 답변이 아니다. 실제 노이즈 모델이나 오류정정 자원 수치를 시연하지는 않는다.

Q6. 높은 수준으로 코딩해도 내부 저수준 기능을 확인할 문서가 있는가?

51:53-53:02. **답변:** 공개 라이브러리 함수는 GitHub에서 구현을 볼 수 있다. 합성 뒤에는 게이트 기반 프로그램이 나오므로 시각화에서 CNOT·RX 등 구체적인 구현을 볼 수 있다고 답한다. **기록 범위:** 공개 함수의 합성 전 구현과 모든 합성 출력의 게이트 수준 확인을 구별한다. 합성 엔진 자체가 모두 공개되어 있다는 답변은 아니다.

Q7. Classiq는 일종의 양자 컴파일러인가?

53:20-53:53. **답변:** 그렇다고 답한다. 처음부터 양자 프로그램을 효율적이고 확장 가능하게 만들기 위한 컴파일러에 집중했다는 설명이다. **의미:** 앞에서 제시한 모델→합성→게이트 회로라는 구조를 한 문장으로 확인하는 질문이다.

Q8. 화학의 분자 시뮬레이션에도 적합한가?

53:57-55:50. **답변:** 적합하다고 답하며, 분자·화학계의 에너지 계산에서 양자 위상 추정(QPE)을 예로 든다. 깊이·회전 게이트·CNOT 수를 줄이는 회로 구현이 중요하고, SoftBank 협업 결과를 찾아보라고 안내한다. **기록 범위:** 이 부분은 응용 가능성과 외부 결과 안내다. 분자 계산을 현장에서 실행하거나 비교 수치를 제시한 시연은 아니다. 회로 자원 감소와 화학 문제에서의 실제 양자 우위는 따로 검증해야 한다.

Q9. 중성원자 기반 양자 하드웨어를 지원하는가?

56:07-57:06. **답변:** 조건부로 그렇다. 해당 장치가 게이트 연산을 지원하면 회로를 만들 수 있지만, 엄격한 아날로그 방식만 지원하는 장치는 당시 Classiq 경로에서 활용하지 못한다고 답한다. **의미:** 큐비트의 물리적 종류뿐 아니라 장치가 제공하는 계산 모델을 확인해야 한다는 답이다. 자동 자막의 `neural atom`은 화면 질문과 문맥에 따라 `neutral atom`, 즉 중성원자로 바로잡았다.

시간 부족으로 57:09 이후 질문을 마감한다. 특정 질문을 읽고 답을 유보하는 장면은 확인되지 않는다. 따라서 위 9건은 '영상에서 구두 답변이 확인된 질문'이며, 나머지를 임의로 '미답변 목록'에 추가하지 않는다.

7. 복습할 때 스스로 답해 볼 질문

이 항목은 실제 청중 질문과 별개로 작성한 학습용 질문이다.

1. 자산 수가 3인데 회로는 왜 12큐비트인가? 정수 표현, 예산 제약, 보조 자원 중 어떤 정보가 추가로 필요한가?
2. 위험–수익이 가장 작은 표본을 하나 얻는 것과 그 표본을 높은 확률로 얻는 것은 어떻게 다른가? `<H>`, 개별 비용, 출력 확률을 각각 설명할 수 있는가?
3. 예산을 6으로 두었는데 최적해가 반드시 6단위를 모두 사용해야 하는가? `≤6`과 `=6`은 같은 문제가 아닌 이유가 무엇인가?
4. Classiq 회로의 벌점 10과 CUDA-Q에서 평가하는 해밀토니안의 벌점이 다르면 어떤 문제가 생기는가?
5. 옵션 예제에서 분포를 정확히 준비했더라도 연속 가격분포에 대한 오차가 남는 이유는 무엇인가?
6. 표시 큐비트가 1일 확률을 알면 왜 기대지급액을 얻을 수 있는가? `√f_j`를 진폭으로 넣는 이유를 설명할 수 있는가?
7. 99% 신뢰구간은 분포의 64점 이산화 오차와 모델 위험까지 포함하는가? 이 시연의 신뢰구간이 다루는 것은 무엇인가?
8. CUDA-Q의 백엔드를 `qpp-cpu`에서 `nvidia`로 바꾼 속도 향상과 QPU에서 얻는 양자 알고리즘 우위는 각각 무엇을 비교해야 판정할 수 있는가?
9. 중성원자 장치에 접근할 때 '지원한다'는 말만 듣지 않고 확인해야 하는 연산 방식은 무엇인가?

복습 질문의 풀이

1. 자산 수는 업무 변수의 개수다. 각 보유량 0-7을 표현하려면 변수마다 3비트가 필요하고, 예산 제약의 slack·산술·보조 공간이 추가될 수 있다. 실제 12큐비트가 어떤 용도로 배정됐는지는 합성 모델과 회로에서 확인한다. 3개 자산을 3큐비트로 표현했다고 계산하지 않는다.
2. `<H>`는 전체 상태의 해밀토니안 기대값이고, −4.6은 선택한 한 배분의 금융 비용이다. 작은 표시 확률의 좋은 해는 반복 실행에서 자주 나오지 않을 수 있다. 최선 표본·기대 비용·유효 해 비율과 성공확률을 함께 보고한다.
3. 예산이 상한이면 모두 쓰지 않아도 된다. 이 입력의 최적 배분 `[2,1,1]`은 4단위만 사용한다. 등식으로 강제하면 허용 집합을 바꾸므로 새로운 최적화 문제가 된다.
4. 회로를 만드는 모델과 평가하는 해밀토니안이 다른 문제를 표현할 수 있다. 원래 목적함수와 제약, 벌점의 계수·상수항·변수 인코딩이 두 경로에서 일치하는지 확인한다.
5. 유한 구간의 64점 분포는 연속 분포를 잘라 이산화한 것이다. 그 이산 분포를 정확히 준비해도 구간 밖 꼬리와 격자 근사 오차는 남는다. 양자 추정 오차와 별도로 평가해야 한다.
6. 보조 비트의 진폭을 정규화 payoff의 제곱근으로 만들면, 측정확률은 payoff 자체에 비례한다. 가격 상태의 확률로 다시 가중해 합산하므로 `a=Σ_j p_j f_j`가 되고 정규화 상수를 곱해 평균 지급액을 복원한다. 임계값 비교만 하면 행사 확률만 얻는다.
7. IQAE의 통계적 구간은 정한 회로·분포·추정 절차의 성공확률을 다룬다. 연속 분포의 이산화, 실제 시장 모형의 부정확성, 구현 편향을 모두 포함하는 총오차 구간으로 읽지 않는다. 99%라는 값은 해당 방법의 반복 실험 보장과 가정 아래

해석한다.

8. 전자는 같은 회로·정밀도·측정 조건을 두 고전 시뮬레이터에서 실행한 비교다. 후자는 실제 QPU가 같은 업무 목표와 오차에서 정한 고전 알고리즘 기준보다 나은지 비교하는 문제다. 전처리·컴파일·대기·후처리를 포함할 범위도 같게 정한다.
9. 게이트 기반 디지털 연산인지 아날로그 해밀토니안 진화인지 확인한다. 발표자가 설명한 게이트 기반 회로의 지원을 모든 중성원자 장치와 모든 연산 방식의 지원으로 확장하지 않는다. 실제 접근 가능한 백엔드·게이트·연결성·계정 조건은 별도로 확인한다.

8. 이 세션을 보고 실제로 판단할 수 있는 것

금융 모델을 작성하는 부분, 게이트 회로를 만드는 부분, 회로를 반복 실행하는 부분은 다른 작업이다. 이 세션은 세 작업을 연결하는 작은 예제를 끝까지 보여준다. 포트폴리오에서는 정수 최적화 모델을 QAOA 회로와 CUDA-Q 기대값 평가로 옮겼고, 옵션에서는 지급액 기대값을 표시 큐비트의 확률로 바꾸어 두 표본추출 경로에서 일관된 결과를 얻었다.

동시에 실제 화면을 기준으로 보면 시연은 CPU 시뮬레이션이며, 포트폴리오 출력은 전역 최적해가 아니고, 옵션 비교는 같은 이산 격자의 정합성을 확인한다. 앞으로 확인할 것은 더 큰 문제에서의 해 품질, 상태 준비 비용, 반복당 표본 수, 회로 깊이, 같은 정밀도의 최선 고전 계산과 비교한 전체 시간이다. 45:21 이후의 확장 논의는 GPU 병렬화·지연 감소와 복잡한 파생상품·VaR·대학 연구 활용으로 이어지지만, 이 항목들은 현장 시연으로 측정한 새 결과와 구별된다.

출처와 검토 범위

- 주된 근거: Classiq 공식 다시보기. 영어 자동 자막 전체를 읽고, 수학적 입력·백엔드 설정·출력 표·질문 슬라이드는 해당 화면과 대조했다. 자동 자막에서 반복적으로 잘못 인식된 Classiq·CUDA-Q·IQAE·중성원자 명칭을 화면과 문맥으로 바로잡았다.
- 행사 날짜: Classiq 공식 행사 안내.
- 보충 원전: QAOA 원 논문, IQAE 원 논문, NVIDIA CUDA-Q 실행 문서.
- 직접 검산: 화면에 나타난 3자산 모델의 가능한 배분 84개를 정확 유리수 연산으로 열거하고 목적함수·최적값을 확인했다. 옵션의 64점 고전 격자 기대지급액도 다시 계산해 표시값과 대조했다. Classiq 합성이나 CUDA-Q 양자회로는 독립 재실행하지 않았다.
- 저자들의 공동 기술 블로그는 추가 학습 자료다. 현재 본문의 자산 배분 결과·옵션 큐비트 수와 구현이 영상과 달라, 이 노트의 시연 수치를 블로그 값으로 대체하지 않았다.
- 작성 보조: OpenAI Codex, 에이전트 역할 `finance_replay`가 다시보기 자료 확보·시간별 정리·화면 대조·소규모 고전 검산을 수행했다. 정확한 모델 식별자는 미보존이다. 편집 기준은 AI Tech Review 2026.08 및 과학 설명 준칙 1.0을 적용했다. 사람의 문장별 검토·청취 평가는 수행 여부를 확인하지 못했다.

03. 자연어 요구를 실행 가능한 양자 프로그램으로 바꾸기

대상 세션: Classiq Quantum Office Hours: From Idea to Quantum Circuit with AI Agents
행사: 2026년 7월 8일 · **다시보기 공개:** 2026년 10월 20일 · **길이:** 43분 55초
발표·진행: Dr. Erik Garcell, Director of Quantum Solutions Architecture; Dr. John (Jack) Glennon, Quantum Applications Engineer
정리 기준: 2026년 10월 22일. 공식 다시보기의 영어 자동자막 전 구간과 공식 행사 페이지를 대조했다. 아래 시각은 녹화물 상대시각이며 몇 초의 자막 오차가 있을 수 있다. 실제 청중 문답, 진행자끼리의 질문, 노트 작성자가 추가한 학습 질문을 구별했다. 화면의 전체 코드·그래프를 독립 실행하거나 픽셀 단위로 검수하지는 않았다.

공식 다시보기 · 공식 행사 소개와 발표자 정보

이 세션에서 판단할 수 있게 되는 것

이 세션의 핵심은 AI가 양자 알고리즘을 새로 발견했다는 데 있지 않다. 사용자가 해를 판별하는 조건을 자연어로 제시했을 때, 에이전트가 Qmod 모델을 만들고 Classiq 합성기를 호출해 회로를 준비하며, 사용자가 모델·회로·실행 결과를 오가며 확인하는 작업 흐름을 보여 준다. 작은 Grover 탐색 시연을 중심으로 시작 장벽을 설명한 다음, 기존 개발자에게는 논문 구현 보조와 하드웨어별 컴파일 비교가 어떻게 유용할지 제시한다.

따라서 학습할 때에는 서로 다른 세 질문을 분리해야 한다. **요구한 문제를 올바르게 모델링했는가**, **그 모델을 실행 가능한 회로로 만들었는가**, **그 회로의 비용과 결과가 목적에 적합한가**이다. 데모는 컴파일과 실행이 이루어진 사례를 보여 준다. 모델과 회로의 의미가 모든 입력에서 같다는 독립 증명이나 업무상의 유용성을 보증하는 결과는 아니다.

1. 전체 흐름과 주요 장면

다시보기	실제로 다루는 내용	학습할 때의 초점
00:00-02:00	발표자 소개, Office Hours의 목적, 청중 경험 설문	제품 중심 시연이며 알고리즘 증명 강의와 범위가 다름
02:04-08:31	Qmod·합성기·시각화·실행, 여러 AI 도구와 당시 로드맵	모델, 컴파일, 실행의 역할 구분
08:34-10:23	학습·개발 병목 설문	시작하기, 알고리즘 배우기, 앱 만들기가 많이 언급됨
10:49-15:14	`x*y=6` Grover 프로그램 생성, 1,000 shots 시뮬레이션, Qmod 열람	자연어에서 oracle과 결과 해석까지 연결
15:17-18:43	width/depth 최적화, 계층별 회로 보기, 재실행	자원 목표가 달라지면 같은 모델의 회로가 달라짐
19:26-24:13	첫 청중 Q&A	필요한 선수지식, 범용 챗봇과의 차이
24:17-32:38	IDE 설문, 플러그인, Studio, 논문 구현 사례와 다중 모듈 예	코드 생성 이외의 활용
32:40-40:53	두 번째 Q&A, 문서 assistant 시연, 팀 구성, QAOA 비교 노트북	설명과 학습, 검토해야 할 벤치마크 조건

다시보기	실제로 다루는 내용	학습할 때의 초점
40:56-43:47	다음 회차 관심 분야 설문, 정리	금융·제약/화학·AI/ML 관심이 대략 세 갈래로 나타남

설문 수치는 발표자가 읽어 준 근삿값이다. 표본 수·응답자 구성·원 설문 데이터가 공개된 통계 연구로 취급하지 않는다.

2. 플랫폼을 네 단계로 나누어 이해하기

02:04-04:59에서 Erik은 Classiq를 모델 작성부터 실행까지 잇는 개발 플랫폼으로 소개한다. 개발자는 Qmod로 계산의 구조를 표현하고, 합성기가 목표와 제약에 맞추어 회로를 만들며, 시각화 도구로 이를 검사한 뒤, 실행 환경을 선택한다. 여기서 자연어 에이전트가 추가되는 위치를 구별하면 뒤의 시연이 선명해진다.

1. **문제와 알고리즘 선택:** 어떤 입력에서 무엇을 구하고 싶은지 결정한다. `x*y=6`에서는 곱셈 조건을 만족하는 정수쌍을 찾는 것이 문제이고, Grover 탐색이 사용자가 요구한 알고리즘이다.
2. **Qmod 모델 생성:** 양자 변수, 조건식, 초기 상태 준비, 반복 구조 등 알고리즘의 의미를 코드로 표현한다. 높은 추상화 수준의 표현을 쓴다고 해서 변수의 범위나 정밀도가 저절로 정해지는 것은 아니다.
3. **회로 합성:** 모델과 width·depth·게이트 집합·연결성 등의 조건을 입력받아 회로를 만든다. 에이전트의 언어 생성과 합성기의 회로 생성은 별도 단계다.
4. **실행과 해석:** 시뮬레이터 또는 접근 가능한 QPU에서 측정하고, 비트열을 모델의 변수와 연결해 읽는다. 적절한 통계량과 고전 기준값을 비교하는 일도 포함한다.

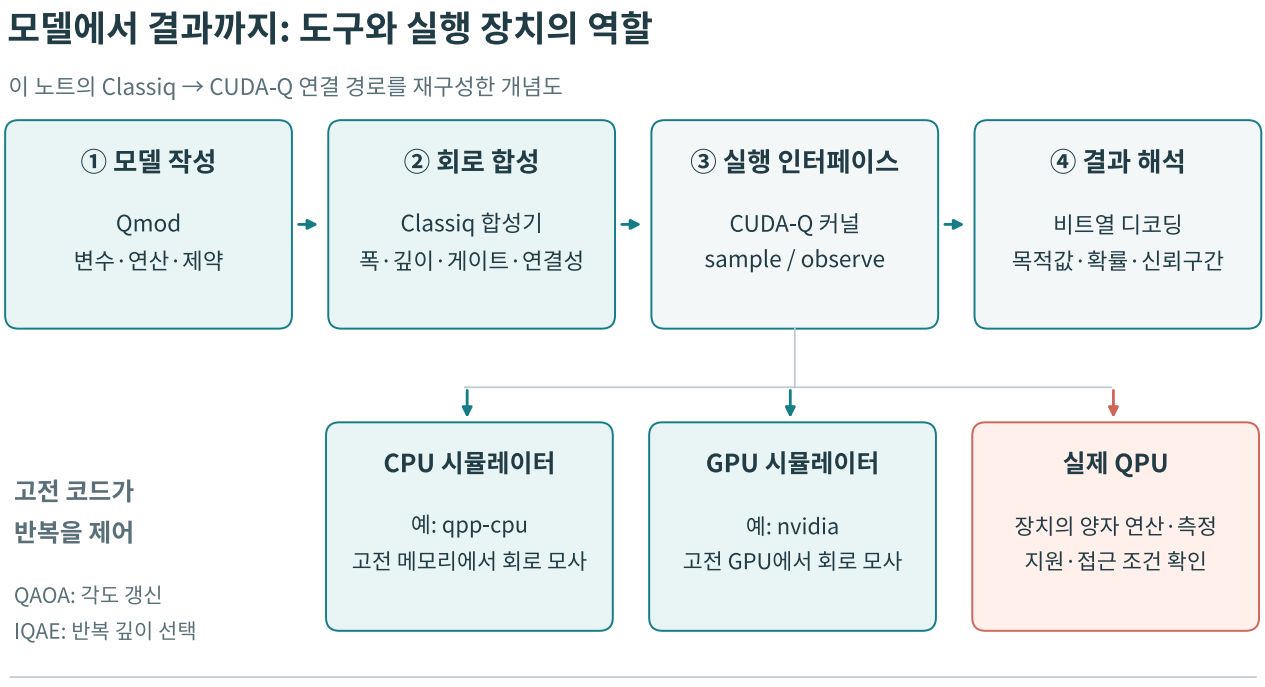


그림 3-1. **모델에서 결과까지 도구와 장치의 역할.** 이 회차의 네 단계에 10월 금융 세션의 CUDA-Q 연동을 이어 붙인 설명용 도식이다. 실행 인터페이스 아래에서 CPU·GPU 시뮬레이터와 실제 QPU가 갈라지는 부분을 읽는다. Classiq 자체의 실행 경로도 있으며, 그림은 CUDA-Q를 사용하는 대표 경로를 보여준다. QAOA의 각도와 IQAE의 증폭 깊이를 갱신하는 고전 반복은 서로 다른 목적을 갖는다.

발표자는 하드웨어의 차이를 개발자가 매번 직접 다루는 부담을 줄인다고 설명한다. 이를 "어떤 모델이든 모든 기계에서 같은 비용과 품질로 실행된다"는 뜻으로 확대하면 안 된다. **상위 모델을 유지하면서 각 하드웨어에 맞추어 다시 합성할 수 있다**는 의미로 이해해야 한다. 큐비트 수, 연결성, 네이티브 게이트와 오류율은 계속 회로의 실행 가능성과 품질을 제한한다.

당시 도구 소개도 시점을 고정해 읽어야 한다. 06:28-08:20에서 Studio agent, 공개 문서 assistant, 홈페이지 assistant와 IDE 플러그인을 소개하고, 일부 기능을 "내일" 유료 고객에게 제공한다고 말한다. 이는 **7월 8일 발표 시점의 제공 계획**이다. 10월 20일에 업로드된 영상의 공개일을 기준으로 "다음 날 출시"라고 옮기지 않는다. `V-Next`라는 내부 이름의 완전한 agentic 플랫폼도 이 자리에서는 방향·로드맵으로 제시된다.

3. 중심 시연: `x*y=6`을 만족하는 정수쌍 찾기

3.1 자연어 요청에서 어떤 명세가 나오는가

10:49-12:33에서 Jack은 양자 초보자의 관점으로 "Grover search로 `x*y=6`인 모든 쌍을 찾아 달라"고 요청한다. assistant의 응답은 문제 설명, 예상되는 해, 변수·함수의 역할을 정리한 표, 관련 라이브러리·문서 링크를 포함한다. 중앙 화면에는 이미 합성된 회로가 나타난다.

여기서 실제로 생성된 것은 자연어 설명만이 아니다. 12:46-13:08에서 Erik이 "컴파일까지 한 것인가"라고 묻고 Jack이 바로 실행할 수 있는 회로라고 확인한다. 이 대화는 **공동 진행자 사이의 설명 질문**이며 청중 Q&A가 아니다.

필기자 보충 — 명세에 반드시 들어가야 하는 것. 수학적으로 `xy=6`은 변수의 영역에 따라 다른 문제다. 양의 정수라면 `(1,6),(2,3),(3,2),(6,1)` 네 쌍이지만, 음의 정수까지 허용하면 `(-1,-6)` 등도 생기고 실수라면 무한히 많은 해가 있다. 실제 데모는 네 양의 정수쌍이 증폭된다고 설명한다. 이 노트에서는 화면 전체의 타입 선언을 재현하지 않았으므로 정확한 비트폭·부호 설정까지 단정하지 않는다. 사용자가 프로그램을 만들 때에는 정수/고정소수점, 부호, 입력 범위, 결과를 한 쌍만 찾는지 여러 해를 수집하는지까지 명세해야 한다.

3.2 시뮬레이터 결과와 모델 읽기

13:11-13:58에서 1,000 shots로 **시뮬레이터**를 실행한다. Jack은 히스토그램에서 네 상태가 증폭됐고, 해석 표에서 `(3,2)`, `(6,1)`과 순서를 뒤집은 쌍들을 볼 수 있다고 설명한다. 이는 조건에 맞는 작은 탐색 예의 실행 확인이다. 실제 QPU 성능, 큰 정수 인수분해, 고전 알고리즘 대비 시간 우위의 실험은 아니다.

14:02-15:14에는 모델 창으로 돌아가 Qmod 코드를 읽는다. `x,y`를 담는 구조, `x*y==6`을 판별하는 oracle, 내장 함수로 구성한 main이 핵심이다. 기호 산술로 조건을 적을 수 있다는 점을 강조한다. 공식 Qmod 설명도 알고리즘의 의도를 높은 수준에서 표현하고 구현 결정을 자동화하는 방향을 설명한다. Vax 외, Qmod: Expressive High-Level Quantum Modeling.

3.3 Grover가 실제로 하는 일을 식으로 보충하기

다음은 시연을 이해하기 위한 **노트 작성자의 수학적 재구성**이다. 영상에서 이 유도를 모두 설명한 것은 아니다. 탐색 영역에 `N`개의 후보가 있고 그중 `M`개가 조건을 만족한다고 하자. 균일한 초기 상태와 정답 상태의 균일 중첩을 각각

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{z} |z\rangle, \qquad |G\rangle = \frac{1}{\sqrt{M}} \sum_{f(z)=1} |z\rangle$$

로 놓는다. `z`는 여기서 `(x,y)`를 하나로 묶은 입력이고 `f(x,y)=1`은 곱셈 조건이 참임을 뜻한다. 정답이 아닌 상태의 정규화된 중첩을 `|B⟩`라고 하면

$$|s\rangle = \sin\theta\,|G\rangle + \cos\theta\,|B\rangle, \qquad \sin^2\theta = \frac{M}{N}.$$

위상 oracle `O_f|z⟩=(-1)^{f(z)}|z⟩`와 초기 상태에 대한 반사를 번갈아 적용하면 이 2차원 평면에서 상태가 회전한다. `r`회 반복 후 정답을 측정할 확률은

$$P_{\text{good}}(r) = \sin^2\big((2r+1)\theta\big)$$

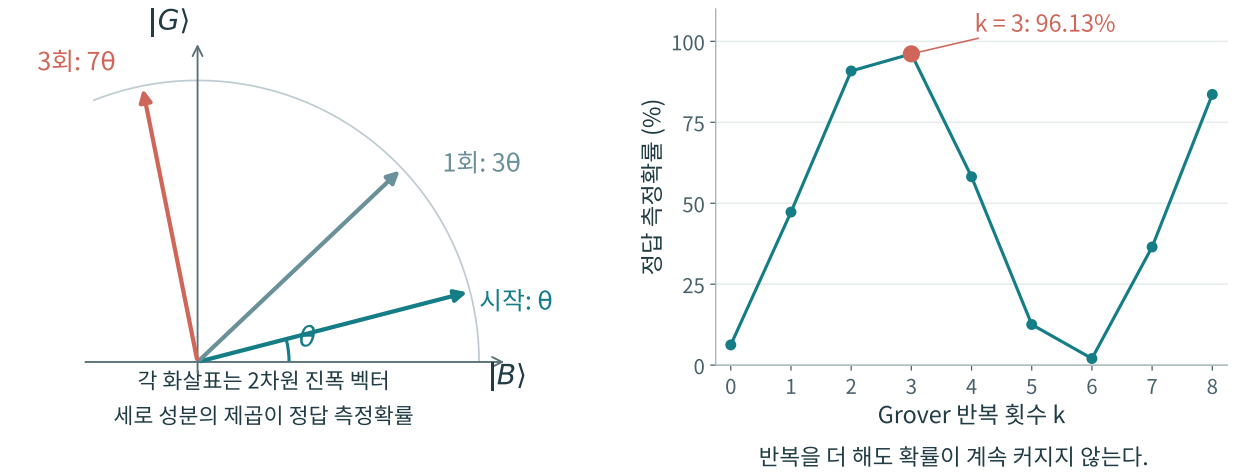


그림 3-2. **진폭의 회전이 확률을 바꾸는 방법.** 왼쪽은 정규화된 정답·비정답 중첩을 축으로 한 평면이며, 화살표의 세로 성분을 제곱하면 정답 확률이다. 오른쪽은 후보 64개 중 정답 4개인 무잡음 균등 준비 예의 직접 계산이다. 3회 반복에서 96.13%지만 이후 감소할 수 있다. 원영상의 측정 히스토그램을 재현한 실험 그래프가 아니다.

이다. 따라서 “정답을 표시한다”와 “정답을 높은 확률로 얻는다”가 구별된다. oracle만으로 모든 해가 읽히는 것이 아니며, 반복을 너무 많이 하면 성공확률이 다시 작아질 수도 있다. 한 번 측정하면 정답 하나가 나오므로, 여러 해를 모두 모으려면 반복 측정과 이미 본 해의 관리가 필요하다. 이 시연의 네 봉우리는 많은 shots로 얻은 분포를 보고한 것이다. Brassard 외, Quantum Amplitude Amplification and Estimation, Classiq의 Grover 연산자 구성 예제.

4. 같은 모델에서 width와 depth를 바꾸는 이유

15:17-16:44에 Jack은 먼저 width 최적화를 선택한다. 자동자막에서 확인되는 보고값은 **13 qubits, depth 1,900을 조금 넘는 회로**이다. 다음으로 depth 최적화를 선택하면 **depth 1,300을 조금 넘는 회로**가 나오며 큐비트 수는 조금 늘었다고 설명한다. 두 번째 정확한 큐비트 수와 두 회로의 정확한 depth 정수는 이 기록에서 확정하지 않는다.

합성 목표	발표에서 읽어 준 결과	이 장면의 의미
Width 줄이기	13 qubits, depth > 약 1,900	작업 공간을 절약하면서 더 긴 계산을 허용
Depth 줄이기	depth > 약 1,300, 큐비트 수 증가	공간을 더 써 계산을 병렬화하거나 재사용 부담을 줄일 여지

필기자 보충. `width`는 회로가 요구하는 큐비트 수이고 `depth`는 병렬 실행 가능한 게이트들을 층으로 묶었을 때의 길이다. 일반적으로 보조 큐비트를 재사용하면 공간은 줄지만 계산·역계산을 반복해야 할 수 있다. 반대로 여분의 보조 공간은 일부 작업을 겹치게 할 수 있다. 다만 모든 회로에서 단순히 "큐비트를 늘리면 depth가 줄어든다"는 법칙이 성립하는 것은 아니다. 또한 논리 depth가 같아도 게이트 시간, 2큐비트 게이트 개수, 라우팅과 오류율에 따라 실제 실행 품질은 달라진다.

16:48-17:40에는 상위 함수 박스를 펼쳐 게이트 수준까지 내려간다. 추상화는 내부를 보지 못하게 하는 장치가 아니라 사람이 우선 읽을 수준을 바꾸는 수단으로 설명된다. 17:42-18:35에는 여러 실행 대상의 메뉴와 재실행 결과를 보여 주며 같은 문제 모델의 재사용을 강조한다. 다만 이때 언급한 여러 하드웨어에서 모두 실제 실행해 결과를 비교한 것은 아니다.

5. 홈페이지 assistant, IDE 플러그인, Studio, 문서 assistant

이 세션에는 비슷한 이름의 AI 도구가 여러 개 나온다. 기능과 쓰이는 맥락으로 구분하면 혼동이 줄어든다.

도구	영상의 활용	사용자가 확인할 대상
홈페이지 assistant	자연어로 Grover 프로그램을 만들고 회로·모델·실행으로 이동	생성 명세, 코드, 회로, 결과가 같은 문제를 가리키는지
Quantum Engineering Agent 플러그인	VS Code·터미널 등 기존 개발 환경에서 모델링·합성·실행·분석을 보조	요청한 변경 범위, 호출된 도구, 생성 코드와 실행 로그
Classiq Studio agent	준비된 브라우저 기반 VS Code 환경에서 노트북 생성·수정·설명	환경 의존성, 원 논문과 구현의 대응, 고전 기준값
공개 문서 assistant	사용자 가이드·예제 검색, 코드의 변수·함수 설명	연결된 문서가 실제 설명을 뒷받침하는지

25:45-27:31에는 플러그인의 모델링·합성·실행·분석·후처리 기능과 문서 서버 연결을 소개한다. 자동자막에서 `MCP`가 여러 방식으로 오인식되지만 문맥상 문서 도구를 연결하는 서버를 뜻한다. 27:36-28:34에는 Studio의 Python·Classiq·의존성이 준비돼 있어 여러 사람의 입문 환경을 맞추는 부담을 줄인다고 설명한다.

논문 구현 보조 사례는 어디까지 확인됐는가

28:38-31:29에서 Erik은 포트폴리오 최적화, DMET를 이용한 분자 문제 분할, 암호화된 양자 복제와 관련된 논문을 구현해 본 경험을 말한다. "약 90%는 수정 없이 진행된다"는 취지의 설명은 **발표자의 경험담**이다. 표본·평가 절차·실패 판정이 제시된 재현성 실험이나 신뢰도 통계가 아니다. 이 노트도 해당 원 논문들의 구현 결과를 독립 검증하지 않았다.

특히 "암호화하면 한 번 복제할 수 있다"는 짧은 설명은 원 논문과 프로토콜 조건을 확인하기 전 일반적인 no-cloning 정리를 수정한 결과로 받아들일 수 없다. 이 세션에는 정확한 논문 식별자와 가정이 자막으로 충분히 남지 않는다. 따라서 사례의 존재만 기록하고 세부 이론은 미확정으로 둔다.

31:38-32:28에는 세 ion-trap 모듈에서 inverse QFT를 구성하고 모듈 간 양자 통신을 줄이는 노트북 사례를 든다. 발표자는 모듈별 2큐비트 게이트와 모듈 간 비율을 요약한 결과를 보여 줬다고 설명하며 `72%`라는 값을 읽는다. 분모·비교 기준·모듈 배정·실제 통신 모델을 이 자료만으로 확정하기 어려우므로 "통신을 72% 감소시켰다"로 바꾸어 쓰지 않는다.

전문가의 작업 시간을 줄이는 사례

38:48-40:45에는 QAOA 구현 방식을 입력 크기별로 비교한 노트북을 소개한다. 비교 지표로 합성 시간·컴파일 시간·transpiled depth를, 후속 비교로 하드웨어 연결성별 결과를 언급한다. 발표자는 약 35개 사례를 분석한 뒤 좋은 사례를 골라 하드웨어 비교를 했다고 말한다. 따라서 이 장면은 자동 분석 노트북을 만드는 활용 예로 읽으며, 선택된 사례의 결과를 전체 문제군에 대한 우위로 일반화하지 않는다.

6. 녹화물에서 확인되는 실제 청중 Q&A

질문은 영어 발화를 한국어로 요약한 것이며 직접 인용문이 아니다. 녹화물은 채팅창 전체를 보존한 원장이 아니므로 아래는 **진행자가 소리 내어 읽거나 명시적으로 받은 질문만** 정리한 것이다.

Q1. Classiq를 쓰려면 어느 정도 양자 배경이 필요한가?

19:48-22:02

답변: Erik은 전문가 수준까지 필요하지 않지만 Grover 탐색, amplitude amplification처럼 주요 알고리즘이 무엇을 하고 어디에서 이득을 낼 수 있는지의 기초는 알아야 한다고 답한다. 기업의 기존 AI·소프트웨어 개발자가 알고리즘을 배우면서 전환할 수 있다는 설명이다. Jack은 agent로 설명이 포함된 튜토리얼과 Jupyter 노트북을 만들어 학습에 활용한 경험을 보탠다.

해석: 하드웨어 펄스·게이트 분해 전문성이 줄어든다는 취지다. 문제를 잘못 고르거나 입력 영역을 잘못 정의한 오류까지 자동으로 사라진다는 답은 아니다.

Q2. AI assistant는 일반 ChatGPT 사용과 무엇이 다른가?

22:04-24:13

답변: Erik은 Qmod·공식 예제·문서에 대한 맥락과 도구 연결, 실행 오류를 읽고 몇 차례 수정하는 반복 절차를 차이로 든다. 높은 수준의 언어 표현도 생성의 부담을 줄인다고 설명한다. 특정 양자 언어 비교 논문을 들어 Qmod의 표현력을 언급한다.

필기자 검토: 이것은 당시 자사 제품의 설계와 기대효과를 설명한 답변이다. 모든 범용 AI 제품의 현재 성능을 측정한 비교는 아니다. 관련성이 높은 Corrales-Garro 외의 언어 비교 논문은 네 가지 기초 알고리즘의 코드 길이·순환 복잡도·Halstead 지표를 비교한다. 이런 정적 지표의 좋은 결과를 모든 앱의 개발 생산성, 실행 효율, AI 정확도에 대한 보증으로 확대할 수 없다. 영상에서 논문 제목을 완전히 읽지는 않아 이 논문과의 동일성은 문맥에 근거한 대응이다.

Q3. AI는 코드만 쓰는가, 양자 개념도 설명하는가?

32:50-35:59

답변·시연: 개념 설명도 가능하다고 답한 뒤 Jack이 공개 문서 assistant를 보여 준다. 사용자 가이드에서 Grover 관련 내용을 찾아 달라는 요청에 controlled phase operation, 양자 수와 산술, Grover workshop·그래프 색칠 튜토리얼을 연결한다. 이어 예제 코드를 요청하고 각 변수·함수의 설명과 관련 문서 링크까지 받아 본다.

학습 포인트: 한 번에 정답 설명을 받는 데서 멈추지 않고 “어느 문서에 근거하는가”, “이 함수가 이 알고리즘에서 왜 필요한가”로 질문을 좁히는 흐름이다.

Q4. Classiq 사용 튜토리얼은 어디에 있는가?

36:01-36:53

답변: 문서의 Getting Started를 보여 주며 Qmod 문법, 첫 알고리즘, 간단한 프로젝트로 이어지는 튜토리얼을 안내한다. Jack은 교육 세션에서 활용한 경험을 말한다. 현재 접근은 Classiq Getting Started와 공식 라이브러리를 출발점으로 삼을 수 있다.

Q5. 좋은 개발자는 있지만 양자 전문가는 없는 팀은 어떻게 시작해야 하는가?

36:56-40:53

답변: 기존의 활발한 소프트웨어·AI 개발팀이 적절한 출발점이라고 Erik이 답한다. 주요 알고리즘이 어떤 문제에 맞는지 배우는 과정은 필요하지만, 저수준 하드웨어 구현 부담을 줄일 수 있다고 설명한다. 이미 양자 전문성이 있는 사람에게도 QAOA 구현 비교, 입력 크기별 자원 분석, 하드웨어별 컴파일 분석을 빠르게 만드는 용도가 있다고 덧붙인다.

실무적으로 읽기: 도메인 담당자가 목표와 데이터 의미를, 개발자가 모델과 검증을, 양자 알고리즘 담당자가 적용 조건과 자원을 검토하는 역할이 필요하다. 이 역할 구분은 노트 작성자의 보충이며 발표자가 채용 인원이나 필수 조직도를 제시한 것은 아니다.

별도 기록: 질문 내용이 녹화물에 남지 않은 청중 개입

28:57-29:17에 Erik이 Mark의 질문에 감사하며 논문 재현 용도를 이야기한다. 질문 전문을 소리 내어 읽지 않았으므로 "청중이 정확히 어떤 문장으로 무엇을 물었다"는 항목은 복원하지 않았다.

또한 18:45-19:24의 "한 번 생성한 뒤 수정 질문을 계속할 수 있는가"는 Erik과 Jack의 대화다. Jack은 oracle의 `*`를 `+`로 바꾸는 식의 반복 수정을 예로 든다. 이를 청중 질문 수에 합산하지 않는다.

7. 복습 질문과 풀이 — 노트 작성자가 추가

아래는 실제 청중 질문이 아니다. 데모의 구조를 스스로 점검하기 위한 연습이다.

문제 1. `x,y∈{0,1,…,7}`이라면 `xy=6`의 해는 몇 개이며 초기 균일 상태에서 정답 확률은 얼마인가? 두 차례 Grover 반복 후에는 어떤가?

풀이. 순서를 구분한 해는 `(1,6),(2,3),(3,2),(6,1)` 네 개다. 후보가 64개이므로 `M/N=1/16`, 초기 성공확률은 6.25%다. `θ=arcsin(1/4)`이고 두 번 반복하면 `sin²(5θ)≈0.9084`다. 입력 영역과 반복 횟수를 정해 만든 계산 예로, 영상의 실제 비트폭이나 Grover 반복 횟수를 추정한 것이 아니다.

문제 2. 히스토그램에서 네 정답 봉우리가 보이면 "모든 해를 항상 반환한다"는 결론을 내려도 되는가?

풀이. 안 된다. 각 shot은 결과 하나를 반환한다. 정답 아닌 상태의 잔여 확률과 유한 shots 때문에 정답 누락이 생길 수 있다. 해의 전체 집합을 요구한다면 누락 확률·중복 수집·종료 조건을 추가해야 한다.

문제 3. depth가 약 1,900에서 약 1,300으로 줄면 실제 QPU 실행 시간이나 오류도 정확히 같은 비율로 줄어드는가?

풀이. 아니다. 동일한 논리 depth에서도 게이트 종류별 시간, 연결성에 따른 라우팅, 2큐비트 게이트 수, 측정·리셋·대기 시간이 다르다. 그 비율은 이 데모의 논리 회로 지표에서만 계산할 수 있다. 다른 하드웨어에서는 그 하드웨어의 게이트 집합으로 다시 합성한 비용을 비교해야 한다.

문제 4. 생성 코드가 정상 실행되는데 왜 별도의 검증이 필요한가?

풀이. 타입·문법·합성 가능성은 요구의 정확성과 다르기 때문이다. 예를 들어 음수를 원했는데 unsigned 변수를 만들었거나, `xy=6` 대신 `x+y=6`을 구현했어도 회로 자체는 정상 실행될 수 있다. 작은 입력을 전수 검사하는 고전 oracle, 기대 해 목록, 유효하지 않은 입력, 측정 분포를 함께 확인하면 이런 오류를 찾기 쉽다.

문제 5. 논문 구현을 에이전트에 맡길 때 비교해야 할 최소 항목은 무엇인가?

풀이. 원 논문의 모형·가정·입력 크기·정밀도·알고리즘 변형을 코드와 대응시키고, 논문의 표·식에 있는 작은 기준값을 재계산한다. 합성 성공·시뮬레이터 성공·실제 QPU 실행을 구별하며, 논문과 다른 입력이나 근사를 사용했다면 그 차이를 결과 옆에 적는다. 이 항목은 이 세션의 재현 성공률 주장을 승인하는 검사가 아니라 후속 실습의 설계다.

8. 다음 회차로 이어지는 질문

이 회차가 보여 준 가장 구체적인 결과는 작은 탐색 문제를 **자연어 → 높은 수준의 모델 → 합성 회로 → 시뮬레이터 결과**로 잇는 데모다. 금융 회차에서는 여기서 한 단계 앞선 질문, 즉 "우리 업무를 어떤 수학 문제로 써야 양자 알고리즘을 선택할 수 있는가"로 이동한다. 두 회차를 연결해 읽으면 AI 도구의 편의성만으로는 양자 활용 사례를 선택할 수 없고, 도메인 문제·알고리즘 가정·자원 비용을 함께 판단해야 함을 이해할 수 있다.

출처와 확인 범위

- **S03-V:** 공식 다시보기. 영어 자동자막 전 구간 수집, 상대시각 대조. 정확한 코드 전체·화면의 작은 수치·하드웨어별 실행을 독립 검증하지 않음.
- **S03-E:** 공식 행사 페이지. 행사일·발표자·예고된 범위를 확인하는 자료. 실제 발언·결과의 대체 근거로 사용하지 않음.
- **S03-Q:** Vax 외, Qmod, arXiv:2502.19368. 언어 설계 설명을 보충.
- **S03-G:** Brassard 외, Quantum Amplitude Amplification and Estimation. Grover 회전과 보충 계산의 근거.
- **S03-D:** Classiq Grover workshop. oracle·반사 구성의 공식 실습.
- **S03-P:** Corrales-Garro 외, 언어 표현력 비교, arXiv:2504.08876v2. 논문 범위와 지표를 확인. 발표자가 언급한 논문과의 대응은 문맥에 근거함.

검토 상태: 자동자막 기반 전 구간 내용 검토 및 주요 개념 보충 완료. 자막 오인식인 Classiq·Qmod·Erik Garcell 등의 이름은 공식 자료에 맞춰 표기를 정리했다. 논문 재현 사례의 원문 식별, 세 ion-trap 예의 정확한 지표, 최적화 회로의 전체 gate count는 미확정이다. 원 자막·메타데이터는 비공개 조사 자료로 보존하며 본문에 재배포하지 않는다.

04. 금융 업무를 양자 알고리즘으로 옮기는 방법

대상 세션: Classiq Quantum Office Hours: Quantum Computing for Finance — From Use Case to Application
행사: 2026년 8월 27일 · **다시보기 공개:** 2026년 10월 20일 · **길이:** 44분 48초
발표·진행: Dr. Erik Garcell, Director of Quantum Solutions Architecture; Dr. John (Jack) Glennon, Quantum Applications Engineer
정리 기준: 2026년 10월 22일. 공식 다시보기의 영어 자동자막 전 구간, 공식 행사 소개, 주요 알고리즘의 원 논문과 공식 문서를 대조했다. 시각은 녹화물 상대시각이다. 실제 발언, 수학적 보충, 발언의 교정과 미확정 사항을 구별한다. 코드·데이터·QPU 성능은 독립 재현하지 않았다.

공식 다시보기 · 공식 행사 소개

이 세션의 중심 질문

금융 기관이 양자를 검토할 때 “어느 양자 알고리즘을 쓸 것인가”부터 시작하면 업무의 목적과 구현이 어긋날 수 있다. 이 회차는 **업무 문제 → 수학적 정식화 → 양자 계산의 적합성 → 알고리즘과 하드웨어 선택**의 순서를 제안한다. Asian option pricing과 자금세탁 탐지를 예로 들고, 후반에는 European call option의 기대 보수를 QAE로 추정하는 원리를 설명한다.

이 회차는 금융 전반에서의 양자 우위를 실험으로 입증한 발표가 아니다. 과제 선택의 절차, 알고리즘의 이론적 질의 복잡도, Classiq로 모델을 작성·합성하는 방법, GPU를 활용한 양자 회로 모사 결과가 한 영상에 들어 있다. 각 결과가 무엇을 측정하는지 구별해야 뒤의 수치도 올바르게 읽을 수 있다.

1. 회차의 전개와 다시보기 길잡이

다시보기	내용	핵심 질문
00:06-05:26	발표자·플랫폼 소개, 청중 경험 설문	누가 어떤 단계에서 이 도구를 쓰는가?
05:29-07:25	금융 활용 동기, 시장 보고서와 역량 준비	장기 전망과 이미 달성한 결과를 구별할 수 있는가?
07:27-13:57	업무에서 알고리즘으로 옮기는 4단계, 세 문제군	우리 문제가 실제로 어떤 계산인가?
14:00-18:43	첫 Q&A 세 질문	가속기 비유, QAOA 우위, 필요한 논리 큐비트
18:48-23:07	Asian option, IQAE, Brownian motion·Karhunen-Loève 사례	경로의 기대 보수를 어떻게 표현하는가?
23:19-24:15	AML의 표·그래프 표현과 quantum kernels	동일한 업무라도 데이터 표현이 어떻게 달라지는가?
24:18-32:14	두 번째 Q&A 두 질문	모델 의도 보존, 고전 최적화와의 비교
32:16-35:24	양자 적용의 적합성·한계, 정리	모든 금융 문제에 양자를 넣어야 하는가?
35:25-40:59	European call·QAE·IQAE, GPU 가속 사례	기대 보수를 확률로 옮긴 뒤 무엇을 추정하는가?

다시보기	내용	핵심 질문
41:03-43:23	AI 플러그인으로 두 연결성·게이트 집합의 합성 비교	실행할 하드웨어를 어떻게 선택하는가?
43:25-44:43	가속 결과 회고, 종료	성능 주장의 조건은 무엇인가?

2. 업무 요구를 네 단계로 분해하기

2.1 먼저 업무에서 바뀌어야 할 값을 정한다

09:19-10:26에서 Erik은 비용이 크거나 경쟁력이 필요한 실제 업무를 골라 수학 문제로 바꾸는 과정을 설명한다. "포트폴리오를 개선한다"만으로는 계산이 정해지지 않는다. 기대 수익, 위험, 거래 비용 중 무엇을 목적함수에 넣을지, 얼마의 시간과 어느 정도의 오차를 허용하는지 정해야 한다.

필기자 보충 예. "주어진 예산에서 30개 종목을 고르며 변동성을 줄인다"는 요구라면 이진 선택 변수 `z_i∈{0,1}`, 수익률 벡터 `μ`, 공분산 행렬 `Σ`, 선택 수 제약 `Σ_i z_i=30` 같은 구조가 생긴다. 반면 "이미 정한 포트폴리오의 손실분포에서 99% 분위수를 계산한다"는 요구는 기대값·확률·분위수 추정으로 옮겨 간다. 둘 다 금융이라는 이유만으로 같은 양자 알고리즘을 선택하지 않는다.

2.2 업무 이름을 수학 구조로 바꾼다

11:24-13:45의 주된 구분은 다음과 같다. 발표자는 이것이 완전한 목록이 아니고, 같은 업무가 여러 방식으로 정식화될 수 있다고 덧붙인다.

수학 문제군	발표에서 든 업무	발표에서 연결한 접근	실제로 확인할 조건
조합 최적화	포트폴리오 선택, 자산 배분, 거래 실행	QAOA 등 변분·혼합 접근	이산 변수·제약 구조, 고전 solver 성능, 해 품질
통계적 샘플링·적분	파생상품 가격, 위험 평가, 신용 위험, 스트레스 테스트	QAE·IQAE 등	분포 준비·보수 함수 비용, 오차·신뢰수준, 회로 깊이
분류·이상 탐지	신용 점수, 자금세탁·사기 탐지	quantum kernel 등	데이터 인코딩, 학습 표본, 강한 고전 기준과 일반화

이 표는 **후보를 고르는 지도**다. 특정 업무 이름 옆에 양자 알고리즘 이름을 놓았다고 해서 우위가 확보되는 것은 아니다.

2.3 양자에 적합한 부분과 고전에 남길 부분을 가른다

10:03-11:23에서 발표자는 양자가 노트북 전체를 대체하는 것이 아니라 특정 종류의 계산에 쓰이는 연산 장치라는 관점을 권한다. 32:16-35:24에서는 모든 문제를 양자 하드웨어에 넣는 것이 적절하지 않으며, 데이터·수학 구조·하드웨어를 함께 봐야 한다고 다시 설명한다.

필기자 검토. 이 원칙은 타당하지만, 영상의 "NP-hard이면 양자에 적합하다", "고차원 데이터이면 양자 ML이 더 잘 구분한다"는 표현은 충분조건이 아니다. 실제 비교에는 입력 로딩, 학습·최적화, 측정 횟수, 통신·대기, 결과 오차까지 들어간다. 특히 고전 알고리즘의 실패가 관측됐더라도 양자 방법의 성공이 자동으로 따라오지 않는다.

2.4 목표 하드웨어에 맞는 알고리즘을 고른다

영상은 NISQ, 초기 내결함성, 완전 내결함성의 세 시기를 구별한다. NISQ에서는 짧은 변분 회로와 고전 피드백을 활용하고, 더 긴 coherent 연산이 필요한 알고리즘은 오류 수정이 가능한 하드웨어를 염두에 둔다는 취지다. 10:52-11:23에 나오는 2028-2030 같은 연도는 **발표자의 당시 미래 전망**이며 이 노트가 검증한 출시 일정이 아니다.

기업의 후속 작업은 "몇 큐비트짜리 장치가 나오면 된다"보다 구체적이어야 한다. 입력 크기·정밀도·허용 오류를 정한 모델을 합성하고, logical width·depth·비Clifford 자원과 오류 정정 비용 등 필요한 지표를 계산해야 한다. 영상도 입력 데이터와 허용오차에 따라 요구 큐비트 수가 달라진다고 Q&A에서 강조한다. 논리 큐비트와 물리 큐비트를 같은 수로 읽지 않는다.

3. 사례 A: Asian option은 왜 경로의 적분 문제인가

18:48-20:34에서 첫 사례로 Asian option pricing을 든다. 만기 한 시점의 가격만 보는 European option과 달리, Asian option의 보수는 여러 관측시점의 평균 가격에 의존한다. 따라서 "하나의 가격을 준비하고 보수를 계산"하는 것을 넘어 **확률적 경로 전체 또는 그 요약량**을 다루어야 한다.

다음 식은 영상을 이해하기 위한 **필기자 보충**이다. 관측 가격이 `S_{t_1},…,S_{t_L}`이고 행사가가 `K`일 때 산술평균 Asian call의 단순한 보수는

$$\bar{S} = \frac{1}{L}\sum_{\ell=1}^{L} S_{t_\ell}, \qquad g = \max(\bar{S} - K, 0), \qquad V_0 = e^{-rT}\mathbb{E}_{\mathbb{Q}}[g]$$

이다. `r`은 이 식에서 가정한 상수 무위험 이자율, `T`는 만기, `Q`는 가격 모형에 맞는 위험중립 측도를 뜻한다. 이 식은 투자 전략을 추천하는 식이 아니라, 영상의 "업무를 기대값 추정으로 바꾼다"는 설명을 구체화한 것이다.

예를 들어 geometric Brownian motion을 가정하면

$$S_t = S_0 \exp\left((r - \tfrac{1}{2}\sigma^2)t + \sigma W_t\right)$$

로 경로를 표현한다. 여기서 `W_t`는 Brownian motion, `σ`는 변동성이다. 양자 계산에서는 분포를 양자 상태에 준비하고, 각 경로의 평균과 보수를 가역적으로 계산한 다음, 보수의 기대값을 특정 큐비트의 측정확률로 옮기는 절차가 필요하다. QAE는 그 확률 추정을 가속할 후보이지 경로 준비와 산술 비용을 삭제하는 연산은 아니다.

Karhunen-Loève 확장과 차원 줄이기

21:46-23:07에서 Erik은 J.P. Morgan 연구진의 *Quantum option pricing via the Karhunen-Loève expansion*을 구현한 경험과 Brownian·fractional Brownian motion을 Asian option 절차에 연결한 예를 소개한다. 자동자막에서 논문명과 `Brownian`, `Chebyshev`가 깨져 있어 제목은 원 논문과 대조했다. Prakash 외, arXiv:2402.10132.

필기자 보충. KL 확장은 확률 과정을 적절한 기저 함수와 확률 계수들의 합으로 나타내고 유한 개 항으로 절단하는 방법이다. 경로의 모든 시점을 독립적인 숫자로 저장하기보다 구조를 이용할 수 있지만, 절단 오차·시간 이산화·숫자 표현·분포 로딩의 비용이 생긴다. 원 논문은 관측시점 수 `T`와 오차 `ε`를 구별한 복잡도를 분석하며, 그 논문의 전체 옵션 가격 알고리즘을 단순히 "`1/ε` 비용"으로 요약할 수 없다. QAE의 질의 복잡도와 응용 전체의 복잡도는 다르다.

fractional Brownian motion을 넣었다는 발표도 모형의 정당성과 별개의 구현 사례다. 금융 가격 계산에서 표준 Brownian 모형을 그대로 대체해도 되는지, 위험중립 가격이 어떻게 정의되는지는 별도 모형 검토가 필요하다. 이 영상에서는 그 가정까지 전개하지 않는다.

4. 사례 B: 자금세탁 탐지는 표현부터 달라진다

23:19-24:15에서 두 번째 사례로 AML을 든다. 동일한 거래 자료도 행이 거래인 표로 쓸 수 있고, 계좌·거래관계를 그래프로 만들 수도 있다. 발표자는 이상 거래의 경로나 구조를 찾는 최적화, 또는 그래프에서 비정상 패턴을 분류하는 머신러닝 방식이 가능하며 quantum kernel을 자주 본다고 설명한다.

필기자 보충. kernel 방식의 한 예는 데이터를 상태 `|ϕ(x)⟩`로 인코딩하고

$$k(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2$$

같은 유사도를 측정한 뒤 고전 분류기에 주는 것이다. 이 경우 핵심은 "공간이 고차원이다"가 아니라, 선택한 특징맵이 업무에 필요한 차이를 보존하고 일반화에 도움이 되는가이다. 거짓 양성, 데이터 누수, 클래스 불균형, kernel 추정 오차와 학습 표본의 관계를 비교해야 한다. 이 식과 평가 항목은 노트의 보충이며 영상에서 AML 정확도·재현율·QPU 우위 수치가 제시된 것은 아니다. 관련 공식 예제는 Classiq Quantum Kernels and Support Vector Machines에서 확인할 수 있다.

5. European call에서 QAE의 역할을 단계별로 이해하기

35:25-40:59에 Jack은 옵션 가격 사례로 알고리즘을 구체화한다. 영상의 설명 순서는 분포와 payoff, 고전 Monte Carlo 오차, QAE의 확률 인코딩, Grover 회전, 추정, canonical QAE와 IQAE의 차이다.

5.1 분포와 보수를 구분한다

European call의 보수는 만기 자산가격 `S_T`에 대해

$$g(S_T) = \max(S_T - K, 0)$$

이다. 영상에서는 `ln S_T`가 정규분포를 따른다는 lognormal 가격 모형을 설명한다. **자산가격 자체가 정규분포를 따른다**고 바꾸면 음수 가격을 허용하는 다른 모형이 된다. 36:21-36:59.

필기자 보충으로 이산화한 가격을 `s_j`, 그 확률을 `p_j`라고 놓자. `Σ_j p_j=1`이어야 한다. 유한 구간에서 `0≤g(s_j)≤C`가 되도록 상수 `C`를 정하고 `f_j=g(s_j)/C∈[0,1]`로 정규화한다. 분포와 보수를 준비하는 연산 `A`는 개념적으로

$$A|0\rangle = \sum_j \sqrt{p_j}|j\rangle \left(\sqrt{1 - f_j}|0\rangle + \sqrt{f_j}|1\rangle \right)$$

을 만든다. 마지막 큐비트가 `1`일 확률은

$$a = \sum_j p_j f_j = \frac{\mathbb{E}[g]}{C}, \qquad V_0 \approx e^{-rT} C a$$

이다. 이 상태 준비와 payoff 인코딩을 한 덩어리로 보고 QAE가 `a`를 추정한다. `a`는 **측정확률**이다. 상태벡터에서 좋은 부분공간에 붙는 진폭은 `√a`이므로 두 값을 혼용하지 않는다. 옵션 가격의 단위는 `C`와 할인 계수를 다시 곱할 때 복원된다. 이 수식은 발표의 "expected value를 amplitude에 넣는다"는 표현을 엄밀하게 풀어 쓴 노트다. Brassard 외, Classiq 옵션 가격 공식 예제.

5.2 good/bad 상태는 payoff가 0인지 여부만을 세는 것이 아니다

영상 37:34-38:35은 good·bad 상태를 이용해 설명한다. 위 식에서 good은 마지막 보조 큐비트가 `1`인 부분공간이다. 경로·가격별로 그 큐비트가 `1`일 **확률을 보수 크기에 비례**하게 해야 평균 보수를 얻는다. 단순히 `S_T>K`인지만 표시하면 추정되는 것은 옵션이 행사될 확률이지 기대 보수 자체가 아니다.

이 차이는 코드 검토에서 중요하다. oracle이 `price>strike`라는 비교만 하는지, `max(price-strike,0)`의 크기를 정규화해 넣는지 확인해야 한다. 분포와 payoff를 별도 함수로 구성하는 고수준 모델은 이 검토를 쉽게 하지만 정규화와 자료형의 의미까지 자동 보증하지는 않는다.

아래의 작은 교육 예에서는 같은 가격별로 확률과 지급액을 짝지어 읽는다. 행사가 110일 때 가격 120과 140에서 각각 10과 30을 지급하므로, 행사될 확률 0.7과 기대지급액 13은 다른 양이다. 이 입력의 전체 손계산은 보충 해설에서 이어서 확인할 수 있다.

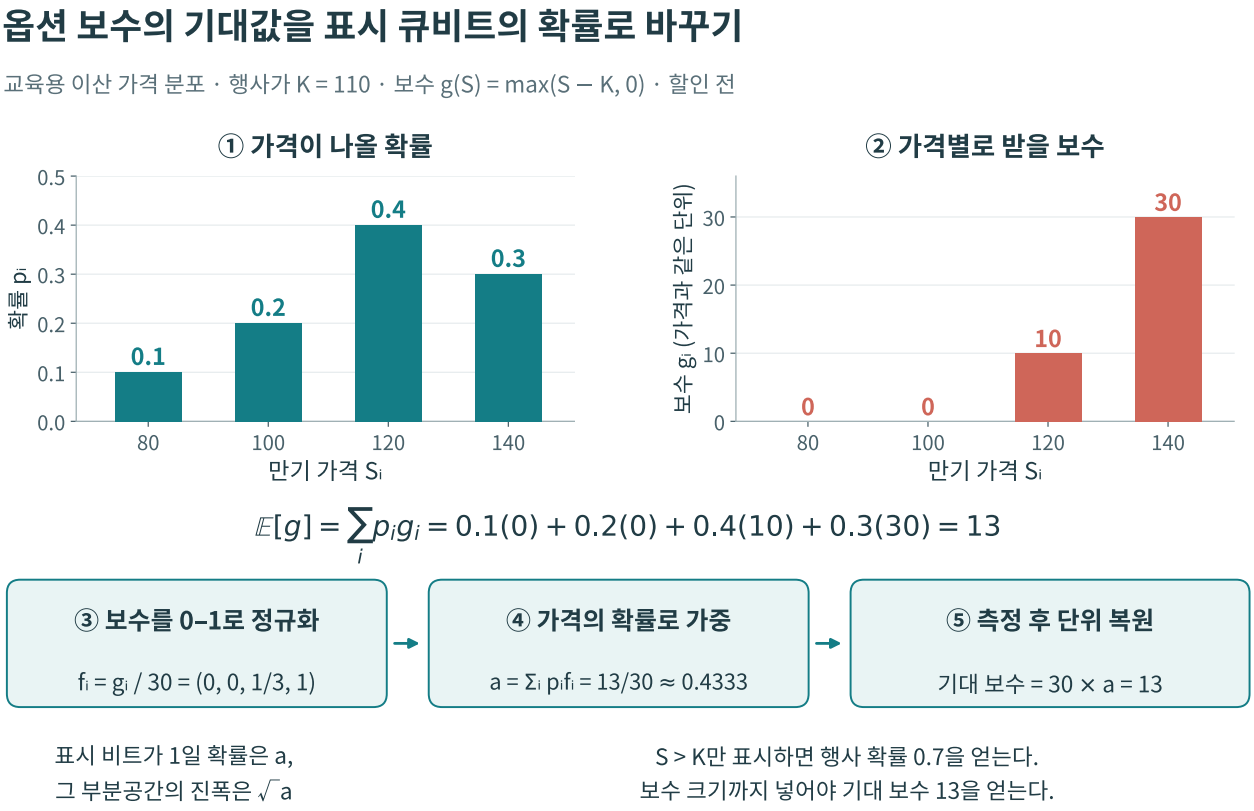


그림 4-1. **지급액의 크기까지 표시 비트의 확률에 넣는다.** 위 두 패널은 같은 네 가격의 확률과 지급액이며 아래는 지급액을 30으로 나눠 정규화하고 확률로 가중한 뒤 단위를 복원하는 순서다. 마지막 비트가 1일 확률은 13/30이고 그 부분공간의 진폭은 그 제곱근이다. 실제 시장 데이터나 발표의 64점 로그정규 예제를 그린 것이 아닌 독립 손계산 도식이다.

5.3 Grover 연산은 미지의 확률을 각도로 바꾸어 읽게 한다

정규화된 상태를

$$A|0\rangle = \cos\theta|B\rangle + \sin\theta|G\rangle, \qquad a = \sin^2\theta$$

라고 놓자. 적절한 반사들의 곱으로 만든 Grover 연산 `Q`를 `k`회 적용한 뒤 마지막 큐비트를 측정하면

$$P_k(1) = \sin^2((2k+1)\theta)$$

를 얻는다. 더 큰 `k`는 `θ`에 대한 변화에 더 민감하지만 사인 제곱의 주기성 때문에 가능한 각도가 여러 개 생긴다. IQAE는 앞서 얻은 신뢰구간을 이용해 다음 `k`를 고르고 각도 범위를 좁힌다. 단순히 무조건 큰 `k`를 선택하거나 모든 회로의 결과를 동일하게 평균하는 알고리즘은 아니다. 이 부분은 발표의 회전 그림을 수식으로 재구성한 설명이다. Grinko 외, Iterative Quantum Amplitude Estimation, §II-III.

5.4 Canonical QAE와 IQAE는 무엇을 교환하는가

방식	영상의 설명	이해를 위한 보충
Canonical QAE	보조 레지스터와 깊은 회로를 사용하는 QPE 기반 접근	controlled Grover powers와 역 QFT가 필요하고 위상을 측정해 `a`로 변환
IQAE	여러 Grover 거듭제곱 회로의 실행·측정 통계와 고전 후처리를 반복	QPE용 보조 레지스터·역 QFT를 피하지만 고차 Grover 회로의 깊이와 순차적 적응 비용은 남음

38:35-40:03. 영상은 고수준 Qmod의 payoff 함수와 IQAE 내장 함수를 연결하는 예를 든다. 정확한 모델을 간결하게 쓰고 합성·실행 단계를 재사용하는 개발상의 이점이다. IQAE가 canonical 방식보다 필요한 일부 자원을 줄인다는 사실만으로 대규모 금융 업무가 현재 noisy QPU에서 유리하게 실행된다고 결론내릴 수 없다.

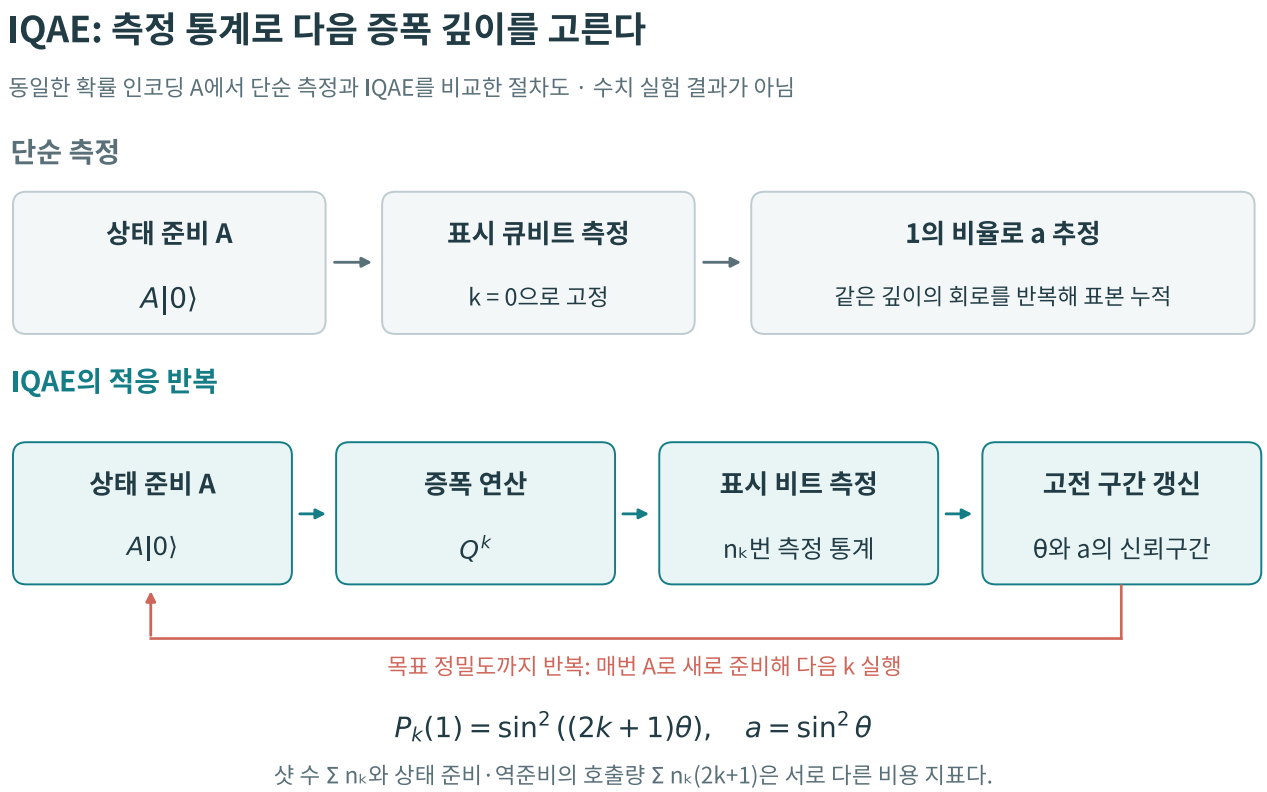


그림 4-2. **IQAE에서 고전 피드백은 무엇을 바꾸는가.** 위는 같은 확률 인코딩을 증폭 없이 반복 측정하는 경로이고, 아래는 측정 통계로 다음 Grover 반복 횟수 k를 고르는 경로다. 매 회로는 상태를 새로 준비한다. 한 샷 안에 들어가는 준비·역준비 호출량이 깊이에 따라 달라지므로 샷 수와 질의 수를 따로 센다. IQAE 원리의 설명용 재구성이며 특정 성능 측정값을 그리지 않았다.

5.5 Monte Carlo와 QAE의 오차 차수를 정확히 읽기

19:47-20:24의 Erik 발언에는 고전 Monte Carlo를 `1/N²`처럼 말하는 부분이 있다. 뒤이어 "오차를 절반으로 줄이려면 고전에서는 4배, 양자에서는 2배"라고 설명하며, Jack은 36:44-37:30에서 올바른 `1/√M` 대 `1/M` 차수를 말한다. 이 노트는 다음과 같이 교정한다.

유한한 분산 `σ_g²`를 갖는 독립 표본의 평균에 대해

$$\mathrm{RMSE}(\widehat{\mathbb{E}[g]}) = \frac{\sigma_g}{\sqrt{N}}.$$

따라서 목표 오차 ε를 맞추는 데 일반적인 Monte Carlo는 $N=O(\varepsilon^{-2})$개의 표본이 필요하다. 이상적인 QAE는 oracle 질의 수를 세면 $O(\varepsilon^{-1})$이고, IQAE에는 신뢰도·정밀도에 따른 로그 인자가 붙는다. **오차율은 $N^{-1/2}$ 대 N^{-1}**, **오차를 맞추는 비용은 ε^{-2} 대 ε^{-1}**이다. 두 표현을 뒤섞으면 영상의 잘못된 지수 표현이 나온다. Grinko 외, §I-III.

또한 영상의 "100만 표본 대 약 1천"은 차수를 직관적으로 보여 주는 예다. 같은 가격 오차·신뢰수준·상수·oracle 비용을 맞춘 실측 시간 비교가 아니다. QAE의 Q^k 한 회로는 k번의 Grover 연산과 상태 준비·역연산을 포함하므로, shot 수와 oracle 질의 수를 동일시하지 않는다.

마지막으로 실제 가격 오차는 추정 통계만으로 결정되지 않는다. 분포 꼬리 절단, 가격·시간 격자, payoff 근사·고정소수점, QAE 통계, 실제 장치의 오류가 각각 기여한다. 추정 오차만 줄여도 모형과 이산화 오차는 남는다.

6. Classiq 합성과 GPU 가속 사례의 정확한 범위

고수준 모델에서 하드웨어별 회로 비교로

41:03-42:42에서 Jack은 이미 있는 European call 예제를 대상으로 "나머지는 유지하고 두 종류 하드웨어에 맞춰 합성하며 네이티브 2큐비트 게이트 수를 줄여 달라"고 AI 플러그인에 요청한다. 하나는 all-to-all 연결과 `RX, RZ, RXX` 게이트 집합, 다른 하나는 heavy-hex 연결과 다른 게이트 집합이다. 코드에 연결성·transpilation 옵션·최적화 목표를 넣어 결과를 비교하는 과정을 설명한다.

여기서 비교하는 대상은 **동일한 옵션 가격 모델의 하드웨어별 컴파일 결과**다. 실제 두 QPU에서 옵션 가격을 측정해 정확도·비용까지 평가했다는 보고는 아니다. 2큐비트 게이트 수는 유용한 비용 지표지만 vendor의 요금이 언제나 그 개수만으로 결정된다는 뜻도 아니다. 샷·태스크·시간·장치별 과금 기준은 별도로 확인해야 한다.

77-120배는 무엇의 가속인가

40:23-40:59에서 Classiq Qmod 모델을 합성해 CUDA-Q kernel에 통합하고 Oracle Cloud Infrastructure에서 실행한 사례를 소개한다. 발표자는 28·29 qubit 예에서 일반적인 CPU workflow에 비해 **77-120배** 가속을 보고하며 GPU에서 양자 회로와 고전 로직을 처리한다고 설명한다. 이 값은 **고전 컴퓨터가 양자 계산을 모사하는 워크플로의 CPU/GPU 비교**다. QPU가 고전 옵션 가격 solver를 이긴 결과가 아니다.

43:25-43:49의 회고에는 "60여 분에서 2분 반"이라는 다른 시간 표현도 나온다. 같은 문제 크기·동일한 비교 기준인지 이 세션의 자막만으로 확인할 수 없으므로 두 표현을 한 계산으로 합치거나 그 비율을 검증값으로 쓰지 않는다. 상세 조건과 후속 데모는 10월 15일 Classiq·NVIDIA 회차에서 따로 검토한다.

7. 실제 청중 Q&A — 발표자의 답변과 학술 검토를 나누어 기록

아래 다섯 질문은 진행자가 녹화물에서 읽어 준 청중 질문이다. 한국어는 발화의 요약이며 직접 인용이 아니다. 채팅에 있었지만 읽지 않은 질문이나 종료 뒤의 답변은 이 자료에서 확인하지 않았다.

Q1. 양자는 GPU처럼 특정 계산에 쓰는 도구인가?

14:09-15:02

발표자의 답변: CPU·GPU에 QPU가 더해지는 것으로 생각하면 유용하며, 개발자는 어떤 수학 문제에 어떤 장치를 사용할지 알아야 한다. Jack도 동의한다.

필기자 보충: 이것은 연산 자원을 선택하는 실무적 비유다. GPU는 고전 계산을 병렬화하고 QPU는 양자 상태·간섭·측정을 사용하므로 계산 모형까지 같은 것은 아니다. 뒤 31:34-32:14에서 Jack도 이 차이를 강조한다.

Q2. 최적화에서 QAOA가 고전 방법보다 나을 것으로 기대할 근거가 있는가?

15:12-16:33

발표자의 답변: 활발한 논쟁 대상이라고 인정한다. QAOA의 알고리즘 구조는 유망하지만 고전 파이프라인에 넣으면 overhead 때문에 이점이 사라질 수 있다고 설명하고, Grover mixer·반복·확률적 변형 등도 검토할 수 있다고 답한다. 동시에 "최적화 전반에서 양자 이점이 분명해질 것"이라는 강한 기대를 말한다.

필기자 검토: QAOA는 조합 최적화의 **근사 해**를 찾는 변분 알고리즘이다. 제한된 문제군의 이론 결과와 모든 금융 문제에 대한 실용 우위는 다르다. 원 논문도 문제·깊이 `p`에 따른 성능을 다룬다. 이 영상에는 강한 고전 solver와 동등 조건으로 비교해 금융 업무의 end-to-end 이점을 입증한 QAOA 결과가 없다. 발표자의 전망은 검증된 보편적 결론으로 옮기지 않는다. Farhi·Goldstone·Gutmann, A Quantum Approximate Optimization Algorithm.

Q3. 가장 적은 논리 큐비트로 이점을 얻을 금융 용도는 무엇인가?

16:44-18:43

발표자의 답변: 단일 답은 없다. 알고리즘 종류뿐 아니라 기관별 입력 데이터, 구현 방식, 필요한 결과 정확도가 큐비트 수를 바꾼다. 우선순위가 높은 문제를 고르고 실제 모델을 합성해 입력 크기·정확도에 따른 자원 변화를 살펴보라고 권한다. 오차를 더 허용하면 필요한 자원이 줄어드는 경우가 있다고 설명한다. Jack은 뒤의 사례에서 하드웨어 선택을 다루겠다고 덧붙인다.

학습 포인트: "자산이 몇 개면 몇 큐비트"라는 표는 인코딩과 정밀도가 정해졌을 때만 의미가 있다. 논리 qubit 수만 줄이는 것이 전체 실행 비용을 최소화하는 것과 같지도 않다.

Q4. 자동화된 합성이 구현을 크게 바꿀 때 원래 의도와 권한은 어떻게 보존하는가?

24:45-27:52

발표자의 답변: 높은 수준의 수학적 모델을 한 번 작성해 두면 하드웨어가 바뀌어도 그 모델을 보존하면서 다시 컴파일할 수 있다고 설명한다. 입력 크기·정밀도와 실행할 하드웨어를 바꿀 수 있으며, 하드웨어 발전에 따라 알고리즘 자체를 바꿀 필요가 생길 수는 있다고 덧붙인다.

질문과 답변의 간격: 원 질문에는 자동화 과정에서의 **의도와 권한(authority)** 보존이 함께 들어 있다. 답변은 주로 하드웨어 변화에 대한 모델 재사용·이식성을 설명한다. 누가 변경을 승인하는지, agent가 허용된 범위만 수정하는지, 컴파일 전후 동등성을 어떻게 검증하는지에 대한 구체적 통제 절차까지 제시하지는 않았다. 따라서 "권한 보존 문제를 해결했다"로 요약하지 않는다. 고수준 모델은 검토할 명세를 제공하지만 거버넌스의 충분조건은 아니다.

Q5. 양자 포트폴리오 최적화는 Gurobi 같은 고전 solver와 무엇이 다른가?

28:06-32:14

발표자의 답변: 고전 연속·볼록 포트폴리오 문제와, 종목 수·섹터 한도·turnover 같은 이산 제약이 붙은 조합 문제를 구분한다. 수천 종목 중 일부를 고르는 경우의 탐색 공간이 크다는 점을 강조하며, 오늘의 양자 하드웨어에서는 작은 문제를 다루고 미래에 필요한 규모·정확도를 자원 분석으로 정하라고 권한다. "약 20-30 assets"라는 수치도 당시 규모를 설명하며 제시하지만 특정 인코딩·회로 조건을 밝힌 보편 한계는 아니다.

필기자 교정 1 — branch-and-bound는 exact 해를 못 주는 휴리스틱만이 아니다. 영상에는 branch-and-bound가 정확한 해를 주지 못한다는 취지의 설명이 있다. 이는 잘못이다. branch-and-bound는 유효한 상·하한으로 가지를 제거하며 충분한 계산을 마치면 모델에 대해 최적성을 증명할 수 있는 방법이다. 실무 solver는 수치 허용오차와 MIP gap을 사용하고, 시간 제한으로 중단하면 incumbent와 bound의 gap을 보고한다. 어려운 인스턴스에서 시간이 많이 든다는 사실과 정확한 해의 증명이 불가능하다는 주장은 다르다. Gurobi의 branch-and-bound 설명, Gurobi MIPGap 문서.

필기자 교정 2 — 중첩에서 모든 후보를 읽어 최적해를 고르는 것은 아니다. 영상은 모든 가지를 동시에 조사한 뒤 좋은 가지를 고른다는 비유를 쓴다. 양자 상태에 많은 후보의 진폭을 둘 수는 있지만 측정 한 번으로 모든 후보의 목적값을 얻을 수 없다. QAOA는 비용·mixer 연산과 고전 매개변수 최적화를 통해 좋은 해가 나올 확률을 높이는 방식이다. 중첩 자체가 NP-hard 문제를 일반적으로 효율적으로 푼다는 증거가 아니다. Farhi 외 원 논문.

필기자 교정 3 — NP-hard는 개별 인스턴스가 모두 불가능하다는 뜻이 아니다. 최악의 경우에 대한 복잡도 성격이며, 구조·제약·데이터에 따라 고전 solver가 큰 문제를 빠르게 풀 수 있다. 공정한 비교에는 동일한 금융 목적함수·제약·허용 gap·시간 제한·데이터로 얻은 고전 기준값이 필요하다.

8. 회차 앞부분의 전망 수치와 후기 사례를 읽는 기준

05:29-07:25에서 Erik은 컨설팅 보고서를 들어 금융이 초기 활용 분야이며 큰 경제적 가치가 기대된다고 설명하고, 수천억 달러 규모를 언급한다. 이 노트에서는 원 보고서의 범주·예측연도·시나리오를 별도 검증하지 않았으므로 **미래 시장 전망을 인용한 발표 내용**으로만 기록한다. 이미 실현된 매출이나 이 회차가 입증한 수익으로 쓰지 않는다.

35:35-36:16에는 여러 고객 협업 사례를 짧게 소개한다. 이 목록은 사용 분야를 알려 주지만 각 사례의 장치·정확도·고전 기준·실행 규모가 이 자리에서 모두 제시되지는 않는다. 특히 “고객 협업”, “컴파일 가능”, “GPU 모사에서 가속”, “실제 QPU에서 업무 우위”는 서로 다른 수준의 결과다.

9. 복습 문제와 풀이 — 노트 작성자가 추가

아래는 녹화물의 실제 청중 질문이 아니다. 금융 모형과 알고리즘의 연결을 확인하는 연습이다.

문제 1. 보수 크기를 인코딩하지 않으면 무엇을 추정하는가?

가격 `S∈{80,100,120}`, 확률 `(0.2,0.5,0.3)`, 행사가 `K=100`이라고 하자. 할인은 생략한다. `C=20`으로 정규화할 때 QAE가 추정할 `a`와 옵션 기대 보수는 얼마인가? 단순 비교 `S>K`의 확률과 왜 다른가?

풀이. 보수는 `(0,0,20)`, 정규화 보수는 `(0,0,1)`이므로 `a=0.3`, 기대 보수는 `C a=6`이다. 이 예에서는 보수가 양수인 경우의 크기가 모두 `C`여서 행사 확률과 `a`가 우연히 같다. 가격을 `{80,110,120}`으로 바꾸면 보수는 `(0,10,20)`, 정규화 보수는 `(0,0.5,1)`이고 `a=0.5×0.5+0.3=0.55`, 기대 보수는 `11`이다. 행사 확률은 `0.8`이므로 비교 oracle만으로 가격을 얻을 수 없다.

문제 2. 정확도를 2배 높이면 필요한 계산은 몇 배인가?

동일한 분산·신뢰수준 조건에서 목표 오차를 `ε`에서 `ε/2`로 줄인다. 고전 Monte Carlo와 이상적인 QAE의 주요 비용은 각각 몇 배가 되는가?

풀이. `ε^{-2}`는 4배, `ε^{-1}`는 2배가 된다. IQAE는 로그 인자와 상수가 있어 정확히 2배라고 단정할 수 없고, 실제 시간에는 더 깊어진 회로와 상태 준비 비용도 들어간다. 이는 오차를 반으로 줄이는 비교이며 신뢰수준을 두 배로 만든다는 뜻이 아니다.

문제 3. 1,000 shots이면 QAE 비용도 1,000인가?

서로 다른 `k`에 대해 `Q^kA` 회로를 실행하고 각 회로를 `n_k`회 측정한다. 단순한 Grover 질의 총량은 무엇이며, 왜 shots 합과 다른가?

풀이. Grover 연산 호출 수만 세면 `Σ_k n_k k`다. `A`와 `A†`의 호출까지 세는 표기에서는 각 `Q`의 구성을 반영해 대략 `Σ_k n_k(2k+1)` 형태의 상태 준비 비용을 세게 된다. 반면 shot 수는 `Σ_k n_k`다. 큰 `k`를 쓰는 회로는 한 shot도 길고 비싸다. 사용하는 논문의 query 정의를 먼저 맞추어 비교해야 한다.

문제 4. QAE 오차 δa는 가격 오차로 어떻게 바뀌는가?

`V_0=e^{-rT}Ca`이고 `C=200`, `e^{-rT}=0.95`라고 하자. 가격의 추정 오차를 `0.19` 이하로 제한하려면 확률의 추정 오차를 얼마 이하로 해야 하는가?

풀이. `|δV|=e^{-rT}C|δa|=190|δa|`이므로 `|δa|≤0.001`이다. 이는 분포 절단·이산화·payoff 구현 오차를 제외한 추정 오차 배정이다. 전체 가격 오차 한도가 `0.19`라면 이들 항에도 예산을 나누어야 한다.

문제 5. 100종목 중 10종목을 선택하면 왜 10큐비트로 충분하지 않은가?

풀이. 각 종목을 선택할지 나타내는 단순 이진 인코딩에는 100개 결정 변수가 필요하며 100개의 데이터 큐비트가 출발점이 된다. 제약을 구현하는 보조 큐비트나 다른 인코딩의 비용이 더해질 수 있다. 압축 인코딩으로 상태 수를 줄일 수도 있지만 상태 준비·mixer·목적함수 구현이 달라진다. "선택할 종목 수"와 "결정 변수 수"는 같지 않다.

문제 6. 같은 금융 문제를 공정하게 비교하려면 어떤 실험을 설계해야 하는가?

풀이. 공개 가능한 동일 데이터와 목적함수·제약을 고정하고, 강한 고전 solver·시뮬레이터·가능한 QPU를 구별한다. 해의 목적값·제약 위반·오차·신뢰수준과 wall-clock time·컴파일·대기·고전 후처리를 함께 기록한다. 최적화에서는 기준 최적값 또는 bound와 gap을, 가격 계산에서는 작은 기준 문제의 해석해나 충분히 정확한 수치해를 제시한다. 결과가 좋은 사례만 고르지 않고 입력 크기·난도·정밀도에 따른 실패도 포함한다.

문제 7. GPU가 양자 회로 모사를 100배 빨리 하면 양자 우위를 입증한 것인가?

풀이. 아니다. 그것은 두 고전 실행 환경의 모사 성능 비교다. 양자 알고리즘 개발·디버깅·벤치마크를 빠르게 할 수 있지만 QPU의 실용 우위에는 별도 장치·고전 알고리즘·업무 규모의 비교가 필요하다. 이 구별이 10월 15일 후속 금융·CUDA-Q 회차를 읽는 출발점이다.

10. 공부를 마친 뒤 남겨야 할 한 장의 명세

이 회차를 실제 프로젝트에 연결하려면 "양자로 금융을 해 보고 싶다"를 다음과 같은 문장으로 바꾸는 것이 유용하다. "어떤 데이터와 모형에서 어떤 가격·확률·포트폴리오를, 얼마의 오차와 시간 안에 얻어야 하며, 현재 가장 좋은 고전 기준은 무엇인가." 그 문장이 정해지면 입력 인코딩, 상태 준비, 알고리즘 선택, 합성 비용, 측정 통계와 하드웨어 비교를 같은 목표에 맞출 수 있다.

7월 회차의 AI 도구는 이 명세를 코드와 분석으로 옮기는 속도를 높이는 수단이다. 8월 회차가 더하는 내용은 **그 전에 어떤 계산을 명세해야 하는지**를 결정하는 절차다. 10월 회차의 GPU 가속은 그 명세와 회로를 개발·모사하는 비용을 줄이는 구체적인 실행 사례로 이어진다.

출처와 확인 범위

- **S04-V:** 공식 다시보기. 영어 자동자막 전 구간, 발언·문답·대략적 수치의 위치를 확인. 독립 재현하지 않음.
- **S04-E:** 공식 행사 페이지. 행사일·발표자·예고 범위 확인. 행사 페이지의 요일·시각 표기에 불일치가 있어 이 노트는 행사 날짜만 사용.
- **S04-AE:** Brassard 외, Quantum Amplitude Amplification and Estimation. 기대값의 확률 인코딩과 진폭 추정의 기초.
- **S04-IQ:** Grinko 외, Iterative Quantum Amplitude Estimation. QPE 제거, 질의 복잡도, 신뢰구간·Grover power의 역할.
- **S04-KL:** Prakash 외, Quantum option pricing via the Karhunen-Loève expansion. 영상에 언급된 경로 의존 옵션 가격 연구와 응용 전체 복잡도의 구분.

- **S04-QAOA:** Farhi·Goldstone·Gutmann, A Quantum Approximate Optimization Algorithm. 근사 최적화와 깊이·문제군에 관한 원 연구.
- **S04-C:** Classiq European option 예제, 공식 예제 라이브러리. 코드 학습의 출발점. 최신 문서의 표현이 영상 시점의 화면과 완전히 같다고 주장하지 않음.
- **S04-BB:** Gurobi branch-and-bound 설명, MIPGap 설명. 고전 solver의 최적성 증명에 대한 발언을 교정.

검토 상태: 자막 전 구간과 실제 Q&A 다섯 항목을 정리하고 Monte Carlo 지수, branch-and-bound, QAOA·중첩의 설명을 학술 근거에 맞춰 보충·교정했다. 자막의 이름·용어 오인식을 정리했으나 화면의 모든 코드와 숫자를 재구성하지 않았다. 77-120배의 정확한 조건, 시장 전망의 원 보고서 범주, 특정 논문 구현의 재현성은 미확정이다. 원 자막·메타데이터는 비공개 조사 자료로 보존한다.

보충 해설 문제에서 회로와 결과로

이 장은 네 세션을 이해하기 위해 독립적으로 작성한 해설이다. 아래 손계산 예제는 발표의 데모를 재현한 실험 결과가 아니다. 입력을 작게 고정해 어떤 계산을 하는지 끝까지 확인하는 것이 목적이다.

1 자연어 요구와 계산 명세 사이에 남는 일

"수익이 높은 포트폴리오를 찾아 달라"는 문장은 아직 계산 문제가 아니다. 얼마를 투자하는지, 보유량을 정수로 제한하는지, 공매도를 허용하는지, 위험을 어떻게 평가하는지에 따라 답이 달라진다. 자연어 에이전트가 코드를 만들 때 가장 먼저 읽어야 할 것은 이 가정들이다. 컴파일러가 프로그램을 처리했다는 사실은 사용자의 경제적 의도가 정확하게 반영됐다는 보증을 대신하지 않는다.

검토 순서는 입력의 단위와 범위를 적고, 목적함수를 정의하고, 제약을 쓰고, 작은 고전 계산으로 기대 답을 만든 다음 양자 구현을 비교하는 것이다. 예를 들어 실수 보유량과 3비트 정수 보유량은 서로 다른 탐색공간이다. "최적값"이라는 말도 수학적으로 허용되는 해의 집합을 먼저 정해야 의미가 생긴다.

프로그램의 정확성에는 여러 질문이 있다. 모델이 요구를 표현하는가, 합성한 회로가 모델의 연산을 구현하는가, 측정과 디코딩이 맞는가, 얻은 해가 유용한가를 차례로 확인한다. 앞의 질문을 통과했다고 뒤의 질문에 자동으로 답해지는 것은 아니다.

2 가역 연산과 임시 큐비트

고전 함수 $f(x)$가 여러 입력을 같은 출력으로 보낸다면 입력을 지우고 출력만 남기는 연산은 일반적으로 가역적이지 않다. 양자 회로에서는 입력과 별도 출력 공간을 함께 두어

$$U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$$

처럼 정의할 수 있다. $\oplus$는 비트별 XOR다. 같은 연산을 두 번 적용하면 $f(x) \oplus f(x) = 0$이 되어 원래 상태로 돌아온다. 실제 산술 회로에는 곱셈의 부분 결과, 비교 결과, carry 등을 담는 임시 큐비트가 더 필요할 수 있다.

위상 오라클에서는 조건을 나타내는 Boolean 함수 $f(x) \in \{0,1\}$를 사용한다. 계산용 임시 공간에 조건값을 만들고 그 값이 1이면 위상을 바꾼 뒤, 계산을 역순으로 풀면

$$|x\rangle|0\rangle \longmapsto (-1)^{f(x)}|x\rangle|0\rangle.$$

입력의 상대위상은 남고 임시 공간은 다시 0이 된다. 이 역연산을 **uncomputation**이라 한다. 임시 비트를 그냥 버리면 입력과 얽혀 있던 정보 때문에 원하는 간섭을 바꿀 수 있다. 따라서 "중간 결과가 필요 없으니 지운다"는 고전 프로그래밍 직관을 그대로 적용할 수 없다.

준비 연산을 V, 그 상태에서 적용할 연산을 U라 하면, 시간 순서는 준비 V → 적용 U → 복원 $V^\dagger$이고 상태에 작용하는 행렬은 $V^\dagger U V$다. Classiq의 `within_apply`는 이런 연산 구조를 표현한다. 합성기가 이 구조와 변수의 수명을 알면 임시 공간을 재사용하는 구현을 탐색할 수 있다. Classiq의 within_apply 공식 설명에서 준비와 역준비의 구체적 사용을 볼 수 있다.

3 Grover 탐색에서 부호가 확률로 바뀌는 과정

N개 후보 중 조건을 만족하는 후보가 M개이고 처음에 균등 중첩을 만들었다고 하자. 좋은 후보의 전체 초기 확률은 $a = M/N$이다. 좋은 후보들의 정규화된 중첩을 $|G\rangle$, 나머지를 $|B\rangle$라 두면

$$|\psi\rangle = \sin\theta|G\rangle + \cos\theta|B\rangle, \qquad \sin^2\theta = a.$$

위상 표시와 초기 상태에 대한 반사를 한 번씩 수행하는 Grover 반복은 이 두 상태가 만드는 평면에서 진폭을 회전시킨다. 이상적 조건에서 r회 반복 후 좋은 후보를 얻을 확률은

$$P_r = \sin^2((2r+1)\theta).$$

손계산으로 $N=8, M=2$를 택하면 $a=1/4$, $\theta=\pi/6$이다. 반복 전 성공확률은 1/4이고 한 번 반복하면 $P_1=\sin^2(\pi/2)=1$이다. 두 번 반복하면 $P_2=\sin^2(5\pi/6)=1/4$로 다시 내려간다. 따라서 반복 횟수를 무조건 늘릴 수는 없다. 실제 문제에서는 좋은 후보 수를 모르거나 회로가 잡음에 노출되어 이 이상적 계산과 달라진다.

질의 복잡도에서 $O(\sqrt{N/M})$라는 향상은 오라클 한 번의 비용과 상태 준비를 따로 세는 추상 모델의 결과다. 곱셈·금융 조건·데이터 조회로 오라클을 구현하는 비용까지 합한 시간은 별도로 평가해야 한다. 진폭 증폭과 추정의 원 논문은 이러한 알고리즘 구조의 기초다.

4 포트폴리오를 QAOA 목적함수로 옮기는 작은 예

두 자산 중 하나를 선택하는 문제를 만들어 보자. 선택 변수 x_1, x_2는 0 또는 1이고 $x_1 + x_2 = 1$이다. 교육용 입력으로 수익률 $\mu = (0.08, 0.12)$, 공분산 행렬과 목적함수를 다음처럼 둔다. 금액 배분이 아닌 이진 선택 예제이며 원영상의 정수 배분 데모와 다르다.

$$\Sigma = \begin{pmatrix} 0.04 & 0.01 \\ 0.01 & 0.09 \end{pmatrix}, \qquad C(x) = x^{\mathsf{T}}\Sigma x - \mu^{\mathsf{T}}x + (x_1 + x_2 - 1)^2.$$

첫 항은 위험, 두 번째는 수익을 높이는 방향, 마지막은 제약을 어기면 더하는 벌점이다. 네 경우를 직접 계산하면 다음과 같다.

선택 (x_1, x_2)	위험	수익	벌점	최소화할 비용
(0,0)	0	0	1	1
(1,0)	0.04	0.08	0	-0.04
(0,1)	0.09	0.12	0	-0.03
(1,1)	0.15	0.20	1	0.95

이 설정에서는 첫 자산만 고르는 해가 가장 작다. 수익만 비교하면 두 번째를 고르지만 위험을 함께 넣으면 결과가 달라진다. 위험과 수익을 더할 때는 계수의 단위와 척도를 정의해야 한다. 이 예에서는 입력 수치의 척도를 고정하고 위험 계수를 1로 두었다. 실제 모델에서는 위험회피 계수와 벌점 계수를 명시해야 한다.

기저 상태 $|x\rangle$의 에너지가 $C(x)$가 되는 대각 연산자를 H_C로 정의한다. 이진 변수는 $x_j = (I - Z_j)/2$로 표현할 수 있다. QAOA는 보통 비용 연산과 비트를 섞는 연산을 번갈아 적용해 매개변수화한 상태를 만든다.

$$|\gamma, \beta\rangle = e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \cdots e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C} |+\rangle^{\otimes n}, \qquad H_M = \sum_j X_j.$$

고전 최적화기는 $\langle H_C \rangle$ 등을 이용해 각도를 바꾼다. 마지막에는 샘플 비트열을 원래 변수로 디코딩하고 제약을 확인한다. **기대 비용, 가장 좋은 측정 해, 그 해의 빈도, 전체 샷 중 제약을 지킨 비율**은 서로 다른 결과다. 기대 비용이 높아도 좋은 해가 낮은 확률로 나올 수 있고, 한 번 좋은 해를 봤다고 안정적 최적화가 입증되지는 않는다.

QAOA의 깊이 p, 초기값, 최적화기, 샷, 벌점과 mixer 선택은 결과에 영향을 준다. 비교할 때는 같은 시간 예산과 성공 기준으로 여러 실행을 평가해야 한다. 원 논문은 조합최적화의 근사 알고리즘을 제안했으며, 모든 금융 문제에서 고전 알고리즘보다 빠르다는 결론을 제공하지 않는다. Farhi 등의 QAOA 논문.

5 옵션 기대값을 진폭으로 인코딩하는 방법

기대값을 알고 싶은 이산 확률변수 S가 있고, 상태 준비 회로가 $\sum_i \sqrt{p_i}|i\rangle$를 만든다고 하자. payoff g_i를 $0 \le g_i \le g_{\max}$ 범위에 놓고 보조 큐비트를 조건부 회전시키면

$$A|0\rangle = \sum_i \sqrt{p_i}|i\rangle \left(\sqrt{1 - g_i/g_{\max}}|0\rangle + \sqrt{g_i/g_{\max}}|1\rangle \right).$$

보조 큐비트가 1일 확률은 $a = \sum_i p_i g_i / g_{\max}$이므로 원하는 기대 payoff는 $g_{\max} a$다. 가격을 구하려면 계약의 확률모형과 할인율도 정해야 한다. European call에서는 $g_i = \max(S_i - K, 0)$이며, Asian option은 경로의 평균에 의존하므로 경로 생성과 평균 계산이 추가된다.

손계산 예로 가격 $(80, 100, 120, 140)$, 확률 $(0.1, 0.2, 0.4, 0.3)$, 행사가 $K = 110$을 둔다. payoff는 $(0, 0, 10, 30)$, 기대 payoff는 $0.4 \times 10 + 0.3 \times 30 = 13$이다. $g_{\max} = 30$이면 $a = 13/30$이다. 이 값은 할인 전이며 금융 예측이나 투자 조언을 위한 실제 시장 입력이 아니다.

이 회로를 독립적으로 N_s번 측정하는 단순 추정에서 보조 비트는 Bernoulli 확률변수다. 확률 추정량과 환산한 payoff의 표준오차는

$$\mathrm{SE}(\hat{a}) = \sqrt{a(1-a)/N_s}, \qquad \mathrm{SE}(\hat{g}) = g_{\max}\sqrt{a(1-a)/N_s}.$$

10,000샷에서는 각각 약 0.004955와 0.14866이다. 같은 이산 payoff 자체를 고전적으로 샘플링하면 분산은 $310 - 13^2 = 141$이고 표준오차는 $\sqrt{141/10000} \simeq 0.11874$다. 확률로 바꾸기만 해서는 샘플 효율이 좋아지는 것이 아님을 이 작은 예에서 볼 수 있다.

QAE는 준비 회로와 역회로를 포함하는 증폭 연산을 일관되게 반복하여 진폭에 관한 정보를 얻는다. IQAE는 QPE 레지스터를 쓰는 표준 QAE와 달리 여러 증폭 깊이에서의 측정과 고전적 신뢰구간 갱신을 사용한다. 위상추정을 없애도 상태 준비·역준비와 긴 반복 회로의 비용은 남는다. Grinko 등의 IQAE 논문은 정확도·신뢰수준과 오라클 호출 수 사이의 관계를 분석한다.

고전 Monte Carlo의 독립 표본 평균은 유한 분산 조건에서 전형적으로 오차가 $N_s^{-1/2}$로 줄어든다. 따라서 오차를 1/10로 만들려면 표본을 약 100배 늘린다. QAE의 오라클 복잡도 향상과 실제 실행시간의 향상 사이에는 상태 준비, 함수 근사, 이산화, 샷, 잡음과 반복 깊이가 있다. 이 조건을 빠뜨리고 두 복잡도 식만으로 실용적 우위를 선언할 수 없다.

6 하이브리드 계산에서는 반복 밖으로 뺄 수 있는 일을 찾는다

양자 커널을 반복 호출하는 작업은 상태 준비·회로 실행 외에도 모델 작성, 합성, 코드 변환, JIT 컴파일, 데이터 이동, 고전 최적화가 필요하다. 전체 시간을 비교할 때는 적어도

$$T_{\text{total}} = T_{\text{model}} + T_{\text{synth}} + T_{\text{convert}} + T_{\text{JIT}} + \sum_{k=1}^{K} (T_{\text{transfer},k} + T_{\text{run},k} + T_{\text{post},k})$$

처럼 무엇을 한 번 하고 무엇을 반복하는지 구별한다. 실제 QPU를 사용하면 대기열과 작업 제출 지연도 포함된다. CPU/GPU 시뮬레이터의 상태 생성 방식과 측정 샘플링 방식도 총비용을 바꿀 수 있다.

교육용 가정으로 컴파일 2초, 실행 0.3초, 후처리 0.2초, 반복 100회를 두자. 매번 컴파일하면 $100(2+0.3+0.2)=250$초이고 컴파일을 한 번 재사용하면 $2+100(0.3+0.2)=52$초다. 약 4.81배의 차이가 생겨도 회로 자체가 빨라졌다는 의미는 아니다. 이러한 개선은 실용적이지만, 어떤 항을 줄였는지를 설명해야 다른 문제에 적용할 수 있다.

CUDA-Q의 `sample`은 측정 분포를 얻고 `observe`는 지정한 관측량의 기대값을 구하는 API다. 타깃이 CPU 시뮬레이터인지 GPU 시뮬레이터인지 실제 QPU인지는 별도로 지정한다. `observe`라는 함수명을 봤다는 것만으로 실제 장치의 유한 샷 측정을 수행했다고 판단하지 않는다. NVIDIA 실행 문서.

7 회로 폭과 깊이를 동시에 읽어야 하는 이유

폭은 회로가 사용하는 큐비트 수이며 깊이는 동시에 할 수 있는 연산을 묶었을 때의 순차 단계 수다. 더 많은 임시 큐비트로 병렬 연산을 늘리면 깊이가 줄 수 있다. 반대로 임시 공간을 반복 재사용하면 폭을 줄이는 대신 게이트와 깊이가 늘 수 있다. 합성의 목표를 바꾸는 실험은 같은 수학적 모델이 서로 다른 비용의 회로를 낳는다는 것을 보여준다.

밀집 상태벡터 시뮬레이션은 n큐비트에 2^n개 복소 진폭을 저장한다. 복소수 하나를 16바이트로 저장한다면 상태벡터만 필요한 메모리는 $16\cdot 2^n$바이트다. 30큐비트는 16 GiB, 40큐비트는 16 TiB이고 큐비트 두 개가 늘면 4배다. 임시 버퍼 등은 제외한 계산이다. 다른 정밀도, tensor network, stabilizer 방식은 다른 비용 구조를 가지므로 이 식을 모든 시뮬레이터에 적용하지 않는다.

QPU에서는 최종 타깃의 네이티브 게이트와 연결성으로 변환한 후의 2큐비트 게이트 수, 깊이, 보정된 오류율과 샷이 중요하다. 논리적 회로에서 깊이를 줄인 비율이 장치 실행시간이나 결과 정확도의 같은 비율 개선을 뜻하지는 않는다. 오류정정 자원 추정은 물리적 큐비트·코드 거리·주기·실패확률 가정을 더 포함하는 별도 단계다.

다음 그림은 같은 모델 안에서 합성 선택을 바꾸는 경우와, 시간 범위·선택 자유도를 바꾸어 문제를 다르게 구성하는 경우를 나눈다. 이 두 경우를 구별해야 “적은 큐비트”가 어떤 의미인지 판단할 수 있다.

큐비트 수를 줄일 때: 회로 비용과 문제 범위를 함께 읽기

윗부분은 일반적 자원 교환의 개념도 · 아랫부분은 발표 화면의 10주식·10일 설계안 재구성

같은 수학적 모델 안에서의 합성 선택

임시 큐비트 재사용

폭을 줄이는 대신 깊이가 늘 수 있다.

⟷

더 많은 임시 큐비트 활용

폭을 늘려 깊이를 줄일 수 있다.

가능한 교환 관계이며, 모든 모델에서 보장되는 법칙이나 실제 깊이 측정값은 아니다.

문제를 나누거나 선택지를 고정하면 비교 범위도 달라진다

400 큐비트 · 10일 전체	**Exact counts · whole horizon** 10주식 × 10일 × 4큐비트	롱·숏 보유 수를 각각 표현 전체 기간을 함께 모델링
40 큐비트 / 하루	**Exact counts · one day at a time** 10주식 × 1일 × 4큐비트	날짜 연결은 고전 동적 계획법(DP) DP 상태·전이 비용을 별도 계산
20 큐비트 / 하루	**Direction-fixed · one day at a time** 10주식 × 1일 × 2큐비트	롱·숏 방향을 미리 고정 방향 선택·헤지 자유도 축소

400 → 40 → 20을 동일한 범위와 표현력을 유지한 단순 압축률로 읽으면 안 된다.

그림 5-1. **회로 비용의 교환과 문제 범위의 변경.** 위는 임시 공간 재사용에 따른 일반적인 폭·깊이 교환 가능성이다. 아래는 새 플랫폼 발표 50:10의 10주식·10일 설계를 한국어로 재구성했다. 400은 전체 기간, 40과 20은 하루 기준이며, 일별 고전 DP 비용과 방향 고정의 헤지 자유도 손실을 함께 읽는다. 같은 문제의 성능을 재측정한 그림이 아니다.

8 데모에서 다음 실험으로 넘어가는 기준

작은 문제는 전수열거로 최적값과 모든 제약을 먼저 확인한다. 중간 규모에서는 문제에 맞는 고전 최적화기와 Monte Carlo·분산감소·준 Monte Carlo 등을 비교 후보로 정한다. 선택한 오차와 신뢰수준, 시간·메모리 예산에서 비교하고, 전처리와 상태 준비를 포함한다. 데이터나 회로의 특정 구조에만 유리하면 그 조건을 함께 적는다.

AI 에이전트가 만든 회로에는 입력 가정·참조 문서·SDK 버전·목적함수·실행 타깃과 결과를 보관한다. 사람이 확인할 핵심은 생성된 코드의 길이보다 문제 명세, 작은 기준 답, 유효한 출력의 비율과 전체 실행 비용이다. 이 네 가지가 있어야 다음 사용자가 데모를 다시 실행하고 차이의 원인을 찾을 수 있다.

한영 용어집

정의는 이 노트에서 사용하는 의미다. 제품 이름, 알고리즘의 개념, 성능 지표를 구분해 읽는다.

용어	뜻과 구별
양자 모델 quantum model	수행할 연산·데이터·제약을 기술한 표현. 특정 장치에서 실행할 게이트 열과 구별한다.
Qmod	Classiq의 양자 모델링 언어. Python을 통한 표현과 native 표현이 있다.
합성 synthesis	모델에서 조건을 만족하는 회로 구현을 만드는 과정. 최종 결과의 업무 적합성을 보증하는 절차는 아니다.
컴파일 compilation	프로그램을 실행 가능한 표현으로 바꾸는 넓은 과정. 합성·최적화·타깃 변환이 포함될 수 있다.
트랜스파일 transpilation	회로를 타깃의 게이트 집합과 연결성에 맞춰 변환하는 과정. 산정한 자원이 어느 단계의 값인지 확인한다.
양자 커널 quantum kernel	호스트 프로그램에서 호출하는 양자 계산 단위. 여기서는 CUDA-Q의 실행 코드 단위를 뜻한다. 머신러닝의 유사도 함수인 kernel과 구별한다.
CUDA-Q	양자 커널과 고전 연산을 함께 다루는 NVIDIA의 프로그래밍 플랫폼. 실행 타깃으로 CPU·GPU 시뮬레이터와 지원되는 QPU 등을 사용한다.
JIT just-in-time compilation	실제 사용 시점에 코드를 컴파일하는 방식. 반복 호출 때 재사용하는지에 따라 총시간이 달라진다.
타깃 target / 백엔드 backend	프로그램을 실행하거나 변환하는 대상. 이름에 quantum이 있어도 시뮬레이터일 수 있다.
QPU quantum processing unit	물리적 양자 프로세서. CPU나 GPU에서 회로를 계산하는 시뮬레이터와 구별한다.
상태벡터 state vector	순수 양자 상태의 복소 진폭 표현. 밀집 표현에는 큐비트 수에 지수적으로 증가하는 저장공간이 든다.
샷 shot	동일한 회로 준비와 측정을 반복하는 통계적 시행 단위. 하나의 샷이 항상 원격 작업 하나와 같은 뜻은 아니다.
샘플링 sampling	측정 분포에서 결과를 뽑는 과정. 기저별 정확한 확률을 전부 읽는 것과 다르다.
관측량 observable	측정할 물리량에 대응하는 에르미트 연산자. 기대값은 상태와 관측량을 함께 정해야 정의된다.
기대값 expectation value	확률로 가중한 평균. 한 번 측정한 값이나 가장 빈번한 값과 다르다.

용어	뜻과 구별
후처리 post-processing	비트열 디코딩, 확률·비용·신뢰구간 계산 등 실행 결과를 해석하는 과정.
회로 폭 circuit width	해당 회로에 사용되는 큐비트 수. 입력 변수 수에 임시 공간이 더해질 수 있다.
회로 깊이 circuit depth	병렬 가능한 연산을 묶었을 때의 순차 단계 수. 전체 게이트 수와 다르다.
2큐비트 게이트 two-qubit gate	두 큐비트에 작용하는 게이트. 타깃에 따라 CX·CZ 등의 네이티브 구현 비용이 다르다.
보조 큐비트 ancilla	중간 계산, 조건 표시, 추정 등을 돕는 공간. 알고리즘의 출력 변수와 구별한다.
가역 연산 reversible computation	입력을 복구할 수 있는 연산. 고전 함수를 양자 회로에 넣을 때 정보 보존을 고려해야 한다.
언컴퓨테이션 uncomputation	중간 계산을 역연산으로 풀어 보조 공간을 정리하는 일. 중간 비트를 임의로 지우는 것과 다르다.
오라클 oracle	조건 검사나 함수 평가를 수행하는 계산 단위. 질의 복잡도에서는 그 내부 비용을 따로 계산한다.
위상 오라클 phase oracle	조건에 따라 입력 진폭의 위상에 부호 등을 붙이는 연산. 그것만으로 측정 확률이 바뀌는 것은 아니다.
진폭 증폭 amplitude amplification	좋은 결과가 나올 진폭을 회전으로 키우는 절차. 반복을 지나치게 하면 성공확률이 다시 줄 수 있다.
Grover 반복 Grover iterate	좋은 상태 표시와 준비 상태에 대한 반사를 조합하는 증폭 연산.
QAE quantum amplitude estimation	준비 회로가 만드는 성공확률 또는 그 진폭에 관한 정보를 추정하는 알고리즘 계열.
IQAE iterative QAE	위상추정 레지스터 대신 반복 증폭과 측정·고전적 갱신을 사용하는 QAE 방식. 깊은 회로 비용이 사라지는 것은 아니다.
QPE quantum phase estimation	유니터리의 고유위상을 추정하는 알고리즘. IQAE의 I와 구별한다.
Monte Carlo	확률적 표본으로 평균 등을 근사하는 방법. 유한 분산의 독립 표본 평균은 전형적으로 표준오차가 표본 수의 제곱근에 반비례한다.
표준오차 standard error	추정량의 표준편차. 개별 데이터 값의 표준편차와 다르다.
신뢰구간 confidence interval	반복 실험에서 정해진 포괄 확률을 갖도록 구성한 구간. 한 번 얻은 구간과 정확도 목표를 혼동하지 않는다.
payoff	계약 조건과 만기 가격·경로로 정해지는 지급액. 할인 전 payoff와 현재 가격을 구별한다.
strike 행사가	옵션 계약에서 정한 기준 가격. 입력 확률분포의 평균과 다르다.

용어	뜻과 구별
European option	정한 만기 시점에 행사하는 옵션. 만기 가격만으로 payoff가 정의되는 대표 예가 European call이다.
Asian option	기간 중 가격 평균 등 경로에 의존하는 옵션. 만기 가격 하나를 표현하는 회로만으로 충분하지 않다.
목적함수 objective	최적화에서 줄이거나 키우려는 수학적 값. 단위·부호·계수의 정의가 필요하다.
제약 constraint	허용되는 해가 만족해야 하는 조건. 합성기의 큐비트 제약과 업무 문제의 예산 제약은 서로 다른 대상이다.
벌점 penalty	제약을 목적함수에 반영하려고 더하는 항. 계수가 작으면 위반 해가 유리해질 수 있다.
QUBO	quadratic unconstrained binary optimization. 이진 변수의 이차식으로 표현한 문제이며 제약은 보통 벌점 등으로 처리한다.
Ising 표현	이진 변수를 Pauli Z 연산자 등으로 옮겨 비용을 에너지로 나타내는 표현.
QAOA	비용 연산과 mixer를 번갈아 적용하고 고전 최적화로 각도를 조절하는 근사 최적화 알고리즘.
mixer 혼합 연산자	탐색 상태를 바꾸는 QAOA 연산자. 단순 X mixer와 제약을 보존하는 mixer는 허용 상태가 다를 수 있다.
feasible 해	문제의 모든 제약을 만족하는 해. 목적값이 좋아도 제약을 어기면 허용 해가 아니다.
최적성 간극 optimality gap	후보 해의 비용과 기준 최적값·하한 사이의 차이. 기준값이 무엇인지 명시해야 한다.
벤치마크 benchmark	조건과 지표를 고정한 비교. 회로 합성 시간, 기대값 평가 시간, 전체 실행시간은 각각 다른 지표다.
Pareto trade-off	여러 비용을 동시에 볼 때 한 비용을 더 줄이려면 다른 비용이 늘어나는 관계. 최적화 목표 한 개로 전체 우열을 정할 수 없다.
양자 이점 quantum advantage	명시한 과제·정확도·비용에서 정한 고전 기준보다 유리한 결과. GPU 시뮬레이터 가속과 구별한다.
오류 완화 error mitigation	잡음 효과를 줄이기 위한 추정·후처리 등의 기법. 오류정정 논리 큐비트를 구현한 것과 다르다.
오류정정 QEC	부호화된 정보와 syndrome 등을 이용해 오류를 제어하는 방법. 물리 큐비트와 논리 큐비트의 수는 다르다.
자원 추정 resource estimation	실행에 필요한 큐비트·게이트·주기·시간 등을 가정하에 계산하는 일. 물리 장치에서 이미 실행했다는 의미가 아니다.
AI skill	에이전트가 특정 작업에 적용하는 지식과 수행 지침. 프로그램이 호출하는 실행 도구와 역할을 구별한다.

용어	뜻과 구별
tool 도구	합성·실행·문서 조회처럼 에이전트가 호출하는 기능. 결과를 다시 검사할 책임은 남는다.
MCP Model Context Protocol	AI 애플리케이션과 도구·자료를 연결하는 프로토콜. 특정 양자 알고리즘이나 정확성 보증 방식의 이름은 아니다.
SDK software development kit	프로그램 작성과 서비스 호출을 위한 라이브러리·인터페이스. 영상의 버전과 현재 설치 버전이 다를 수 있다.

같은 단어가 다른 뜻으로 쓰일 때

금융의 risk는 수익의 분산이나 손실위험 같은 문제 정의이며, 하드웨어 오류율은 회로의 물리적 실패와 관련된다. 양자 커널을 CUDA-Q 코드 단위라는 뜻으로 쓰는 문단과, AML에서 데이터의 유사도 함수라는 뜻으로 쓰는 문단도 구분한다. “최적화”는 업무 목적함수를 푸는 QAOA, 회로 자원을 줄이는 합성, 실행 비용을 줄이는 GPU·런타임 개선을 각각 가리킬 수 있다. 무엇을 목적값으로 삼는지 문장 안에서 확인한다.

복습 질문과 완전 풀이

이 장의 16문제는 편집 과정에서 만든 학습 문제다. 실제 청중 질문은 각 회차의 질의응답 절에서 따로 정리한다. 손계산 예제는 보충 해설의 입력을 사용한다.

문제 1 컴파일 성공이 보장하는 것

질문. "예산 6"을 "정확히 6을 투자한다"와 "6 이하를 투자한다"로 구현한 모델은 언제 달라지는가? 둘 다 합성되면 어느 쪽이 맞는가?

풀이. 첫 모델의 허용 집합은 합계가 6인 해이며 둘째는 합계가 0부터 6인 해까지 포함한다. 위험이나 음의 수익이 있으면 전액을 쓰지 않는 해가 유리할 수 있다. 합성 성공만으로 의도를 결정하지 못한다. 요구 문장에서 예산이 상한인지 의무 투자액인지 확인하고 작은 입력으로 두 모델의 해를 대조한다.

문제 2 입력을 남기는 이유

질문. $f(0) = f(1) = 0$일 때 $|x\rangle \mapsto |f(x)\rangle$가 유니터리일 수 없는 이유는?

풀이. 직교하는 $|0\rangle$과 $|1\rangle$이 같은 벡터로 보내져 내적이 0에서 1로 바뀐다. 유니터리는 내적을 보존하므로 불가능하다. 입력을 보존한 $|x, y\rangle \mapsto |x, y \oplus f(x)\rangle$ 같은 확장으로 가역 계산을 구성한다.

문제 3 위상 표시만으로는 왜 충분하지 않은가

질문. 균등 중첩의 일부 진폭에 -1을 곱한 직후 계산 기저에서 측정하면 표시된 항이 더 잘 나오는가?

풀이. 확률은 진폭의 절댓값 제곱이므로 부호만 바꿔서는 달라지지 않는다. Grover 반복의 후속 반사가 상대위상 차이를 진폭 크기의 차이로 바꾼다. 따라서 오라클만 실행한 히스토그램과 전체 증폭 후 히스토그램을 구분한다.

문제 4 Grover 반복의 과잉 실행

질문. 8개 후보 중 좋은 후보가 2개라면 0·1·2회 반복의 이상적 성공확률을 구하라.

풀이. $\theta = \arcsin\sqrt{2/8} = \pi/6$이다. $\sin^2\theta = 1/4$, $\sin^2(3\theta) = 1$, $\sin^2(5\theta) = 1/4$다. 1회가 이 예의 최고점이다. 이를 좋은 후보 수가 다른 문제의 고정 반복 횟수로 복사할 수 없다.

문제 5 큐비트 수가 의미하는 후보 수

질문. 3비트 정수 변수 세 개는 몇 조합을 표현하는가? 합계가 6 이하인 비음수 정수 조합은 몇 개인가?

풀이. 각 변수는 0부터 7까지이므로 전체는 $8^3 = 512$개다. 합계 6 이하에서는 각 변수의 상한 7이 걸리지 않는다. slack $s \geq 0$를 도입해 $w_1 + w_2 + w_3 + s = 6$을 세면 $\binom{9}{3} = 84$개다. 이것은 업무 변수의 후보 수이며 산술용 ancilla를 포함한 전체 회로 폭은 별도로 센다.

문제 6 위험을 넣으면 선택이 바뀌는가

질문. 보충 해설의 두 자산 예제에서 수익만 최대화한 해와 위험·수익을 함께 고려한 해를 비교하라.

풀이. 하나를 고른다는 제약에서 수익은 두 번째 자산의 0.12가 더 크다. 비용 $x^{\mathsf{T}}\Sigma x - \mu^{\mathsf{T}}x$는 첫 자산이 -0.04, 둘째가 -0.03이므로 최소화하면 첫째다. 위험 계수를 바꾸면 다시 비교해야 한다. 서로 다른 목적함수의 해를 같은 최적해라고 부르면 안 된다.

문제 7 벌점 계수가 작을 때

질문. 같은 예제에서 벌점 항의 계수를 1에서 0.001로 줄이면 무엇을 먼저 확인해야 하는가?

풀이. (1,1)의 위험에서 수익을 뺀 값은 -0.05다. 벌점 0.001을 더하면 -0.049로 유효 해 (1,0)의 -0.04보다 낮다. 최적화기는 제약을 어긴 해를 고를 수 있다. 벌점 계수는 경제적 목적과 같은 척도에서 충분한지 확인해야 한다.

문제 8 기대값과 최선 샘플

질문. 한 번의 실행에서 좋은 해를 얻었지만 그 해의 확률이 0.001이면 어떤 결과를 더 기록해야 하는가?

풀이. 총샷, 좋은 해의 발생 횟수, 유효 해 비율, 기대 비용과 여러 seed에서의 재현성을 기록한다. 독립 샷이라고 단순화하면 1,000샷에서 한 번 이상 볼 확률은 $1-(1-0.001)^{1000} \simeq 0.6323$이다. 약 36.8%의 실행에서는 아예 못 볼 수 있다. 하나의 좋은 샘플과 안정적 성공은 다른 주장이다.

문제 9 payoff와 현재 가격

질문. 가격 80·100·120·140, 확률 0.1·0.2·0.4·0.3, 행사가 110의 call에서 할인 전 기대 payoff와 성공확률 인코딩을 구하라.

풀이. payoff는 0·0·10·30이고 평균은 13이다. 상한 30으로 나누면 성공확률 $a = 13/30$이다. 현재 가격을 구하려면 해당 확률이 위험중립 분포인지와 할인계수 등을 정해야 하므로 13을 곧바로 실제 시장 옵션 가격으로 제시하지 않는다.

문제 10 오차를 절반으로 줄이는 비용

질문. 독립 측정의 표준오차가 $N^{-1/2}$로 줄면 10,000샷의 오차를 절반으로 줄이려면 몇 샷이 필요한가?

풀이. $1/\sqrt{N'} = (1/2)/\sqrt{10000}$에서 $N' = 40000$이다. 표본을 두 배로 하면 오차는 $1/\sqrt{2}$배가 된다. 이 관계는 표본 분산이 유한하고 독립이라는 가정에 대한 통계적 진술이다.

문제 11 IQAE가 없앤 것과 남긴 것

질문. IQAE가 QPE를 사용하지 않는다는 말은 깊은 회로가 필요 없다는 말인가?

풀이. 아니다. 위상추정 레지스터와 관련 구조를 줄이지만 준비 회로·역회로 및 여러 번의 Grover 반복이 필요하다. 요구 정확도에 따라 반복 깊이와 총 호출 수를 확인해야 한다. 각 반복 회로의 오류와 실행시간도 포함한다.

문제 12 컴파일 재사용의 가속

질문. 컴파일 2초·실행 0.3초·후처리 0.2초인 계산 100회에서 매번 컴파일하는 방식과 한 번 재사용하는 방식의 시간은?

풀이. 각각 250초와 52초다. 가속비는 $250/52 \simeq 4.8077$이다. 양자 회로의 실행시간 0.3초가 바뀌지 않아도 가속은 생긴다. 비교표에는 포함한 단계와 반복 횟수를 적어야 한다.

문제 13 시뮬레이터 메모리

질문. 16바이트 복소수로 저장하는 밀집 상태벡터의 30큐비트와 32큐비트 메모리는?

풀이. $16 \cdot 2^{30}$ 바이트는 16 GiB이고, 32큐비트는 64 GiB다. 두 큐비트 증가는 4배다. 프로그램의 임시 버퍼를 포함하지 않았으며 이 계산이 실제 QPU의 물리적 메모리 요구량은 아니다.

문제 14 같은 프로그램을 여러 타깃에 실행한다는 의미

질문. CPU 시뮬레이터에서 결과가 맞았고 GPU 타깃으로 바꿀 수 있다면 실제 QPU에서도 같은 정확도인가?

풀이. 이상적 연산의 의미를 확인한 것과 장치의 잡음·연결성·게이트 분해·보정·샷에 따른 결과는 다르다. 타깃 변환 후 자원과 유한 샷·잡음 조건에서의 오차를 확인한다. CPU와 GPU는 회로를 고전적으로 계산하며, QPU에서 측정했다는 별도 기록이 필요하다.

문제 15 벤치마크 표에 빠진 정보

질문. "기존 100초, 새 방식 5초"라는 표만으로 같은 결과를 더 빨리 얻었다고 볼 수 있는가?

풀이. 같은 문제 크기·오차·성공확률·샷·정밀도·전처리·하드웨어인지 알아야 한다. 한쪽은 처음 컴파일 시간을 넣고 다른 쪽은 제외했을 수도 있다. 반복 분포와 warm-up 여부도 확인한다. 시간비 20배는 주어진 두 숫자의 비율일 뿐 동등한 계산의 성능비라는 결론은 추가 조건을 필요로 한다.

문제 16 발표자에게 이어서 물을 질문 만들기

질문. "양자 이점이 있나요?"를 검증 가능한 세 질문으로 구체화하라.

풀이. 예시는 다음과 같다. ① 어떤 입력 크기와 정확도·신뢰수준에서 어떤 고전 기준과 비교했는가? ② 데이터 준비·합성·JIT·대기열·후처리까지 포함한 총시간과 비용은 얼마인가? ③ 여러 seed·입력에서 유효 해 비율과 최적성 간극 또는 추정 오차가 어떻게 분포하는가? 이는 본 노트의 후속 확인 질문이며 실제 청중 발언으로 인용하지 않는다.

출처와 작성 검증 공개

원영상과 행사 안내

1. Classiq 새 플랫폼 다시보기, 행사 2026-10-21. 공식 행사 안내.
2. Classiq와 NVIDIA CUDA-Q 금융 웨비나 다시보기, 행사 2026-10-15. 공식 행사 안내.
3. AI 에이전트 다시보기, 행사 2026-07-08. 공식 행사 안내.
4. 금융 문제의 양자 응용 다시보기, 행사 2026-08-27. 공식 행사 안내.

공식 시리즈명은 Quantum Office Hours이며 NVIDIA 공동 웨비나는 별도 행사다. 두 종류를 최근 제품·응용 세션이라는 범위로 함께 정리했다. 공식 채널에는 별도의 Classiq Talks 재생목록도 있다. 이 노트의 네 회차와 동일한 재생목록은 아니다.

보충 학습 자료

- Classiq 문서: Qmod, 합성, 실행, 응용 예제. 움직이는 latest 문서이므로 재현에는 SDK 버전 고정이 필요하다.
- Classiq 공식 응용 저장소: 세션의 수학과 구현을 더 읽는 출발점. 현재 파일이 영상 촬영 시점의 파일과 같다고 가정하지 않는다.
- Classiq within_apply와 Grover 연산자 예제: 준비·적용·역준비 구조.
- NVIDIA CUDA-Q 실행 문서: sample·observe와 실행 타깃을 구별할 자료.
- Brassard 등 진폭 증폭과 추정, 2000: 진폭 증폭과 QAE의 원 논문.
- Grinko 등 IQAE, 최초 공개 2019, npj Quantum Information 7, 52 (2021): 위상추정 없이 반복하는 진폭 추정.
- Farhi 등 QAOA, 2014: 근사 조합최적화 알고리즘의 원 제안.

확인 범위

각 회차는 실제 영상의 시간정보가 있는 전사 내용을 읽고 발표의 순서와 질문·답변을 재구성했다. 자동자막의 전문용어 오인은 공식 명칭과 문맥으로 바로잡았다. 최신 회차는 로컬 음성인식을 사용했다. 화면으로 직접 확인한 기능 명칭이나 실행 타깃은 본문에 그 범위를 적었다. 발표자가 보고한 외부 성능 수치를 이 작업에서 재현한 측정값으로 표시하지 않는다.

손계산 예제의 목적함수·성공확률·표본오차·메모리·총시간은 독립 계산으로 대조했다. 실제 Classiq 클라우드 합성이나 QPU 작업을 새로 제출하지 않았다. 작은 전수열거는 해당 유한 입력의 해를 확인하며 일반 알고리즘 성능을 평가하지 않는다.

발표 화면과 설명 도식

발표 화면의 원출처는 각 캡션에 연결한 Classiq Technologies 공식 다시보기다. Classiq·NVIDIA 발표자의 코드·화면·출력 중 설명에 필요한 부분을 발췌했다. 브라우저 도구막대와 계정·경로·관련 없는 화면은 제외하고, 원본의 수치·코드·곡선을 그대로 보존했다. 그래프의 축이나 출력값을 다시 그려 실제 화면처럼 표시하지 않았다. 코드와 비교표의 한국어 설명은 본문과 캡션에서 제공한다.

별도 설명 도식은 노트에서 정의한 작은 입력·가정·알고리즘 구조로 제작했다. Grover 반복의 이상적 확률, QAOA의 유효 상태 수, 기대 payoff와 진폭 인코딩은 계산 예이며 실제 QPU 측정값이 아니다. 자원 비교 그림은 전체 기간·하루 단위의 처리 범위와 방향 고정에 따른 자유도 변화를 함께 표시한다.

그림마다 종류·출처·영상 시각 또는 계산 조건을 캡션에 적었다. 최종 선택 목록과 파일 해시는 읽기용 폴더의 `figures/manifest.json`에 남겼다. SVG·PNG에 개인 환경의 경로와 계정 정보를 포함하지 않았는지 검사하고 HTML·PDF에서 가독성과 잘림을 확인했다. 원영상·전체 대본·음성 파일과 검수용 화면 모음은 읽기용 파일에 수록하지 않는다.

작성과 검토

책임 편집자는 김현중이며 AI 지원 시스템은 OpenAI Codex다. 작업자 **root**는 범위 확인, 통합 편집, 보충 수학·용어집·문제, HTML·PDF와 출력 검사를 맡았다. **find_talk_series**는 공식 영상·행사 조사와 1회 전사를, **finance_replay**는 2회의 전사·영상 화면·계산 대조를, **earlier_office_hours**는 3·4회와 질의응답 검토를 맡았다. 정확한 실행 모델 식별자는 별도 실행 기록에 보존하지 않아 추정하지 않는다.

1.1판에서는 **concept_figures**가 계산·개념 도식, **platform_screens**가 새 플랫폼 발표 화면, **finance_screens**가 금융 발표 화면의 선별·출처 기록을 맡았다. root는 본문과 캡션을 연결하고, 원자료와 도식의 의미·수치를 대조하며 읽기용 파일에 통합했다.

적용 편집 원칙은 AI Tech Review Editorial Harness 2026.08이다. 용어·실행 수준·발표와 보충 설명의 경계를 검토하고, 브라우저와 PDF에서 글꼴·수식·표·페이지를 확인했다. 사람의 문장별 감수나 강사의 승인을 마쳤다고 주장하지 않는다. 후속 자막·공식 코드·정정이 나오면 행사일과 판본을 유지하면서 해당 근거를 갱신할 수 있다.